

Network Programming in C/C++ — Complete Topic List

0. Prerequisites

- Pointers, memory management, structs, unions, bit manipulation
- Byte ordering, endianness, alignment, padding
- File descriptors, `errno`, POSIX basics
- Multithreading (`pthread`s, `std::thread`), mutexes, condition variables
- Build tooling: Makefiles/CMake, linking (`-lpthread`, `-lssl`, `-lcrypto`)

1. Networking Theory (enough to write correct code)

- OSI vs TCP/IP model
- Ethernet frames, MAC addressing, ARP
- IPv4/IPv6 headers, subnetting, CIDR, NAT
- ICMP, routing basics, MTU and fragmentation
- TCP: 3-way handshake, sequence/ACK numbers, sliding window, flow control, congestion control (slow start, AIMD), Nagle's algorithm, `TIME_WAIT`, teardown
- UDP: connectionless semantics, when to prefer it
- Ports, well-known vs ephemeral

2. Socket API Fundamentals

- `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()`, `read()/write()`, `close()`, `shutdown()`
- Address structures: `sockaddr`, `sockaddr_in`, `sockaddr_in6`, `sockaddr_storage`, `sockaddr_un`
- Byte-order conversion: `htons`, `htonl`, `ntohs`, `ntohl`
- Address conversion: `inet_pton`, `inet_ntop` (avoid `inet_addr/inet_ntoa`)
- Name resolution: `getaddrinfo`, `freeaddrinfo`, `getnameinfo`, `gai_strerror`
- `getsockname`, `getpeername`
- Error handling, `EINTR`, retry loops

3. TCP Programming

- Iterative echo server and client
- Concurrent server: fork-per-client, thread-per-client, thread pool
- **Stream framing** — the single most important concept: TCP has no message boundaries
 - Length-prefix framing, delimiter framing, fixed-size records
- Partial reads/writes → `send_all` / `recv_all` helpers
- Graceful shutdown, half-close, detecting peer disconnect
- Zombie process reaping (`SIGCHLD`, `waitpid`)
- Handling `SIGPIPE` / `MSG_NOSIGNAL`

4. UDP Programming

- `sendto()`, `recvfrom()`, connected UDP sockets
- Datagram loss, duplication, reordering
- Building reliability on UDP: ACKs, sequence numbers, timeouts, retransmission
- Broadcast (`SO_BROADCAST`) and multicast (`IP_ADD_MEMBERSHIP`, `IP_MULTICAST_TTL`)
- Path MTU considerations, datagram size limits

5. Socket Options & Control

- `setsockopt/getsockopt`
- `SO_REUSEADDR`, `SO_REUSEPORT`, `SO_KEEPALIVE`, `SO_LINGER`
- `SO_RCVBUF`, `SO_SNDBUF`, `SO_RCVTIMEO`, `SO_SNDTIMEO`
- `TCP_NODELAY`, `TCP_CORK`, `TCP_QUICKACK`, TCP keepalive tuning
- `IPV6_V6ONLY` and dual-stack sockets
- `fcntl` with `O_NONBLOCK`, `FD_CLOEXEC`

6. I/O Models & Multiplexing

- Blocking, non-blocking, I/O multiplexing, signal-driven, asynchronous
- `select()` — limits, `FD_SETSIZE`
- `poll()`
- `epoll` (Linux): `epoll_create1`, `epoll_ctl`, `epoll_wait`, level-triggered vs edge-triggered
- `kqueue` (BSD/macOS), IOCP (Windows) for portability awareness
- `io_uring` (modern Linux async I/O)
- Reactor and Proactor patterns
- Event loop design, timer wheels, timeouts
- The C10K / C10M problem

7. Advanced I/O

- Scatter/gather: `readv`, `writew`, `sendmsg`, `recvmsg`
- Zero-copy: `sendfile`, `splice`, `mmap`
- Ancillary data / `cmsg`, passing file descriptors over Unix sockets
- Unix domain sockets (`AF_UNIX`), abstract sockets
- Raw sockets (`SOCK_RAW`), packet sockets (`AF_PACKET`)
- Custom checksum computation, hand-crafting IP/TCP headers
- `libpcap` for packet capture and sniffing

8. Concurrency & Architecture

- Thread pools, work queues, lock-free queues
- Per-core accept with `SO_REUSEPORT`
- Multi-process (pre-fork) vs multi-threaded servers
- Connection state machines, buffer management, ring buffers
- Backpressure, slow-client handling, connection limits
- Graceful shutdown and connection draining

9. Application Protocols (implement at least two)

- HTTP/1.1: request parsing, headers, chunked encoding, keep-alive
- HTTP/2 basics (framing, streams, HPACK), HTTP/3 & QUIC overview
- WebSocket: handshake, frame format, masking, ping/pong
- DNS message format and a resolver client
- SMTP, FTP, Telnet as protocol-design exercises
- MQTT, gRPC/Protobuf, custom binary protocols
- Serialization: Protobuf, FlatBuffers, MessagePack, JSON

10. Security

- TLS/SSL concepts: handshake, certificates, cipher suites, PKI
- OpenSSL API: `SSL_CTX`, `SSL_new`, `SSL_connect`, `SSL_accept`, `SSL_read/write`
- Certificate verification, hostname validation, SNI
- mbedTLS / BoringSSL as alternatives
- Common vulnerabilities: buffer overflows, integer overflow in length fields, format strings, unvalidated input, DoS via slowloris/resource exhaustion
- Secure coding practices for parsers

11. Libraries & Frameworks

- Boost.Asio / standalone Asio (C++)
- libevent, libev, libuv
- POCO, cpp-httplib, Beast (Boost)
- ZeroMQ, nanomsg
- DPDK, netmap, XDP/eBPF for kernel-bypass and high-performance packet processing
- `std::net` proposals / C++ networking TS (status awareness)

12. Performance

- Latency vs throughput, measuring both
- Syscall overhead reduction, batching
- Buffer sizing, memory pools, avoiding allocations in hot paths
- CPU affinity, NUMA awareness, cache-line alignment, false sharing
- Profiling: `perf`, `strace`, `ltrace`, flamegraphs
- Load testing: `wrk`, `ab`, `iperf3`, custom benchmark clients

13. Portability

- Winsock: `WSAStartup`, `SOCKET` type, `closesocket`, differences from POSIX
- Cross-platform abstraction layers
- IPv4/IPv6 agnostic code with `getaddrinfo`

14. Tools & Debugging

- `netstat`, `ss`, `lsof`, `ip`, `ifconfig`, `route`
- `ping`, `traceroute`, `dig`, `nslookup`
- `telnet`, `netcat`, `socat`, `curl`
- Wireshark / `tcpdump` filters

- `iptables/nftables` basics
- `gdb`, Valgrind, ASan/TSan/UBSan for network code
- `tc` for simulating latency/loss

15. Suggested Project Progression

1. TCP echo server + client
2. Multi-client chat server (threads → `select` → `epoll`)
3. Length-prefixed binary protocol with a custom message format
4. File transfer utility over TCP
5. Reliable UDP layer (ACK + retransmit + sequencing)
6. Minimal HTTP server serving static files
7. WebSocket server
8. DNS resolver client
9. HTTP proxy / reverse proxy
10. TLS-enabled server with OpenSSL
11. Port scanner or packet sniffer with raw sockets/libpcap
12. High-performance event-loop server benchmarked against nginx

Recommended reading: Stevens' *UNIX Network Programming, Vol. 1* (the canonical reference), *TCP/IP Illustrated Vol. 1*, and Beej's Guide to Network Programming (free, best starting point).

If you tell me your current level and goal (interview prep, systems job, hobby project), I can compress this into an ordered study plan with time estimates.

Network Programming in C/C++ — Complete Topic List

0. Prerequisites

- Pointers, memory management, structs, unions, bit manipulation
- Byte ordering, endianness, alignment, padding
- File descriptors, `errno`, POSIX basics
- Multithreading (`pthread`s, `std::thread`), mutexes, condition variables
- Build tooling: Makefiles/CMake, linking (`-lpthread`, `-lssl`, `-lcrypto`)

1. Networking Theory (enough to write correct code)

- OSI vs TCP/IP model
- Ethernet frames, MAC addressing, ARP
- IPv4/IPv6 headers, subnetting, CIDR, NAT
- ICMP, routing basics, MTU and fragmentation
- TCP: 3-way handshake, sequence/ACK numbers, sliding window, flow control, congestion control (slow start, AIMD), Nagle's algorithm, `TIME_WAIT`, teardown
- UDP: connectionless semantics, when to prefer it
- Ports, well-known vs ephemeral

Networking Theory — What You Actually Need to Write Correct Code

Framing rule: theory matters here only where it changes what your code must do. I've marked those spots with → **Code impact**.

1. OSI vs TCP/IP Model

OSI (7 layers): Physical, Data Link, Network, Transport, Session, Presentation, Application. **TCP/IP (4 layers, what's actually implemented):** Link, Internet, Transport, Application.

OSI is vocabulary, not architecture. Nobody implements a "session layer." What you need is the mapping:

Layer	Unit	Addressing	Your API
Link	Frame	MAC (48-bit)	AF_PACKET, libpcap
Internet	Packet	IP address	SOCK_RAW
Transport	Segment (TCP) / Datagram (UDP)	Port	SOCK_STREAM, SOCK_DGRAM
Application	Message	URL, name	your protocol

Encapsulation is the concept that matters: each layer prepends a header to the payload from above. A 100-byte HTTP request on Ethernet becomes 14 (Eth) + 20 (IP) + 20 (TCP) + 100 = 154 bytes on the wire. That ~54-byte overhead per packet is why chatty protocols are slow.

→ **Code impact:** When you write `send(fd, buf, 100, 0)`, you are handing 100 bytes to the transport layer. Everything below is the kernel's job — *unless* you use raw sockets, at which point you build headers yourself and compute checksums by hand.

2. Ethernet, MAC, ARP

Ethernet II frame:

Dest MAC (6) Src MAC (6) EtherType (2) Payload (46-1500) FCS (4)
--

EtherType: 0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP.

MAC addresses are **link-local only**. They're rewritten at every router hop; IP addresses are not. This is the single most important fact about layer 2 vs layer 3.

ARP resolves "I have IP 192.168.1.5, what's its MAC?" via broadcast on the local segment. The reply is cached (`ip neigh` on Linux). ARP only works within a broadcast domain — for off-subnet destinations, the host ARPs for the *default gateway's* MAC instead.

→ **Code impact:** Essentially zero for normal socket programming — the kernel handles it. It matters when: writing packet sniffers, building raw-socket tools, debugging "why can't these two containers see each other," or explaining ARP spoofing in a security context.

3. IPv4 / IPv6 Headers

IPv4 header (20 bytes minimum):

Field	Size	Notes
Version / IHL	4b / 4b	IHL in 32-bit words; 5 = no options
TOS / DSCP	8b	QoS marking
Total Length	16b	header + payload, max 65535
Identification	16b	fragment reassembly key
Flags / Fragment Offset	3b / 13b	DF, MF bits
TTL	8b	decremented per hop; 0 → drop + ICMP
Protocol	8b	6=TCP, 17=UDP, 1=ICMP
Header Checksum	16b	header only, recomputed per hop
Source / Dest Address	32b each	

IPv6 header — fixed 40 bytes, no fragmentation by routers, no checksum: Version, Traffic Class, Flow Label, Payload Length, Next Header, Hop Limit, 128-bit src/dst. Options moved to *extension headers* chained via Next Header. Simpler to parse, faster to route.

Subnetting & CIDR. **192.168.1.0/24** means the first 24 bits are network, last 8 are host. Mask = **255.255.255.0**. Usable hosts = $2^{(32-\text{prefix})} - 2$ (network address + broadcast reserved).

The one operation you'll actually code:

```
bool same_subnet(uint32_t a, uint32_t b, int prefix) {
    uint32_t mask = prefix ? htonl(~0u << (32 - prefix)) : 0;
    return (a & mask) == (b & mask);
}
```

Private ranges: **10.0.0.0/8**, **172.16.0.0/12**, **192.168.0.0/16**, plus **127.0.0.0/8** loopback and **169.254.0.0/16** link-local.

NAT rewrites source IP:port on the way out and maintains a translation table to reverse it on the way back. Consequences you must design around:

- Inbound connections to a NATed host fail without port forwarding.
- NAT mappings **expire** (often 30s–5min for UDP, longer for TCP) → long-lived idle connections die silently. This is *the* reason for application-level keepalives.

- P2P requires hole punching (STUN/TURN/ICE).

→ **Code impact:** `getaddrinfo` gives you an address family you must not assume. Write `sockaddr_storage`-based code and iterate the `addrinfo` list. Never `strcpy` an address into a `char[16]` — that's IPv4-only thinking and it breaks the moment you touch IPv6 (need `INET6_ADDRSTRLEN` = 46).

4. ICMP, Routing, MTU, Fragmentation

ICMP is the control channel: Echo Request/Reply (ping), Destination Unreachable, Time Exceeded (drives traceroute via incrementing TTL), and critically **Fragmentation Needed (Type 3, Code 4)**, which carries Path MTU Discovery.

Routing on a host is simple: longest-prefix match against the routing table. Destination on-link → send directly; otherwise → default gateway. Routers do the same at scale with BGP/OSPF-learned routes.

MTU is the largest payload a link can carry — 1500 bytes on standard Ethernet, less over tunnels (PPPoE ~1492, IPsec/VPN often 1400ish), up to 9000 with jumbo frames.

MSS = MTU – IP header – TCP header = 1500 – 20 – 20 = **1460** for IPv4 Ethernet. Negotiated in SYN options.

Fragmentation: IPv4 routers may split oversized packets; IPv6 never does — it drops and returns ICMPv6 Packet Too Big. Fragmentation is bad: losing one fragment loses the whole packet, and reassembly costs memory.

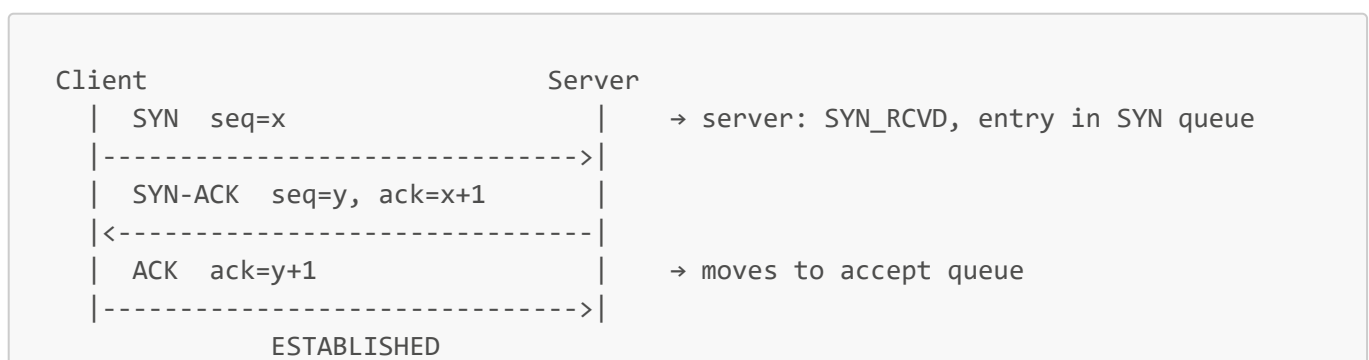
PMTU black holes are a classic production bug: a firewall drops all ICMP, so the sender never learns to reduce packet size, and the connection hangs after the handshake succeeds. Small packets work, large ones vanish.

→ **Code impact:**

- **UDP:** keep datagrams under ~1400 bytes to avoid fragmentation entirely. DNS chose 512 for this reason. This is a real design constraint on any custom UDP protocol.
- **TCP:** the kernel handles segmentation; you never care about MTU. But if a connection completes the handshake then stalls on the first big transfer, suspect PMTU.
- Setting `IP_MTU_DISCOVER/DF` matters only for raw or specialized UDP work.

5. TCP — The Part Worth Real Study

Three-Way Handshake



Two queues exist on the server: the SYN (incomplete) queue and the accept (completed) queue. `listen(fd, backlog)` sizes the accept queue. When it overflows, the kernel drops or resets connections — clients see timeouts or `ECONNREFUSED` under load. **SYN cookies** (`net.ipv4.tcp_syncookies`) defend the SYN queue against flood attacks.

→ **Code impact:** `connect()` blocks for one RTT. `accept()` pops from a queue that was filled *before* you called it — the connection already exists whether or not you're ready. A too-small backlog is a real production failure mode; 128–1024 is typical for servers.

Sequence and ACK Numbers

Sequence numbers count **bytes**, not packets, and start at a random ISN. `ack = next byte expected`. SYN and FIN each consume one sequence number.

Duplicate ACKs signal a gap; three of them trigger **fast retransmit** without waiting for the RTO. **SACK** lets the receiver report non-contiguous received ranges so the sender retransmits only what's missing.

Sliding Window & Flow Control

The receiver advertises a window (`rwnd`) = free space in its receive buffer. The sender may have at most `rwnd` unacknowledged bytes outstanding. If the app stops calling `recv()`, the buffer fills, the window shrinks to zero, and the sender stops. That's **flow control** — protecting a slow receiver, and it's automatic backpressure you get for free.

Window scaling (a SYN option) allows windows beyond 64 KB, required for high bandwidth-delay-product links.

→ **Code impact:** This is your backpressure mechanism. A slow client naturally throttles a server that respects `send()` return values. It also means a server that doesn't drain its receive buffer stalls the peer — which is why one blocked handler can wedge a connection.

Congestion Control

Separate from flow control: `rwnd` protects the receiver, `cwnd` protects the *network*. Sender is limited by `min(rwnd, cwnd)`.

- **Slow start:** `cwnd` starts ~10 MSS, doubles each RTT (exponential) until `ssthresh` or loss.
- **Congestion avoidance (AIMD):** +1 MSS per RTT (additive increase); on loss, halve `cwnd` (multiplicative decrease). The sawtooth pattern.
- **Fast recovery:** avoid dropping back to slow start on isolated loss.
- **Modern algorithms:** CUBIC (Linux default), **BBR** (models bandwidth and RTT instead of treating loss as congestion — significantly better on lossy or buffer-bloated paths).

→ **Code impact:** New connections start slow. This is the entire argument for connection reuse — HTTP keep-alive, connection pools, HTTP/2 multiplexing. Opening a fresh TCP connection per request pays handshake RTT *plus* slow-start ramp every time.

Nagle's Algorithm

Buffers small writes until either a full MSS accumulates or all outstanding data is ACKed. It prevents "tinygram" flooding (one byte of payload per 40 bytes of header).

Pathological interaction with **delayed ACK** (receiver waits up to ~40–200 ms before ACKing): sender waits for an ACK, receiver waits for more data. You get 40 ms stalls on request/response workloads.

→ **Code impact:**

```
int flag = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));
```

Disable Nagle for latency-sensitive request/response protocols (RPC, games, interactive). Better still: **make one write() per logical message** — build the full buffer, or use `writev()`. Header-then-body in two separate writes is the classic mistake that Nagle punishes.

Teardown and TIME_WAIT

```
FIN → ACK → FIN → ACK    (four-way, each direction closes independently)
```

The side that closes first enters **TIME_WAIT** for $2 \times \text{MSL}$ (60 s on Linux). Two reasons: absorb delayed duplicate segments from the old connection, and ensure the final ACK reaches the peer if it's lost.

→ **Code impact:**

- A server closing many connections accumulates thousands of TIME_WAIT sockets, exhausting ephemeral ports.
- **SO_REUSEADDR is mandatory on servers** — without it, restarting a server fails with `EADDRINUSE` because the listening address is still in TIME_WAIT.
- `shutdown(fd, SHUT_WR)` sends FIN but keeps reading — this is **half-close**, essential for protocols where you signal "done sending" then read the response.
- Prefer having the *client* close first when you control the protocol.
- `close()` on a socket with unread data sends RST, not FIN.

6. UDP — Connectionless Semantics

No handshake, no ordering, no retransmission, no flow control, no congestion control. 8-byte header: source port, dest port, length, checksum. **Message boundaries are preserved** — one `sendto()` = one `recvfrom()`.

What you must accept:

- Datagrams can be **lost, duplicated, reordered**
- A short read **truncates and discards** the remainder of the datagram (unlike TCP)
- No backpressure — you can trivially overrun a receiver or the network
- Zero congestion awareness means an unthrottled UDP flood is antisocial

Prefer UDP when:

- Latency beats reliability: voice, video, gaming state updates (a stale packet is worthless — retransmitting it is *harmful*)
- Request/response fits one datagram: DNS, NTP, DHCP
- Multicast/broadcast (TCP can't do it)
- You need to build custom reliability semantics — partial reliability, unordered delivery, or head-of-line-blocking avoidance (this is exactly why QUIC is built on UDP)

Prefer TCP for bulk transfer, anything where every byte must arrive in order, and anything where you don't want to reimplement 40 years of congestion control badly.

→ **Code impact:** "Connected" UDP via `connect()` on a `SOCK_DGRAM` socket is worth knowing — it lets you use `send()/recv()`, filters incoming datagrams to that peer, and lets you receive ICMP errors (unconnected UDP sockets silently swallow `ECONNREFUSED`).

7. Ports

16-bit, so 0–65535, and **the namespace is per-protocol** — TCP/80 and UDP/80 are unrelated.

- **0–1023 well-known:** binding requires root or `CAP_NET_BIND_SERVICE` on Linux
- **1024–49151 registered**
- **49152–65535 dynamic/ephemeral** (Linux actually uses `net.ipv4.ip_local_port_range`, default 32768–60999)

A TCP connection is identified by the **4-tuple** (`src IP`, `src port`, `dst IP`, `dst port`). This is why one server port serves millions of clients, and why a *client* is limited to ~28k concurrent connections to a single destination IP:port before ephemeral port exhaustion — mitigated by more source IPs or more destination ports.

`bind()` to port 0 = "kernel, pick one." Clients normally skip `bind()` entirely; `connect()` implicitly binds.

→ **Code impact:** `SO_REUSEPORT` (Linux 3.9+) lets multiple sockets bind the same port with kernel-level load balancing across them — the modern way to scale accept across cores without a single-accepter bottleneck.

What to Actually Verify Before Moving On

You should be able to answer these without looking:

1. Why does a server need `SO_REUSEADDR` but a client generally doesn't?
2. Your `recv()` returns 500 bytes when you sent 1000. What happened, and what does that imply about your protocol design?
3. Why does splitting a header and body into two `write()` calls cause a 40 ms delay?
4. `recv()` returns 0 vs returns -1 with `EAGAIN` — what does each mean?
5. A connection handshakes fine but hangs on the first large transfer. First hypothesis?
6. Why can't you reliably send a 4 KB UDP datagram across the internet?
7. Why is opening a new TCP connection per HTTP request slow, for two separate reasons?

If any of those are shaky, that's the sub-topic to reread — not the whole section.

Best practical exercise for this section: run `tcpdump -i any -nn 'port 8080'` (or Wireshark) against your own echo server. Watch the SYN/SYN-ACK/ACK, watch the sequence numbers advance, kill the client and watch FIN/TIME_WAIT. Twenty minutes of that teaches more than any diagram.

Ready for **Socket API Fundamentals** whenever you are.

2. Socket API Fundamentals

- `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()`, `read()/write()`, `close()`, `shutdown()`
- Address structures: `sockaddr`, `sockaddr_in`, `sockaddr_in6`, `sockaddr_storage`, `sockaddr_un`
- Byte-order conversion: `htons`, `htonl`, `ntohs`, `ntohl`
- Address conversion: `inet_pton`, `inet_ntop` (avoid `inet_addr/inet_ntoa`)
- Name resolution: `getaddrinfo`, `freeaddrinfo`, `getnameinfo`, `gai_strerror`
- `getsockname`, `getpeername`
- Error handling, `EINTR`, retry loops

Socket API Fundamentals

This is where theory becomes code. Everything here is muscle memory you'll use in every program you write.

1. The Call Sequence

SERVER		CLIENT
<code>socket()</code>		<code>socket()</code>
<code>bind()</code>		
<code>listen()</code>		
<code>accept()</code>	<----- handshake -----	<code>connect()</code>
<code>recv()/send()</code>	<-- data -->	<code>send()/recv()</code>
<code>close()</code>		<code>close()</code>

Mental model: a **socket is a file descriptor with an address and a protocol**. `bind()` assigns the local address, `connect()` assigns the remote address, `listen()` converts an ordinary socket into a passive one that manufactures new sockets via `accept()`.

2. The Core Calls

`socket()`

```
int socket(int domain, int type, int protocol);
```

Arg	Common values
domain	<code>AF_INET</code> , <code>AF_INET6</code> , <code>AF_UNIX</code>
type	<code>SOCK_STREAM</code> (TCP), <code>SOCK_DGRAM</code> (UDP), <code>SOCK_RAW</code>
protocol	<code>0</code> (infer from type), or <code>IPPROTO_TCP</code> / <code>IPPROTO_UDP</code>

Returns fd, or `-1` with `errno`. On Linux, OR the type with `SOCK_CLOEXEC` and/or `SOCK_NONBLOCK` to avoid a race between `socket()` and a later `fcntl()`:

```
int fd = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
```

bind()

```
int bind(int fd, const struct sockaddr *addr, socklen_t addrlen);
```

Assigns a local address. Servers always call it. Clients almost never do — `connect()` binds implicitly to an ephemeral port.

Always set `SO_REUSEADDR` before `bind()` on a server, or restarts fail with `EADDRINUSE` while old connections sit in `TIME_WAIT`:

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
```

Bind to `INADDR_ANY` / `in6addr_any` to accept on all interfaces; bind to a specific address to restrict.

listen()

```
int listen(int fd, int backlog);
```

`backlog` sizes the completed-connection (accept) queue. On Linux it's capped by `net.core.somaxconn`. Use 128–1024 for real servers; `SOMAXCONN` is a portable choice. Too small = dropped connections under burst load.

accept()

```
int accept(int fd, struct sockaddr *addr, socklen_t *addrlen);
int accept4(int fd, struct sockaddr *addr, socklen_t *addrlen, int flags); //
Linux
```

Returns a **new** fd for the connection; the listening fd stays open. `addrlen` is value-result: set it to the buffer size going in, read the actual size coming out. Pass `NULL`, `NULL` if you don't need the peer address.

```
struct sockaddr_storage peer;
socklen_t len = sizeof peer;
int cfd = accept(lfd, (struct sockaddr *)&peer, &len);
```

Pitfall: on a blocking listener, `accept()` can still fail with `ECONNABORTED` (client vanished between handshake and accept) — retry, don't die. Under `epoll` edge-triggered, loop `accept()` until `EAGAIN`.

connect()

```
int connect(int fd, const struct sockaddr *addr, socklen_t addrlen);
```

Blocking: returns after the handshake completes (one RTT) or fails — `ECONNREFUSED` (RST received), `ETIMEDOUT`, `EHOSTUNREACH`, `ENETUNREACH`.

Non-blocking: returns `-1` with `EINPROGRESS`. Wait for writability, then check the real result:

```
int err; socklen_t elen = sizeof err;
getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &elen);
if (err) { /* connect failed with errno == err */ }
```

That `SO_ERROR` check is the standard way to implement connect timeouts.

On UDP, `connect()` doesn't send anything — it just fixes the default peer, enabling `send()/recv()` and ICMP error delivery.

send() / recv() vs read() / write()

```
ssize_t send(int fd, const void *buf, size_t len, int flags);
ssize_t recv(int fd, void *buf, size_t len, int flags);
```

Identical to `read/write` when `flags == 0`. Use `send/recv` for the flags:

Flag	Meaning
<code>MSG_NOSIGNAL</code>	Return <code>EPIPE</code> instead of raising <code>SIGPIPE</code> — use this on every send
<code>MSG_DONTWAIT</code>	One-shot non-blocking
<code>MSG_PEEK</code>	Read without consuming
<code>MSG_WAITALL</code>	Block until full <code>len</code> received (still interruptible)

The rule that governs all TCP code: `send()` and `recv()` return the number of bytes *actually* transferred, which may be less than requested. Short transfers are normal, not errors.

Return-value semantics for `recv()` on TCP:

- `> 0` — bytes received
- `== 0` — peer performed an orderly shutdown (FIN). **This is EOF, not an error.**
- `< 0` — error; check `errno` (`EAGAIN/EWOULDBLOCK` = no data on non-blocking socket, `EINTR` = signal)

`close()` vs `shutdown()`

```
int close(int fd);           // drops YOUR reference to the fd
int shutdown(int fd, int how); // affects the CONNECTION, all references
```

how: `SHUT_RD`, `SHUT_WR`, `SHUT_RDWR`.

`close()` decrements a refcount — after `fork()` or `dup()`, the connection stays alive until the last fd closes. `shutdown(fd, SHUT_WR)` sends FIN immediately regardless of refcount, and is how you do a **half-close**: "I'm done sending, but I'll keep reading your response." That's the correct pattern for request/response protocols where the request end is signalled by EOF.

Pitfall: calling `close()` while unread data sits in the receive buffer sends an RST instead of a clean FIN, and the peer may lose data you already sent. Drain before closing.

3. Address Structures

C has no inheritance, so the socket API fakes polymorphism: every concrete address type begins with a family field, and you cast to `struct sockaddr *` at the API boundary.

```
struct sockaddr {           // generic – never instantiate, only cast to
    sa_family_t sa_family;
    char        sa_data[14];
};

struct sockaddr_in {        // IPv4, 16 bytes
    sa_family_t sin_family;  // AF_INET
    in_port_t   sin_port;    // network byte order!
    struct in_addr sin_addr;  // network byte order!
    char        sin_zero[8];
};

struct sockaddr_in6 {       // IPv6, 28 bytes
    sa_family_t sin6_family; // AF_INET6
    in_port_t   sin6_port;
    uint32_t     sin6_flowinfo;
    struct in6_addr sin6_addr;
    uint32_t     sin6_scope_id; // link-local interface index
};
```

```

struct sockaddr_un {           // Unix domain
    sa_family_t sa_family;     // AF_UNIX
    char        sun_path[108]; // filesystem path
};

struct sockaddr_storage;      // big enough + correctly aligned for ANY of
them

```

Rules:

1. **Always zero the struct before filling it** (`memset` or `= {0}`) — padding and `sin_zero` must be clean.
2. **Always declare receive buffers as `sockaddr_storage`**. Using `sockaddr_in` for `accept()` or `recvfrom()` silently truncates IPv6 addresses. Using bare `sockaddr` is worse — it's only 16 bytes.
3. Cast only at the call: `(struct sockaddr *)&ss`.
4. Ports and addresses inside these structs are **always network byte order**.

Extracting the port generically:

```

uint16_t port_of(const struct sockaddr_storage *ss) {
    if (ss->ss_family == AF_INET)
        return ntohs(((const struct sockaddr_in *)ss)->sin_port);
    if (ss->ss_family == AF_INET6)
        return ntohs(((const struct sockaddr_in6 *)ss)->sin6_port);
    return 0;
}

```

For `sockaddr_un`, note `sun_path` is a fixed 108 bytes and the socket file persists after close — `unlink()` it before `bind()`. Linux also supports abstract sockets (`sun_path[0] == '\0'`), which live in a namespace with no filesystem entry.

4. Byte Order

Network byte order is **big-endian**. x86/ARM are little-endian. Converting is mandatory for anything crossing the wire.

```

uint16_t htons(uint16_t);  uint16_t ntohs(uint16_t);  // 16-bit, "short"
uint32_t htonl(uint32_t);  uint32_t ntohl(uint32_t);  // 32-bit, "long"

```

On big-endian machines these are no-ops — which is exactly why you call them unconditionally rather than testing endianness yourself.

Applies to: `sin_port`, `sin_addr`, and every multi-byte integer in *your own* protocol headers. **Does not apply to:** byte arrays, strings, or single bytes — those have no ordering.

No standard `htonll` exists. For 64-bit values use `htobe64/be64toh` (`<endian.h>`, glibc), or serialize byte-by-byte for full portability:

```
void put_u32(uint8_t *p, uint32_t v) {
    p[0] = v >> 24; p[1] = v >> 16; p[2] = v >> 8; p[3] = v;
}
uint32_t get_u32(const uint8_t *p) {
    return ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) |
           ((uint32_t)p[2] << 8) | (uint32_t)p[3];
}
```

That version is immune to both endianness *and* alignment problems, which is why serious protocol code tends to look like this rather than casting structs onto buffers.

5. Address ↔ String Conversion

```
int inet_pton(int af, const char *src, void *dst);           // presentation →
network
const char *inet_ntop(int af, const void *src,
                      char *dst, socklen_t size);          // network →
presentation
```

`inet_pton` returns 1 on success, **0 if the string is malformed** (not -1!), -1 on bad family. That 0 case is the one people forget to check.

```
struct sockaddr_in sa = {0};
sa.sin_family = AF_INET;
sa.sin_port = htons(8080);
if (inet_pton(AF_INET, "192.168.1.10", &sa.sin_addr) != 1) { /* invalid */ }

char buf[INET6_ADDRSTRLEN]; // 46 – always size with this, never 16
inet_ntop(AF_INET, &sa.sin_addr, buf, sizeof buf);
```

Avoid `inet_addr()` — IPv4 only, and it returns `INADDR_NONE` (0xFFFFFFFF) for errors, which is indistinguishable from the valid address 255.255.255.255. **Avoid `inet_ntoa()`** — IPv4 only and returns a pointer to a static buffer, so it's not reentrant and two calls in one `printf` clobber each other.

6. Name Resolution — `getaddrinfo`

This is the modern, protocol-agnostic entry point. It replaces `gethostbyname`, `getservbyname`, manual `sockaddr` construction, and hardcoded address families all at once. It handles DNS, `/etc/hosts`, service-name lookup, and literal IPv4/IPv6 strings.


```

struct addrinfo {
    int     ai_flags, ai_family, ai_socktype, ai_protocol;
    socklen_t ai_addrlen;
    struct sockaddr *ai_addr;
    char     *ai_canonname;
    struct addrinfo *ai_next;    // linked list – try each in order
};

int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
void freeaddrinfo(struct addrinfo *res);
const char *gai_strerror(int errcode);

```

It returns its own error codes (`EAI_*`), **not** `errno` — use `gai_strerror`, not `perror`.

Useful hint flags: `AI_PASSIVE` (for servers — yields wildcard address when `node` is `NULL`), `AI_NUMERICHOST` (skip DNS), `AI_NUMERICSERV`, `AI_ADDRCONFIG` (only return families the host actually has configured).

Idiomatic client — works for IPv4 and IPv6 with zero changes

```

int tcp_connect(const char *host, const char *port) {
    struct addrinfo hints = {0}, *res, *rp;
    hints.ai_family      = AF_UNSPEC;    // v4 or v6, whatever DNS gives
    hints.ai_socktype    = SOCK_STREAM;

    int rc = getaddrinfo(host, port, &hints, &res);
    if (rc != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rc));
        return -1;
    }

    int fd = -1;
    for (rp = res; rp; rp = rp->ai_next) {
        fd = socket(rp->ai_family, rp->ai_socktype | SOCK_CLOEXEC, rp-
>ai_protocol);
        if (fd < 0) continue;
        if (connect(fd, rp->ai_addr, rp->ai_addrlen) == 0) break; // success
        close(fd);
        fd = -1;
    }
    freeaddrinfo(res);
    return fd;    // -1 if every candidate failed
}

```

The loop is the point: DNS may return several addresses and some may be unreachable. Code that uses only `res` and ignores `ai_next` breaks on dual-stack hosts.

Idiomatic server

```

int tcp_listen(const char *port, int backlog) {
    struct addrinfo hints = {0}, *res, *rp;
    hints.ai_family    = AF_INET6;        // dual-stack: v6 socket accepts v4 too
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags     = AI_PASSIVE;

    int rc = getaddrinfo(NULL, port, &hints, &res);
    if (rc != 0) { fprintf(stderr, "%s\n", gai_strerror(rc)); return -1; }

    int fd = -1;
    for (rp = res; rp; rp = rp->ai_next) {
        fd = socket(rp->ai_family, rp->ai_socktype | SOCK_CLOEXEC, rp-
>ai_protocol);
        if (fd < 0) continue;

        int yes = 1;
        setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

        int no = 0;    // accept both IPv6 and IPv4-mapped addresses
        setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no);

        if (bind(fd, rp->ai_addr, rp->ai_addrlen) == 0 &&
            listen(fd, backlog) == 0) break;
        close(fd);
        fd = -1;
    }
    freeaddrinfo(res);
    return fd;
}

```

Important caveat: `getaddrinfo` is **blocking** and can stall for seconds on slow DNS. In an event-loop server, never call it on the event thread — offload to a thread pool or use an async resolver (c-ares, or `getaddrinfo_a`).

`getnameinfo` — the reverse

```

char host[NI_MAXHOST], serv[NI_MAXSERV];
getnameinfo((struct sockaddr *)&ss, len, host, sizeof host,
            serv, sizeof serv, NI_NUMERICHOST | NI_NUMERICSERV);

```

Those two flags give you "print the address and port as numbers" — the safe default for logging. Without them it performs reverse DNS, which is slow and attacker-influenced. Never log an unvalidated reverse-DNS name as if it were trustworthy.

7. `getsockname` / `getpeername`

```
int getsockname(int fd, struct sockaddr *addr, socklen_t *len); // local end
int getpeername(int fd, struct sockaddr *addr, socklen_t *len); // remote end
```

Uses:

- Discover the ephemeral port after binding to port 0
- Learn which local interface `connect()` chose
- Recover the peer address when you didn't save it from `accept()`
- Distinguish connected from unconnected (`getpeername` fails with `ENOTCONN`)

Both are value-result on `len`, same as `accept`.

8. Error Handling, `EINTR`, and Retry Loops

Every socket call returns `-1` on failure and sets `errno`. Errors you must handle by name:

errno	Meaning	Response
<code>EINTR</code>	Interrupted by a signal	Retry — not a real error
<code>EAGAIN/EOULDBLOCK</code>	Non-blocking, nothing ready	Wait for readiness, retry
<code>EPIPE</code>	Wrote to a closed connection	Connection is dead, clean up
<code>ECONNRESET</code>	Peer sent RST	Connection is dead, clean up
<code>ECONNREFUSED</code>	Nothing listening	Report to caller
<code>ETIMEDOUT</code>	No response	Report to caller
<code>EADDRINUSE</code>	Port taken / <code>TIME_WAIT</code>	Set <code>SO_REUSEADDR</code> , or pick another port
<code>EMFILE/ENFILE</code>	fd limit hit	Shed load — don't spin; a busy-loop here is a real outage pattern

`errno` is only meaningful immediately after a failure. Save it before calling anything else (including `printf`), because intervening library calls can overwrite it.

`SIGPIPE`

Writing to a socket whose peer has closed raises `SIGPIPE`, whose default action **kills your process**. Two fixes, use both:

```
signal(SIGPIPE, SIG_IGN); // once at startup
...
send(fd, buf, len, MSG_NOSIGNAL); // per-call (portable to macOS:
SO_NOSIGPIPE)
```

The two helpers you will write in every project

TCP does not preserve message boundaries and does not guarantee full transfers. These wrappers are non-optional:

```
// Send exactly n bytes, or fail.
ssize_t send_all(int fd, const void *buf, size_t n) {
    const char *p = buf;
    size_t sent = 0;
    while (sent < n) {
        ssize_t r = send(fd, p + sent, n - sent, MSG_NOSIGNAL);
        if (r > 0) { sent += (size_t)r; continue; }
        if (r < 0 && errno == EINTR) continue;           // retry
        return -1;                                       // real error
    }
    return (ssize_t)sent;
}

// Receive exactly n bytes. Returns n, or 0 on clean EOF, or -1 on error.
ssize_t recv_all(int fd, void *buf, size_t n) {
    char *p = buf;
    size_t got = 0;
    while (got < n) {
        ssize_t r = recv(fd, p + got, n - got, 0);
        if (r > 0) { got += (size_t)r; continue; }
        if (r == 0) return 0;                           // peer closed early
        if (errno == EINTR) continue;
        return -1;
    }
    return (ssize_t)got;
}
```

(On a non-blocking socket these change shape entirely — you return partial progress to the event loop rather than looping. Same principle, different control flow.)

A general retry macro

```
#define RETRY_EINTR(expr) ({
    long _r;
    do { _r = (long)(expr); }
    while (_r == -1 && errno == EINTR);
    _r; })
```

Pitfall Checklist

Print this next to your keyboard:

1. Forgetting `htons()` on the port → connecting to a random port

2. Assuming `send()/recv()` transfer everything → truncated messages under load
3. Treating `recv() == 0` as an error instead of EOF
4. Not handling `EINTR` → spurious failures whenever a signal arrives
5. No `SIGPIPE` handling → process dies when a client disconnects
6. Missing `SO_REUSEADDR` → server won't restart for 60 seconds
7. Not zeroing `sockaddr_in` → garbage in padding
8. Using `sockaddr_in` where `sockaddr_storage` belongs → IPv6 truncation
9. Ignoring `ai_next` in `getaddrinfo` → fails on dual-stack hosts
10. Passing the wrong `addrlen`, or forgetting it's value-result
11. Sizing a text buffer at 16 instead of `INET6_ADDRSTRLEN`
12. Using `perror` for `getaddrinfo` errors instead of `gai_strerror`
13. Leaking fds by not closing on the error paths inside the connect loop

Exercise

Write `tcp_connect()` and `tcp_listen()` as above into a small `net.c` / `net.h`, plus `send_all/recv_all`, and build an echo server and client on top. Then verify:

- Client connects to `localhost`, `127.0.0.1`, and `:::1` with no code change
- Server survives `Ctrl-C` on the client (no `SIGPIPE` death)
- Server restarts immediately after being killed (`SO_REUSEADDR` works)
- `getsockname()` on the client prints the ephemeral port your kernel picked
- Sending 1 MB in a single `send_all()` arrives complete — instrument it to print how many `send()` calls it actually took, and watch it be more than one

That last check is the moment TCP's stream semantics stop being abstract. Keep `net.c` — sections 3 through 9 all build on it.

Ready for **TCP Programming** (framing, concurrency models, graceful shutdown) when you are.

3. TCP Programming

- Iterative echo server and client
- Concurrent server: fork-per-client, thread-per-client, thread pool
- **Stream framing** — the single most important concept: TCP has no message boundaries
 - Length-prefix framing, delimiter framing, fixed-size records
- Partial reads/writes → `send_all` / `recv_all` helpers
- Graceful shutdown, half-close, detecting peer disconnect
- Zombie process reaping (`SIGCHLD`, `waitpid`)
- Handling `SIGPIPE` / `MSG_NOSIGNAL`

TCP Programming

This section is where most people's mental model breaks. The core lesson: **TCP is a byte stream, not a message channel**. Everything else follows from that.

1. Iterative Echo Server and Client

The simplest possible server. One client at a time, fully sequential.

```
// echo_server_iterative.c
#include "net.h" // tcp_listen, send_all, recv_all from section 2

int main(int argc, char **argv) {
    signal(SIGPIPE, SIG_IGN);

    int lfd = tcp_listen(argc > 1 ? argv[1] : "8080", SOMAXCONN);
    if (lfd < 0) return 1;

    for (;;) {
        struct sockaddr_storage peer;
        socklen_t plen = sizeof peer;

        int cfd = accept(lfd, (struct sockaddr *)&peer, &plen);
        if (cfd < 0) {
            if (errno == EINTR || errno == ECONNABORTED) continue;
            perror("accept");
            break;
        }

        char host[NI_MAXHOST], serv[NI_MAXSERV];
        getnameinfo((struct sockaddr *)&peer, plen, host, sizeof host,
                    serv, sizeof serv, NI_NUMERICHOST | NI_NUMERICSERV);
        fprintf(stderr, "connected: [%s]:%s\n", host, serv);

        // --- handle one client to completion ---
        char buf[4096];
        for (;;) {
            ssize_t n = recv(cfd, buf, sizeof buf, 0);
            if (n == 0) break; // clean EOF
            if (n < 0) {
                if (errno == EINTR) continue;
                perror("recv");
                break;
            }
            if (send_all(cfd, buf, (size_t)n) < 0) { perror("send"); break; }
        }
        close(cfd);
        fprintf(stderr, "disconnected: [%s]:%s\n", host, serv);
    }
    close(lfd);
}
```

Client:

```
// echo_client.c
int main(int argc, char **argv) {
    signal(SIGPIPE, SIG_IGN);
    int fd = tcp_connect(argv[1], argv[2]);
    if (fd < 0) return 1;

    char line[4096], reply[4096];
    while (fgets(line, sizeof line, stdin)) {
        size_t len = strlen(line);
        if (send_all(fd, line, len) < 0) break;

        ssize_t n = recv(fd, reply, sizeof reply, 0);
        if (n <= 0) break;
        fwrite(reply, 1, (size_t)n, stdout);
    }
    shutdown(fd, SHUT_WR); // signal "no more requests"
    close(fd);
}
```

What's wrong with the iterative server: a second client sits in the accept queue until the first disconnects. Fine for learning, useless for anything real. Note that it *appears* to work with two clients — because the backlog silently queues them — which makes this a deceptive bug in production code.

Also note: this echo server ignores message boundaries entirely, which is why it works. It doesn't care where a "message" starts or ends. The moment your protocol has structure, that stops being true. See section 3.

2. Concurrent Servers

2a. Fork per client

```
static void reap_children(int sig) {
    (void)sig;
    int saved = errno; // handlers must preserve errno
    while (waitpid(-1, NULL, WNOHANG) > 0) // loop: signals coalesce
        ;
    errno = saved;
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = reap_children;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
    sigaction(SIGCHLD, &sa, NULL);
    signal(SIGPIPE, SIG_IGN);

    int lfd = tcp_listen("8080", SOMAXCONN);

    for (;;) {
```

```

    int cfd = accept(lfd, NULL, NULL);
    if (cfd < 0) {
        if (errno == EINTR || errno == ECONNABORTED) continue;
        break;
    }

    pid_t pid = fork();
    if (pid == 0) {          // child
        close(lfd);          // child doesn't need the listener
        handle_client(cfd);
        close(cfd);
        _exit(0);            // _exit, not exit – skip parent's atexit/stdio
flush
    }
    close(cfd);              // parent doesn't need the connection
    if (pid < 0) perror("fork");
}
}

```

The two `close()` calls are mandatory. fds are refcounted across `fork()`. If the parent doesn't close `cfd`, the connection never fully closes when the child exits — you leak fds until `EMFILE`. If the child doesn't close `lfd`, a running child keeps the listening port alive after the parent dies.

Zombie reaping. A terminated child stays in the process table until reaped. Without a `SIGCHLD` handler you accumulate zombies until the process table fills.

Two critical details in that handler:

- **while (`waitpid(...)` > 0), not a single call.** Standard signals don't queue. If three children exit while the handler is running, you get *one* `SIGCHLD` — a single `waitpid` reaps one and leaves two zombies forever.
- **`WNOHANG`** so the handler doesn't block when there's nothing left.

The lazy alternative — `signal(SIGCHLD, SIG_IGN)` — genuinely works on Linux/POSIX and auto-reaps, but you lose exit statuses. Fine for a pure echo server, wrong if you care why a child died.

`SA_RESTART` makes interrupted syscalls resume automatically instead of returning `EINTR`. It doesn't cover everything (`accept` on some systems, `select/poll/epoll_wait` never restart), so keep your `EINTR` checks anyway.

Trade-offs: true isolation — a crashing child can't take down the server — but ~1 MB and a few hundred microseconds per connection, and sharing state requires shared memory or pipes. Good for a few hundred connections, not thousands.

2b. Thread per client

```

struct conn { int fd; struct sockaddr_storage peer; socklen_t plen; };

static void *worker(void *arg) {
    struct conn *c = arg;          // heap-allocated: no stack lifetime races

```



```

    handle_client(c->fd);
    close(c->fd);
    free(c);
    return NULL;
}

int main(void) {
    signal(SIGPIPE, SIG_IGN);
    int lfd = tcp_listen("8080", SOMAXCONN);

    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED); // self-cleaning
    pthread_attr_setstacksize(&attr, 256 * 1024); // 8 MB default
    is wasteful

    for (;;) {
        int cfd = accept(lfd, NULL, NULL);
        if (cfd < 0) { if (errno == EINTR || errno == ECONNABORTED) continue;
        break; }

        struct conn *c = malloc(sizeof *c);
        if (!c) { close(cfd); continue; }
        c->fd = cfd;

        pthread_t tid;
        if (pthread_create(&tid, &attr, worker, c) != 0) {
            close(cfd); free(c); // shed load, don't spin
        }
    }
}

```

Pitfalls:

- **Never pass `&cfd`** — the address of a stack variable that the next loop iteration overwrites. Classic race. Heap-allocate or, if you must, pass the int by value through the `void*` (ugly but common: `(void *) (intptr_t)cfd`).
- **Detach or join.** A joinable thread that's never joined leaks its descriptor.
- Default 8 MB stacks × 1000 threads = 8 GB of virtual address space. Set it explicitly.
- Any shared state now needs a mutex. `errno` is thread-local (good); most `strtok`/`inet_ntoa`-style functions are not.

Trade-offs: cheaper than fork, shared memory is trivial, but a crash kills everything, and context-switching thousands of threads thrashes the scheduler and cache.

2c. Thread pool

Fixed worker count, bounded queue. This is what production code actually looks like before you move to an event loop.

```

#define QCAP 1024

typedef struct {
    int            q[QCAP];
    size_t         head, tail, count;
    pthread_mutex_t mu;
    pthread_cond_t not_empty, not_full;
    bool           shutdown;
} fdqueue;

static void q_init(fdqueue *q) {
    q->head = q->tail = q->count = 0;
    q->shutdown = false;
    pthread_mutex_init(&q->mu, NULL);
    pthread_cond_init(&q->not_empty, NULL);
    pthread_cond_init(&q->not_full, NULL);
}

static bool q_push(fdqueue *q, int fd) {
    pthread_mutex_lock(&q->mu);
    while (q->count == QCAP && !q->shutdown)
        pthread_cond_wait(&q->not_full, &q->mu);
    if (q->shutdown) { pthread_mutex_unlock(&q->mu); return false; }
    q->q[q->tail] = fd;
    q->tail = (q->tail + 1) % QCAP;
    q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->mu);
    return true;
}

static bool q_pop(fdqueue *q, int *out) {
    pthread_mutex_lock(&q->mu);
    while (q->count == 0 && !q->shutdown)
        pthread_cond_wait(&q->not_empty, &q->mu);    // while, not if – spurious
wakeups
    if (q->count == 0) { pthread_mutex_unlock(&q->mu); return false; }
    *out = q->q[q->head];
    q->head = (q->head + 1) % QCAP;
    q->count--;
    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->mu);
    return true;
}

static void *pool_worker(void *arg) {
    fdqueue *q = arg;
    int fd;
    while (q_pop(q, &fd)) {
        handle_client(fd);
        close(fd);
    }
    return NULL;
}

```

```
}

int main(void) {
    signal(SIGPIPE, SIG_IGN);
    fdqueue q; q_init(&q);

    long n = sysconf(_SC_NPROCESSORS_ONLN);
    pthread_t *tids = calloc(n, sizeof *tids);
    for (long i = 0; i < n; i++) pthread_create(&tids[i], NULL, pool_worker, &q);

    int lfd = tcp_listen("8080", SOMAXCONN);
    for (;;) {
        int cfd = accept(lfd, NULL, NULL);
        if (cfd < 0) { if (errno == EINTR || errno == ECONNABORTED) continue;
        break; }
        if (!q_push(&q, cfd)) close(cfd);
    }
}
```

Sizing: CPU-bound work → threads ≈ core count. Blocking I/O per connection → more, since threads sleep. This design still dedicates a thread to a connection for its whole lifetime, so it caps concurrency at pool size — that's the wall that pushes you to `epoll` in section 6.

`pthread_cond_wait` **must** be in a `while` loop, never `if` — spurious wakeups are permitted by the standard and do happen.

Comparison

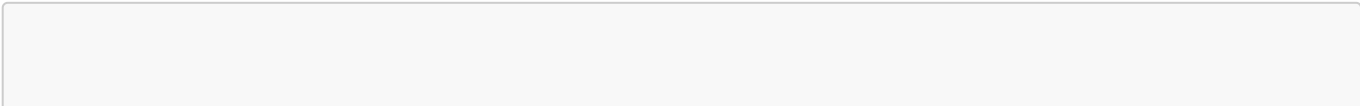
Model	Cost/conn	Isolation	Shared state	Practical ceiling
Iterative	0	n/a	trivial	1
Fork	~1 MB + μs	strong	hard (shm/pipes)	~hundreds
Thread/conn	64 KB–8 MB	none	trivial (+locks)	~1–5k
Thread pool	bounded	none	trivial (+locks)	pool size
Event loop (\$6)	~KB	none	trivial (single thread)	100k+

3. Stream Framing — The Core Concept

TCP guarantees: bytes arrive in order, without duplication or corruption. **TCP does not guarantee:** that your `recv()` boundaries match the sender's `send()` boundaries.

Three `send()` calls of 100 bytes can arrive as one `recv()` of 300, or 300 of 1, or 7/293. The stack coalesces (Nagle), splits (MSS), and delivers whatever's buffered when you ask.

This means a "message" is a concept **your protocol** must define. TCP has no idea one exists.



```

Sender:  send("HELLO")  send("WORLD")
Wire:    H E L L O W O R L D
Receiver:  recv() → "HELLOWOR"      recv() → "LD"

```

Code that does `recv(fd, buf, sizeof buf, 0)` and treats the result as one message is **broken**. It usually works in testing — small messages on loopback rarely fragment — and fails in production under load or across a real network. This is the number one bug in hand-rolled network code.

3a. Length-prefix framing (preferred for binary)

Send a fixed-size length header, then that many payload bytes.

```

+-----+-----+
| len(4) | payload      |
+-----+-----+

```

```

#define MAX_MSG (1u << 20) // 1 MB – bound it, always

int send_msg(int fd, const void *payload, uint32_t len) {
    if (len > MAX_MSG) { errno = EMSGSIZE; return -1; }
    uint8_t hdr[4];
    hdr[0] = len >> 24; hdr[1] = len >> 16; hdr[2] = len >> 8; hdr[3] = len;

    // Single writev – avoids Nagle/delayed-ACK interaction with two writes
    struct iovec iov[2] = {
        { .iov_base = hdr, .iov_len = 4 },
        { .iov_base = (void *)payload, .iov_len = len }
    };
    return writev_all(fd, iov, 2);
}

// Returns 1 = message read, 0 = clean EOF, -1 = error
int recv_msg(int fd, void *buf, uint32_t cap, uint32_t *out_len) {
    uint8_t hdr[4];
    ssize_t r = recv_all(fd, hdr, 4);
    if (r == 0) return 0;
    if (r < 0) return -1;

    uint32_t len = ((uint32_t)hdr[0] << 24) | ((uint32_t)hdr[1] << 16) |
                  ((uint32_t)hdr[2] << 8) | (uint32_t)hdr[3];

    if (len > MAX_MSG || len > cap) { errno = EMSGSIZE; return -1; } // ← the
    security check

    if (len > 0) {
        r = recv_all(fd, buf, len);
        if (r == 0) return -1; // EOF mid-message = protocol violation, not
    clean close
    }
}

```

```

        if (r < 0) return -1;
    }
    *out_len = len;
    return 1;
}

```

The `len > MAX_MSG` check is not optional. A hostile peer sends length `0xFFFFFFFF`; you `malloc()` it and either die or get an integer overflow in `malloc(len + 1)`. Length-prefix protocols without a sanity bound are a standard remote-DoS vulnerability.

EOF mid-message is an error, not a clean shutdown — distinguishing those two cases is what the return codes above are for.

Pros: works for arbitrary binary data, no escaping, receiver can pre-allocate exactly. Cons: must read the header before you know the size; peer can lie.

3b. Delimiter framing (natural for text)

Messages end with a sentinel — `\n` for line protocols, `\r\n\r\n` for HTTP headers.

Requires a **buffer that persists across `recv()` calls**, because a delimiter can land anywhere:

```

typedef struct {
    char    buf[8192];
    size_t  len;          // valid bytes in buf
} rdbuf;

// Returns line length (excluding \n) on success, -1 error, -2 EOF, -3 line too
long
ssize_t read_line(int fd, rdbuf *rb, char *out, size_t cap) {
    for (;;) {
        // 1. Is a complete line already buffered?
        char *nl = memchr(rb->buf, '\n', rb->len);
        if (nl) {
            size_t linelen = (size_t)(nl - rb->buf);
            if (linelen >= cap) return -3;
            memcpy(out, rb->buf, linelen);
            out[linelen] = '\0';
            // consume line + delimiter
            size_t consumed = linelen + 1;
            memmove(rb->buf, rb->buf + consumed, rb->len - consumed);
            rb->len -= consumed;
            return (ssize_t)linelen;
        }
        // 2. Buffer full with no delimiter → attacker sending endless data
        if (rb->len == sizeof rb->buf) return -3;

        // 3. Read more
        ssize_t n = recv(fd, rb->buf + rb->len, sizeof rb->buf - rb->len, 0);
        if (n == 0) return -2;
        if (n < 0) { if (errno == EINTR) continue; return -1; }
    }
}

```

```
        rb->len += (size_t)n;
    }
}
```

Note the **two-phase loop**: check the buffer *first*, read only when necessary. A single `recv()` may deliver three complete lines; if you don't drain the buffer before reading again, you deadlock waiting for data that already arrived.

Never use `read_line` with a one-byte-at-a-time `recv()` — it's correct but makes one syscall per byte.

Pros: human-readable, debuggable with `telnet/netcat`. Cons: binary payloads need escaping/encoding, and unbounded lines are a DoS vector — hence check (2).

3c. Fixed-size records

Every message is exactly N bytes. Framing is trivial: `recv_all(fd, &rec, sizeof rec)`.

Only viable when all messages genuinely are the same size (sensor readings, fixed market-data ticks). If you're tempted to send a `struct` directly over the wire — **don't**:

```
struct msg { uint8_t type; uint32_t id; uint16_t flags; };
send(fd, &m, sizeof m, 0);    // WRONG
```

`sizeof` differs across compilers due to padding, field order differs by endianness, and it's undefined behavior to reinterpret arbitrary bytes as a struct on the receiving side. Serialize explicitly, field by field, in defined byte order.

Framing choice

	Length-prefix	Delimiter	Fixed
Binary-safe	yes	needs escaping	yes
Pre-allocate	yes	no	yes
Debuggable by hand	no	yes	no
Used by	gRPC, MQTT, Redis (hybrid)	HTTP, SMTP, IRC	some telemetry

Real protocols mix them: HTTP uses `\r\n\r\n` for headers, then `Content-Length` (a length prefix) for the body.

4. Partial Reads/Writes

Covered in section 2, but here's why it matters more than it looks.

`send()` returning short is not rare. Send 1 MB and the kernel accepts whatever fits in the socket send buffer — often 64–256 KB — then blocks or returns short. Every byte after that needs another call.

```

ssize_t send_all(int fd, const void *buf, size_t n) {
    const char *p = buf;
    size_t sent = 0;
    while (sent < n) {
        ssize_t r = send(fd, p + sent, n - sent, MSG_NOSIGNAL);
        if (r > 0) { sent += (size_t)r; continue; }
        if (r < 0 && errno == EINTR) continue;
        return -1;
    }
    return (ssize_t)sent;
}

```

And the vectored version, so you can send header + body in one syscall:

```

ssize_t writev_all(int fd, struct iovec *iov, int cnt) {
    while (cnt > 0) {
        ssize_t r = writev(fd, iov, cnt);
        if (r < 0) { if (errno == EINTR) continue; return -1; }
        // Advance iovecs past the bytes consumed
        while (cnt > 0 && (size_t)r >= iov->iiov_len) {
            r -= (ssize_t)iov->iiov_len;
            iov++; cnt--;
        }
        if (cnt > 0 && r > 0) {
            iov->iiov_base = (char *)iov->iiov_base + r;
            iov->iiov_len -= (size_t)r;
        }
    }
    return 0;
}

```

That iovec-advance loop is fiddly and universally gotten wrong on first attempt — write it once, test it with a tiny `SO_SNDBUF` to force short writes, and reuse it.

On non-blocking sockets, `send_all` is an anti-pattern — you can't loop, you must return "wrote M of N" to the event loop and register for writability with the remainder buffered. That restructuring is section 6's main content.

5. Graceful Shutdown, Half-Close, Detecting Disconnect

Detecting peer disconnect

Signal	Meaning
<code>recv()</code> returns 0	Peer called <code>close()</code> or <code>shutdown(SHUT_WR)</code> — clean FIN
<code>recv()</code> returns -1, <code>ECONNRESET</code>	Peer sent RST — crashed, or closed with unread data

Signal	Meaning
<code>send()</code> returns <code>-1</code> , <code>EPIPE</code>	Peer closed; you already got an RST
Nothing at all	Peer's machine lost power, cable pulled — TCP is silent forever

That last row is the important one. **A dead peer on a silent connection is undetectable at the TCP layer.** Half-open connections accumulate. Two mitigations:

```
// 1. TCP keepalive – cheap, coarse
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &yes, sizeof yes);
int idle = 60, intvl = 10, cnt = 3;      // Linux-specific tuning
setsockopt(fd, IPPROTO_TCP, TCP_KEEPIIDLE, &idle, sizeof idle);
setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &intvl, sizeof intvl);
setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &cnt, sizeof cnt);
```

Default keepalive idle is **2 hours** — useless without tuning.

```
// 2. Read timeout – often simpler and more portable
struct timeval tv = { .tv_sec = 30 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
// recv() now returns -1/EAGAIN after 30s of silence
```

Application-level heartbeats (PING/PONG) are strictly better than both: they traverse NAT (keeping mappings alive — recall section 1), they prove the *application* is alive rather than just the kernel, and they work identically everywhere.

Half-close

`shutdown(fd, SHUT_WR)` sends FIN while leaving your read side open. The peer sees `recv() == 0` and knows you're done sending, but can still reply.

```
// Client: send request, signal completion, read response
send_all(fd, request, req_len);
shutdown(fd, SHUT_WR);           // "that's the whole request"

char buf[4096];
ssize_t n;
while ((n = recv(fd, buf, sizeof buf, 0)) > 0)
    fwrite(buf, 1, (size_t)n, stdout);
close(fd);
```

This is how `netcat`, HTTP/1.0 **Connection: close**, and any "send until EOF" protocol work. Without half-close you'd need length framing on the request too.

Graceful server shutdown

```
static volatile sig_atomic_t running = 1;
static void on_term(int s) { (void)s; running = 0; }

// ... sigaction(SIGINT/SIGTERM, ...) WITHOUT SA_RESTART, so accept() returns
EINTR

while (running) {
    int cfd = accept(lfd, NULL, NULL);
    if (cfd < 0) {
        if (errno == EINTR) continue; // loop re-checks `running`
        ...
    }
    ...
}
close(lfd); // stop accepting
drain_active_connections(); // let in-flight requests finish
```

Deliberately **omit** `SA_RESTART` here — you *want* the signal to break `accept()` so the loop re-tests the flag. `volatile sig_atomic_t` is the only type safe to touch from a handler.

Closing a connection correctly

```
void close_gracefully(int fd) {
    shutdown(fd, SHUT_WR); // FIN: we're done sending

    struct timeval tv = { .tv_sec = 5 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);

    char sink[4096]; // drain peer's remaining data
    while (recv(fd, sink, sizeof sink, 0) > 0)
        ;
    close(fd);
}
```

Skipping the drain means `close()` with unread data → RST → the peer may lose the response you just sent. The bounded timeout stops a malicious peer from holding the fd open forever.

Avoid `SO_LINGER` unless you specifically want `close()` to block until data is flushed (or to send an immediate RST with `l_linger = 0`). It's a common source of mysterious hangs.

6. `SIGPIPE` — say it once more

Writing to a socket whose peer has closed raises `SIGPIPE`. **Default action: terminate the process.** A server that dies whenever a client hits Ctrl-C is the single most embarrassing bug in this section.

```
signal(SIGPIPE, SIG_IGN);           // process-wide, at startup
send(fd, buf, len, MSG_NOSIGNAL);   // per-call (Linux)
```

Portability: macOS/BSD lack `MSG_NOSIGNAL` but have a per-socket option:

```
#ifdef SO_NOSIGPIPE
    int one = 1;
    setsockopt(fd, SOL_SOCKET, SO_NOSIGPIPE, &one, sizeof one);
#endif
```

Use `SIG_IGN` **and** the per-call flag. With `SIGPIPE` ignored, the write returns `EPIPE` instead — which you handle like any other dead-connection error.

In a library, prefer `MSG_NOSIGNAL/SO_NOSIGPIPE` over `signal()` — changing global disposition from a library surprises the host application.

Pitfall Checklist

1. Assuming one `send()` = one `recv()` — **the bug**
 2. No framing at all; "it works on localhost"
 3. Unbounded length prefix → remote OOM/DoS
 4. Treating EOF mid-message as a clean close
 5. Passing `&cfd` to `pthread_create`
 6. Parent not closing `cfd` after fork → fd leak
 7. Single `waitpid` in `SIGCHLD` handler → zombies
 8. Not preserving `errno` in a signal handler
 9. `pthread_cond_wait` under `if` instead of `while`
 10. No `SIGPIPE` handling
 11. Ignoring short `send()` returns
 12. Sending raw structs across the wire
 13. Relying on TCP to notice a dead peer
 14. Not draining before `close()` → RST truncates your own response
 15. `exit()` instead of `_exit()` in a forked child
-

Exercises

Build these in order; each exposes a distinct failure mode.

1. **Iterative echo**, then connect two clients. Observe the second one hang. Then observe that it *works* if the first disconnects quickly — that's why this bug survives testing.
2. **Prove framing is necessary**. Client sends `send(fd, "AAAA", 4); send(fd, "BBBB", 4);` in a tight loop, server prints each `recv()` result with its length. On loopback you'll mostly see clean 4-byte reads. Now

add `TCP_NODELAY` off and a 1 MB burst, or run it across a real network, and watch the boundaries dissolve. Then repeat with length-prefix framing and confirm every message arrives intact.

3. **Length-prefixed chat server**, thread-per-client, with a mutex-protected client list for broadcast. Test with a malicious client that sends length `0xFFFFFFFF` — confirm your server rejects it instead of dying.
4. **Fork server, deliberately omit the `SIGCHLD` handler**. Watch `ps aux | grep defunct` fill up. Add the handler with a single `waitpid` (no loop), hammer it with 50 simultaneous clients, and watch zombies *still* accumulate. Then add the `while` loop. This is the clearest demonstration of signal coalescing you'll ever get.
5. **Force short writes**. Set `SO_SNDBUF` to 4096, send 1 MB, and instrument `send_all` to count iterations. Then remove the loop and see exactly how much data you lose.
6. **Half-close**. Implement a "send file until EOF, then read a checksum back" protocol. It's impossible without `shutdown(SHUT_WR)`.

Keep the length-prefix framing code — every section from here on assumes you have it.

Ready for **UDP Programming** when you are.

4. UDP Programming

- `sendto()`, `recvfrom()`, connected UDP sockets
- Datagram loss, duplication, reordering
- Building reliability on UDP: ACKs, sequence numbers, timeouts, retransmission
- Broadcast (`SO_BROADCAST`) and multicast (`IP_ADD_MEMBERSHIP`, `IP_MULTICAST_TTL`)
- Path MTU considerations, datagram size limits

UDP Programming

The mental flip from section 4: TCP gives you a reliable stream with no message boundaries. UDP gives you message boundaries with no reliability. Every problem you *didn't* have in TCP, you now have — and every problem you *did* have (framing) disappears.

1. `sendto()` / `recvfrom()`

```
ssize_t sendto(int fd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest, socklen_t destlen);

ssize_t recvfrom(int fd, void *buf, size_t len, int flags,
                 struct sockaddr *src, socklen_t *srclen);
```

No `listen()`, no `accept()`, no `connect()` required. A UDP socket is just an endpoint; every datagram carries its own address.

Minimal echo server

```

int main(void) {
    struct addrinfo hints = {0}, *res;
    hints.ai_family    = AF_INET6;
    hints.ai_socktype  = SOCK_DGRAM;
    hints.ai_flags     = AI_PASSIVE;
    getaddrinfo(NULL, "9000", &hints, &res);

    int fd = socket(res->ai_family, res->ai_socktype | SOCK_CLOEXEC, res-
>ai_protocol);
    int yes = 1, no = 0;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
    setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no);
    bind(fd, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);

    for (;;) {
        char buf[2048];
        struct sockaddr_storage src;
        socklen_t slen = sizeof src;           // value-result: reset EVERY
iteration

        ssize_t n = recvfrom(fd, buf, sizeof buf, 0,
                             (struct sockaddr *)&src, &slen);
        if (n < 0) { if (errno == EINTR) continue; break; }

        // n == 0 is a legitimate zero-length datagram, NOT EOF
        sendto(fd, buf, (size_t)n, 0, (struct sockaddr *)&src, slen);
    }
}

```

Note what's absent: no per-client state, no threads, no framing. **One server socket serves every client** — there's no connection to fork or thread for. That's UDP's structural advantage.

Client

```

int fd = socket(AF_INET, SOCK_DGRAM, 0);
// No bind needed – sendto() implicitly binds an ephemeral port on first send
sendto(fd, "hello", 5, 0, (struct sockaddr *)&server, sizeof server);

char buf[2048];
struct sockaddr_storage from;
socklen_t flen = sizeof from;
ssize_t n = recvfrom(fd, buf, sizeof buf, 0, (struct sockaddr *)&from, &flen);

```

The semantics that differ from TCP

Message boundaries are preserved, exactly. One `sendto()` of 500 bytes produces exactly one `recvfrom()` of 500 bytes, or nothing at all. Never 300 then 200.

Truncation is silent and lossy. If the datagram is 2000 bytes and your buffer is 1024, you get 1024 bytes and **the remaining 976 are discarded forever**. There is no way to get them back.

```
struct msghdr msg = {0};
// ... set up iov, name ...
ssize_t n = recvmsg(fd, &msg, 0);
if (msg.msg_flags & MSG_TRUNC) {
    // Datagram was larger than our buffer — we lost data
}
```

Only `recvmsg()` reports this. `recvfrom()` gives you no indication at all. **Always size your receive buffer larger than any datagram you'll accept** — 64 KB if you're being safe, 2 KB if your protocol bounds itself.

`recvfrom()` returning 0 means a zero-length datagram, not EOF. Zero-length UDP datagrams are valid and are sometimes used as heartbeats or probes. Code ported from TCP that treats 0 as "peer closed" will silently break.

A `sendto()` that succeeds means nothing. It means the datagram left your kernel. Not that it reached the network, the peer's kernel, or the peer's application. There is no acknowledgement anywhere in the stack.

Never trust the source address. UDP has no handshake, so source IPs are trivially spoofable. This is why UDP services are amplification-attack vectors (DNS, NTP, memcached): attacker sends a small query with a spoofed victim source, you send a large reply to the victim. If your UDP service replies with more data than it receives, you've built a weapon. Keep the amplification factor ≤ 1 , or require a handshake/token before sending large responses. QUIC does exactly this with its address-validation token.

2. Connected UDP Sockets

`connect()` on a `SOCK_DGRAM` socket sends **no packets**. It just records a default peer in the kernel.

```
int fd = socket(AF_INET, SOCK_DGRAM, 0);
connect(fd, (struct sockaddr *)&server, sizeof server);

send(fd, buf, len, 0);           // no address needed
recv(fd, buf, sizeof buf, 0);    // only datagrams from `server` are delivered
```

Four concrete benefits:

1. **Filtering.** Datagrams from any other source are dropped by the kernel. Without this, anyone who knows your ephemeral port can inject packets into your client.
2. **ICMP errors become visible.** This is the big one. An *unconnected* UDP socket silently discards ICMP port-unreachable messages — send to a dead server and you wait forever with no error. A *connected* socket surfaces it as `ECONNREFUSED` on the next `send()` or `recv()`. Debugging "why does my client hang" almost always ends here.
3. **Performance.** The kernel caches the route lookup instead of redoing it per datagram — meaningfully faster in tight loops.

4. **Simpler code.** `send/recv` instead of `sendto/recvfrom`, no address plumbing.

You can re-`connect()` to a different peer, or dissolve the association by connecting to an address with `sa_family = AF_UNSPEC`.

Rule of thumb: clients that talk to one server should always connect. Servers talking to many clients cannot (one socket, many peers) — unless you spawn a per-client connected socket, which is how QUIC and some game servers do it.

3. Loss, Duplication, Reordering

The three failure modes, all of which *will* happen on a real network:

Failure	Cause	Frequency
Loss	Router queue overflow, congestion, wireless interference, bad checksum	0.01%–5% typical; much worse on congested/mobile links
Reordering	ECMP load balancing across paths, route changes	Uncommon but real, especially across ISPs
Duplication	Retransmission at the link layer, routing loops	Rare but happens

On loopback and a quiet LAN you'll see essentially none of these, which is why UDP code that "works fine in testing" collapses in production. Simulate them deliberately:

```
# Linux traffic control – inject 5% loss, 100ms±20ms delay, 1% duplication, 2%
reorder
sudo tc qdisc add dev lo root netem loss 5% delay 100ms 20ms duplicate 1% reorder
2%
sudo tc qdisc del dev lo root          # remove
```

Run every UDP program you write under `netem` before you believe it works.

Also: **checksums.** The UDP checksum is optional in IPv4 (a zero checksum means "not computed"), mandatory in IPv6. Don't disable it. Even when enabled it's only 16 bits, so it misses roughly 1 in 65536 corruptions — if data integrity genuinely matters, add your own CRC32 or stronger.

Local loss you can fix: if `recvfrom` can't keep up, the kernel's socket receive buffer overflows and drops datagrams silently. Raise it and monitor:

```
int rcvbuf = 4 * 1024 * 1024;
setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &rcvbuf, sizeof rcvbuf);
```

Check drops with `netstat -su` (look for "packet receive errors") or `ss -u -m`. A slow handler in your receive loop is the most common cause of "UDP loss" that turns out not to be the network at all.

4. Building Reliability on UDP

If you need *all* of ordering + reliability + congestion control, use TCP. You will not beat 40 years of tuning. Build on UDP when you need **partial** reliability — some messages retransmitted, others deliberately not; no head-of-line blocking; custom timing.

The pieces

Sequence numbers — identify and order messages, detect gaps and duplicates. **ACKs** — receiver confirms receipt. **Timeout + retransmission** — sender resends unacknowledged data. **Duplicate detection** — receiver discards what it's already seen.

Wire format

```
#define MAGIC 0x55445031u    /* "UDP1" */

enum { PKT_DATA = 1, PKT_ACK = 2 };

// 12-byte header, explicitly serialized — never send a struct
// magic(4) | type(1) | flags(1) | seq(2) | ack(2) | len(2)
```

```
static void put_u16(uint8_t *p, uint16_t v) { p[0] = v >> 8; p[1] = v; }
static uint16_t get_u16(const uint8_t *p) { return (uint16_t)((p[0] << 8) | p[1]); }
}
```

Stop-and-wait (start here)

Send one packet, wait for its ACK, retransmit on timeout. Simple, correct, and slow — throughput is capped at one packet per RTT regardless of bandwidth.

```
#define MAX_RETRIES 5

int rel_send(int fd, uint16_t seq, const void *payload, uint16_t len) {
    uint8_t pkt[1400];
    put_u32(pkt, MAGIC);
    pkt[4] = PKT_DATA; pkt[5] = 0;
    put_u16(pkt + 6, seq);
    put_u16(pkt + 8, 0);
    put_u16(pkt + 10, len);
    memcpy(pkt + 12, payload, len);

    int timeout_ms = 200;                // initial RTO

    for (int try = 0; try < MAX_RETRIES; try++) {
        if (send(fd, pkt, 12u + len, 0) < 0) return -1;

        struct pollfd pfd = { .fd = fd, .events = POLLIN };

```

```

    int64_t deadline = now_ms() + timeout_ms;

    for (;;) {
        int64_t remain = deadline - now_ms();
        if (remain <= 0) break;                // timeout → retransmit

        int r = poll(&pfd, 1, (int)remain);
        if (r < 0) { if (errno == EINTR) continue; return -1; }
        if (r == 0) break;

        uint8_t rsp[64];
        ssize_t n = recv(fd, rsp, sizeof rsp, 0);
        if (n < 12) continue;                  // runt – ignore
        if (get_u32(rsp) != MAGIC) continue;   // not ours
        if (rsp[4] == PKT_ACK && get_u16(rsp + 8) == seq)
            return 0;                          // ACKed
        // else: stale ACK for an old seq – ignore, keep waiting
    }
    timeout_ms *= 2;                          // exponential backoff
    if (timeout_ms > 3000) timeout_ms = 3000;
}
errno = ETIMEDOUT;
return -1;
}

```

Four things this demonstrates that people get wrong:

- **The deadline is recomputed inside the loop.** A stale ACK or an `EINTR` must not reset your timeout. Naively calling `poll(&pfd, 1, timeout_ms)` after each spurious wakeup can extend the wait indefinitely.
- **Exponential backoff.** Retransmitting at a fixed fast interval during congestion makes the congestion worse. This is the minimum congestion-awareness a UDP protocol owes the network.
- **Sanity checks before parsing.** `n < 12` and the magic check reject garbage and stray traffic. Every field you read must be bounds-checked — the length field especially, exactly as in section 4's framing.
- **Old ACKs are ignored, not fatal.** They arrive constantly due to retransmission.

Receiver side — duplicate detection

```

// Retransmission means you WILL receive duplicates. Handle them or corrupt your
// state.
static uint16_t expected = 0;

void handle_data(int fd, const uint8_t *pkt, size_t n,
                 const struct sockaddr *src, socklen_t slen) {
    if (n < 12 || get_u32(pkt) != MAGIC) return;
    uint16_t seq = get_u16(pkt + 6);
    uint16_t len = get_u16(pkt + 10);
    if (n < 12u + len) return;                // truncated/lying header

    // ALWAYS ACK – even duplicates. A lost ACK looks like a lost packet to

```



```

    // the sender, and it will retransmit. Not re-ACKing deadlocks the peer.
    uint8_t ack[12];
    put_u32(ack, MAGIC); ack[4] = PKT_ACK; ack[5] = 0;
    put_u16(ack + 6, 0); put_u16(ack + 8, seq); put_u16(ack + 10, 0);
    sendto(fd, ack, sizeof ack, 0, src, slen);

    if (seq != expected) return;           // duplicate or out-of-order:
drop
    expected++;
    deliver_to_app(pkt + 12, len);
}

```

"Always ACK, even duplicates" is the rule people miss. A dropped ACK is indistinguishable from a dropped data packet from the sender's perspective. If the receiver only ACKs new data, the sender retransmits forever.

Sliding window (the real version)

Stop-and-wait gives you one packet per RTT — over a 100 ms link that's ~10 packets/sec no matter your bandwidth. A window of N in-flight packets multiplies throughput by N.

Structure:

```

typedef struct {
    uint8_t data[1400];
    uint16_t len;
    uint16_t seq;
    int64_t sent_at;      // for RTO calculation
    int     retries;
    bool    acked;
} pkt_slot;

typedef struct {
    pkt_slot win[WINDOW]; // circular
    uint16_t base;        // oldest unacked seq
    uint16_t next_seq;    // next to send
    int     rto_ms;
    int     srtt, rttvar; // RFC 6298 RTT estimation
} sender_state;

```

Design decisions you now have to make:

- **Go-Back-N** (retransmit everything from the loss point — simple, wasteful) vs **Selective Repeat** (retransmit only what's missing — needs receiver buffering and SACK-style ACKs)
- **Cumulative vs selective ACKs**
- **RTO estimation** — use RFC 6298's algorithm ($SRTT, RTTVAR, RTO = SRTT + 4 \times RTTVAR$), not a fixed constant. Karn's algorithm says: don't sample RTT from a retransmitted packet, you can't tell which transmission the ACK belongs to.
- **Sequence number wraparound** — 16 bits wraps at 65536. Compare with modular arithmetic, never <:

```
static inline bool seq_lt(uint16_t a, uint16_t b) { return (int16_t)(a - b)
< 0; }
```

- **Congestion control** — if you skip this and just blast the window, you're a bad network citizen and you'll perform terribly under load. Implement at least AIMD.

At this point you are reimplementing TCP. That's fine as an exercise and it's exactly what QUIC does — but be honest about whether you need it. **If you want reliability without writing this, look at QUIC ([ngtcp2](#), [msquic](#), [quiche](#)), SCTP, KCP, or ENet** before rolling your own.

5. Broadcast and Multicast

Two things TCP structurally cannot do.

Broadcast

Send to every host on the local subnet. Requires an explicit opt-in — the kernel refuses otherwise with [EACCES](#).

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &yes, sizeof yes);

struct sockaddr_in bcast = {0};
bcast.sin_family = AF_INET;
bcast.sin_port = htons(9000);
inet_pton(AF_INET, "192.168.1.255", &bcast.sin_addr); // subnet-directed
// or INADDR_BROADCAST (255.255.255.255) — limited broadcast, never routed

sendto(fd, msg, len, 0, (struct sockaddr *)&bcast, sizeof bcast);
```

Receivers just [bind\(\)](#) to the port (with [SO_REUSEADDR](#) so multiple processes can listen).

Broadcast is IPv4-only — IPv6 removed it entirely in favor of multicast. It's also never routed off the local segment, and it interrupts every host on the subnet including those with no interest. Use it for service discovery on a LAN (DHCP works this way) and nothing else.

Multicast

Send once, delivered only to hosts that explicitly joined the group. Scales properly and works on both IPv4 and IPv6.

Address ranges: IPv4 [224.0.0.0/4](#), with [239.0.0.0/8](#) reserved for private/local use — start there. IPv6 uses [ff00::/8](#).

Sender:

```

int fd = socket(AF_INET, SOCK_DGRAM, 0);

unsigned char ttl = 1;    // 1 = local subnet only. CRITICAL default.
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof ttl);

unsigned char loop = 1;   // do we receive our own sends?
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_LOOP, &loop, sizeof loop);

// Pin to a specific interface – essential on multi-homed hosts
struct in_addr ifaddr;
inet_pton(AF_INET, "192.168.1.10", &ifaddr);
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_IF, &ifaddr, sizeof ifaddr);

struct sockaddr_in grp = {0};
grp.sin_family = AF_INET;
grp.sin_port   = htons(9000);
inet_pton(AF_INET, "239.1.2.3", &grp.sin_addr);
sendto(fd, msg, len, 0, (struct sockaddr *)&grp, sizeof grp);

```

`IP_MULTICAST_TTL` defaults to **1**, meaning your traffic never leaves the local subnet. That's a safe default and the cause of most "multicast doesn't work" confusion. Raising it requires multicast-enabled routers (PIM/IGMP snooping), which most networks — and essentially all cloud providers — don't have. **Multicast generally does not work on AWS/GCP/Azure or across the internet.** It's a datacenter/LAN technology.

Receiver:

```

int fd = socket(AF_INET, SOCK_DGRAM, 0);
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

struct sockaddr_in addr = {0};
addr.sin_family      = AF_INET;
addr.sin_port        = htons(9000);
addr.sin_addr.s_addr = htonl(INADDR_ANY);    // bind ANY, not the group address
bind(fd, (struct sockaddr *)&addr, sizeof addr);

struct ip_mreq mreq;
inet_pton(AF_INET, "239.1.2.3", &mreq.imr_multiaddr);
mreq.imr_interface.s_addr = htonl(INADDR_ANY); // or a specific interface
setsockopt(fd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof mreq);

// ... recvfrom() ...

setsockopt(fd, IPPROTO_IP, IP_DROP_MEMBERSHIP, &mreq, sizeof mreq);

```

The join is what triggers IGMP membership reports, which tell switches and routers to forward the group's traffic to this port. Joining is per-socket *and* per-interface — on a multi-homed host you must join on each interface you care about, or specify the right one.

For IPv6: `IPV6_JOIN_GROUP` with `struct ipv6_mreq`, `IPV6_MULTICAST_HOPS`, `IPV6_MULTICAST_IF` (which takes an interface *index*, not an address). `setsockopt` with `MCAST_JOIN_GROUP` / `struct group_req` is the protocol-independent modern form and works for both families.

Source-specific multicast (SSM) — `IP_ADD_SOURCE_MEMBERSHIP` / `MCAST_JOIN_SOURCE_GROUP` — filters to a specific sender. Preferred where supported; it avoids the group-hijacking problem where anyone can inject traffic into a group.

Real uses: market-data feeds (the dominant one), service discovery (mDNS/Bonjour uses `224.0.0.251`, SSDP uses `239.255.255.250`), IPTV, cluster heartbeats.

6. Datagram Size and Path MTU

The hard limits

- **Theoretical maximum:** 65507 bytes for IPv4 (65535 – 20 IP – 8 UDP). IPv6 jumbograms can exceed this but effectively never do.
- **Practical maximum:** ~1472 bytes on standard Ethernet (1500 MTU – 20 IP – 8 UDP).
- **Safe maximum: 1200 bytes.** This is what QUIC uses and it's the right default for anything crossing the internet.

Why 1200 and not 1472? Tunnels. VPN, PPPoE, IPsec, GRE, and IPv6-in-IPv4 all shave bytes off the path MTU. 1500 is the *link* MTU, not the *path* MTU.

Why fragmentation is worse for UDP than TCP

If a datagram exceeds the path MTU, IPv4 routers fragment it. **Losing any one fragment loses the entire datagram** — the receiver can't reassemble and discards everything. A 4000-byte datagram is three fragments; at 1% per-packet loss, the effective datagram loss rate is ~3%. The math gets ugly fast.

IPv6 doesn't fragment in routers at all — it drops and returns ICMPv6 Packet Too Big. If ICMP is filtered (very common), you get a **PMTU black hole**: small datagrams work, large ones vanish silently. This is the classic "our protocol works on the LAN and fails over the VPN" bug.

Controlling it

```
// Set DF and let the kernel track path MTU
int mtu_disc = IP_PMTUDISC_DO;
setsockopt(fd, IPPROTO_IP, IP_MTU_DISCOVER, &mtu_disc, sizeof mtu_disc);
// Now oversized sends fail with EMSGSIZE instead of fragmenting

// Query the discovered path MTU
int mtu; socklen_t len = sizeof mtu;
getsockopt(fd, IPPROTO_IP, IP_MTU, &mtu, &len);
```

`IP_PMTUDISC_DONT` (fragment freely), `IP_PMTUDISC_WANT` (kernel default — fragment only when needed), `IP_PMTUDISC_PROBE` (set DF, ignore MTU, for probing).

The practical rule

Design your protocol so a message fits in 1200 bytes. If it doesn't, do application-layer fragmentation — split into numbered chunks and reassemble yourself, so you can retransmit individual chunks rather than losing everything.

DNS chose 512 bytes for exactly this reason, and EDNS0's negotiated larger sizes have caused real-world fragmentation problems ever since.

Pitfall Checklist

1. Receive buffer smaller than the largest datagram → silent truncation
 2. Treating `recvfrom() == 0` as EOF (it's a zero-length datagram)
 3. Not resetting `socklen_t` before every `recvfrom()`
 4. Not using `connect()` on clients → hangs instead of `ECONNREFUSED`
 5. Trusting the source address (spoofing, amplification)
 6. Reply larger than request → you built an amplifier
 7. Testing only on loopback, where loss/reorder never happen
 8. Fixed retransmission interval with no backoff
 9. Not ACKing duplicates → sender retransmits forever
 10. Sequence comparison with `<` instead of modular arithmetic
 11. Datagrams over 1200 bytes across the internet
 12. Forgetting `SO_BROADCAST` → `EACCES`
 13. Multicast TTL of 1 with an off-subnet receiver
 14. Binding to the group address instead of `INADDR_ANY` for multicast receive
 15. Slow receive loop → kernel buffer overflow blamed on "the network"
 16. Sending raw structs (still wrong, still endianness- and padding-dependent)
-

Exercises

1. **UDP echo server + client.** Confirm message boundaries: send 5 datagrams of 100 bytes rapid-fire, verify you get exactly 5 `recvfrom()` calls of 100 bytes each. Contrast with the TCP version from section 4, which will coalesce them.
2. **Prove the `connect()` difference.** Point a client at a port with nothing listening. Unconnected: it hangs forever. Connected: `ECONNREFUSED`. Same code otherwise.
3. **Prove truncation.** Send 3000 bytes, receive into a 1024 buffer. Note that you get 1024 with no error indication. Switch to `recvmsg` and check `MSG_TRUNC`.
4. **Stop-and-wait file transfer** under `tc netem loss 20%`. Transfer a 1 MB file, verify a byte-exact checksum. Then remove the duplicate-detection check and watch the file corrupt.
5. **Measure the stop-and-wait ceiling.** Under `netem delay 100ms`, measure throughput. It should be roughly `packet_size / 0.2s` regardless of your bandwidth. Then implement a window of 32 and measure again.

6. **LAN discovery via broadcast.** Client broadcasts "WHO_HAS_SERVICE", servers reply with their address and port. This is how most home-network device discovery works.
7. **Multicast pub/sub.** One publisher on 239.1.2.3, three subscribers on different machines (or with IP_MULTICAST_LOOP on one machine). Then run `tcpdump -i any 'net 239.0.0.0/8'` and watch a single transmitted packet reach all three.
8. **Find your path MTU.** Set `IP_PMTUDISC_DO`, then binary-search datagram sizes until `MSGSIZE`. Compare with `ping -M do -s <size>`.

Do exercise 4 under `netem` before you believe any UDP code you write. It's the difference between "works" and "works."

Ready for **Socket Options & Control** when you are.

5. Socket Options & Control

- `setsockopt/getsockopt`
- `SO_REUSEADDR`, `SO_REUSEPORT`, `SO_KEEPAIVE`, `SO_LINGER`
- `SO_RCVBUF`, `SO_SNDBUF`, `SO_RCVTIMEO`, `SO_SNDTIMEO`
- `TCP_NODELAY`, `TCP_CORK`, `TCP_QUICKACK`, TCP keepalive tuning
- `IPV6_V6ONLY` and dual-stack sockets
- `fcntl` with `O_NONBLOCK`, `FD_CLOEXEC`

Socket Options & Control

Socket options are the knobs between "works" and "works well." Most defaults are wrong for servers, a few are actively dangerous, and several are the difference between a 40 ms latency and a 0.1 ms one.

1. `setsockopt` / `getsockopt`

```
int setsockopt(int fd, int level, int optname,
               const void *optval, socklen_t optlen);
int getsockopt(int fd, int level, int optname,
               void *optval, socklen_t *optlen);
```

Levels determine which protocol layer interprets `optname`:

Level	Scope	Header
<code>SOL_SOCKET</code>	Generic socket layer	<code><sys/socket.h></code>
<code>IPPROTO_TCP</code>	TCP-specific	<code><netinet/tcp.h></code>
<code>IPPROTO_IP</code>	IPv4	<code><netinet/in.h></code>
<code>IPPROTO_IPV6</code>	IPv6	<code><netinet/in.h></code>

Level	Scope	Header
IPPROTO_UDP	UDP-specific	<netinet/udp.h>

Almost every boolean option takes an `int` (0/1), **not a `bool` or `char`**. Passing a `char` gives you `EINVAL` or, worse, reads three bytes of adjacent stack:

```
int yes = 1; // correct
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

char yes_c = 1; // WRONG for most options
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes_c, sizeof yes_c);
```

The exceptions are IPv4 multicast options (`IP_MULTICAST_TTL`, `IP_MULTICAST_LOOP`), which historically take `unsigned char` on BSD-derived stacks — Linux accepts both. Structured options take structs: `SO_LINGER` → `struct linger`, `SO_RCVTIMEO` → `struct timeval`.

`getsockopt`'s `optlen` is **value-result**, like `accept`:

```
int val; socklen_t len = sizeof val;
getsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &val, &len);
```

A wrapper worth writing

Silent `setsockopt` failures are a real source of mystery. Most code ignores the return value:

```
static int set_int_opt(int fd, int level, int opt, int val, const char *name) {
    if (setsockopt(fd, level, opt, &val, sizeof val) < 0) {
        fprintf(stderr, "setsockopt(%s) failed: %s\n", name, strerror(errno));
        return -1;
    }
    return 0;
}

#define SET_OPT(fd, lvl, opt, val) set_int_opt((fd), (lvl), (opt), (val), #opt)
```

Timing matters

Some options must be set at specific points:

Option	When
SO_REUSEADDR, SO_REUSEPORT	Before <code>bind()</code> — no effect after
SO_RCVBUF, SO_SNDBUF	Before <code>listen()/connect()</code> to affect the TCP window scale negotiation

Option	When
<code>TCP_NODELAY</code> , <code>TCP_QUICKACK</code>	Any time; per-connection
<code>IPV6_V6ONLY</code>	Before <code>bind()</code>
<code>SO_LINGER</code> , timeouts	Any time before use

Accepted sockets do not inherit everything. On Linux, an `accept()`ed fd inherits most `SOL_SOCKET` options from the listener (including buffer sizes) but **not** `O_NONBLOCK` and **not** `FD_CLOEXEC`. On BSD, `O_NONBLOCK` is inherited. Never rely on inheritance — set per-connection options explicitly after `accept()`, and use `accept4()` on Linux for the flags.

2. Address Reuse

`SO_REUSEADDR`

The single most important server option. Without it, restarting your server fails with `EADDRINUSE` for up to 60 seconds while old connections sit in `TIME_WAIT` (section 1).

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
// ... then bind()
```

What it actually does on Linux/BSD: allows `bind()` to a local address that has connections in `TIME_WAIT`. It does **not** let two live sockets listen on the same `addr:port` — that still fails. It also permits binding a specific address when the wildcard is bound (and vice versa) in some cases.

Windows is different and dangerous: there, `SO_REUSEADDR` genuinely allows two sockets to bind the same address, letting another process hijack your port. The Windows equivalent of the Unix behavior is `SO_EXCLUSIVEADDRUSE`. Don't port this option blindly.

Set it on every listening socket. There is essentially no downside on Unix.

`SO_REUSEPORT` (Linux 3.9+, BSD)

Genuinely different: allows **multiple sockets to bind the exact same `addr:port`**, with the kernel load-balancing incoming connections across them via a hash of the 4-tuple.

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes);
```

This solves the **thundering herd** problem. The old approach — one listening socket shared across *N* processes all blocked in `accept()` — wakes many processes for one connection and creates a contended accept path. With `SO_REUSEPORT`, each worker has its own listener and its own accept queue:


```
// N worker processes, each running this
int worker_listen(const char *port) {
    int fd = socket(AF_INET6, SOCK_STREAM | SOCK_CLOEXEC, 0);
    int yes = 1, no = 0;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
    setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes);
    setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no);
    // ... bind, listen
    return fd;
}
```

This is how nginx, HAProxy, and Envoy scale accept across cores. It also enables **zero-downtime restarts**: start the new binary, it binds the same port, drain and stop the old one.

Three caveats:

- **All sockets must set it**, including the first. If the first binder omits it, subsequent binds fail.
- Linux requires the sockets have the same effective UID — prevents port hijacking.
- **Connections queued on a worker that exits are lost**. The kernel's hash still points at the closed socket for in-flight SYNs. Mitigate with `SO_REUSEPORT` + eBPF (`SO_ATTACH_REUSEPORT_EBPF`) or by draining carefully.

For UDP, `SO_REUSEPORT` distributes datagrams across sockets by 4-tuple hash — useful for multi-threaded UDP servers, but it means datagrams from one client consistently land on one socket (usually what you want).

3. Connection Lifetime Options

`SO_KEEPALIVE` and TCP keepalive tuning

Recall from section 4: a peer whose machine loses power is undetectable at the TCP layer. Keepalive is the kernel's answer.

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &yes, sizeof yes);
```

The defaults are useless: 2 hours idle, then 9 probes 75 seconds apart — over 2.5 hours to detect a dead peer. Always tune (Linux/`IPPROTO_TCP`):

```
int idle = 60; // TCP_KEEPIDL: seconds of idle before first probe
int intvl = 10; // TCP_KEEPINTVL: seconds between probes
int cnt = 3; // TCP_KEEPCNT: failed probes before declaring dead
setsockopt(fd, IPPROTO_TCP, TCP_KEEPIDL, &idle, sizeof idle);
setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &intvl, sizeof intvl);
setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &cnt, sizeof cnt);
// Detection in ~90s. The connection then fails with ETIMEDOUT.
```

macOS uses `TCP_KEEPAIVE` for the idle time; other BSDs vary. Guard with `#ifdef`.

Linux also offers `TCP_USER_TIMEOUT`, which is often better: it bounds how long *unacknowledged data* may remain outstanding before the connection is dropped, regardless of idleness.

```
unsigned int ms = 30000;
setsockopt(fd, IPPROTO_TCP, TCP_USER_TIMEOUT, &ms, sizeof ms);
```

Keepalive detects dead *idle* connections; `TCP_USER_TIMEOUT` detects dead *active* ones. They're complementary.

Application-level heartbeats still beat both — they keep NAT mappings alive, prove the application (not just the kernel) is responsive, and work identically on every platform. Use keepalive as a backstop, not a primary mechanism.

`SO_LINGER` — mostly a trap

```
struct linger lg;
lg.l_onoff = 1;
lg.l_linger = 5; // seconds
setsockopt(fd, SOL_SOCKET, SO_LINGER, &lg, sizeof lg);
```

Three configurations:

<code>l_onoff</code>	<code>l_linger</code>	Behavior
0	—	Default. <code>close()</code> returns immediately, kernel flushes in background, sends FIN
1	> 0	<code>close()</code> blocks until data is flushed and ACKed, or timeout → then RST
1	0	<code>close()</code> returns immediately and sends RST — data discarded, no TIME_WAIT

The `l_linger = 0` "abortive close" is occasionally used to avoid TIME_WAIT accumulation. **Don't.** It discards unsent data, the peer gets `ECONNRESET` instead of a clean EOF, and you've traded a manageable resource issue for data loss. If TIME_WAIT is genuinely your bottleneck, fix the design so the *client* closes first.

Blocking-linger has its own problem: `close()` can now block for seconds in the middle of your event loop, and even on success it only proves data reached the peer's *kernel*, not its application. If you need that guarantee, put an acknowledgement in your protocol.

The correct default is to leave `SO_LINGER` alone and use the graceful-close pattern from section 4 (`shutdown(SHUT_WR)`, `drain`, `close`).

4. Buffers and Timeouts

`SO_RCVBUF` / `SO_SNDBUF`

```
int size = 512 * 1024;
setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &size, sizeof size);

int actual; socklen_t len = sizeof actual;
getsockopt(fd, SOL_SOCKET, SO_RCVBUF, &actual, &len);
// Linux: actual == 2 * size (doubled for bookkeeping overhead)
```

Linux doubles the value you set and reports the doubled number back. This surprises everyone once. It's also clamped to `net.core.rmem_max / wmem_max` unless you have `CAP_NET_ADMIN` (then use `SO_RCVBUFFORCE`).

Bandwidth-delay product determines the size you need for full throughput:

```
BDP (bytes) = bandwidth (bytes/sec) × RTT (sec)

1 Gbps, 100 ms RTT → 125 MB/s × 0.1 s = 12.5 MB
```

If the receive buffer is smaller than the BDP, the advertised window caps throughput below line rate. This is why bulk transfers over long-haul links underperform.

Important: on Linux, **leave TCP buffers alone by default**. Auto-tuning (`net.ipv4.tcp_rmem/tcp_wmem`) grows them dynamically and usually does better than a fixed guess. Setting `SO_RCVBUF` explicitly **disables auto-tuning** for that socket. Only set it when you have a specific reason (a known high-BDP path, or deliberately shrinking to limit per-connection memory across 100k connections).

Set before `listen()/connect()` — the window scale factor is negotiated in the SYN and can't change afterward.

For **UDP**, sizing `SO_RCVBUF` generously is usually right — there's no auto-tuning, and overflow means silent datagram loss (section 5). Monitor drops via `netstat -su`.

SO_RCVTIMEO / SO_SNDTIMEO

```
struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof tv);
```

A blocking `recv()` now returns `-1` with `EAGAIN/EWOULDBLOCK` after 5 seconds of no data.

The clean, simple way to add timeouts to blocking code. Caveats:

- **Doesn't apply to `connect()`** on most systems — for connect timeouts use non-blocking + `poll` + `SO_ERROR` (section 3).
- A partial `recv()` resets the timer; the timeout is per-operation, not per-message. So a slow attacker dripping one byte every 4 seconds keeps the connection alive forever. For a real defense you need an absolute deadline in your read loop.

- `{0, 0}` disables the timeout.
- Irrelevant on non-blocking sockets — you're managing timing in the event loop anyway.

5. TCP Latency Options

TCP_NODELAY — disable Nagle

The most commonly needed TCP option after `SO_REUSEADDR`.

```
int yes = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &yes, sizeof yes);
```

Recall the pathology from section 1: Nagle buffers small writes until outstanding data is ACKed; the peer's **delayed ACK** waits up to 40 ms before ACKing. Result: a 40 ms stall on a request/response exchange that should take microseconds.

The canonical trigger is **two writes then a read**:

```
send(fd, header, 16, 0);      // Nagle sends this immediately (nothing
outstanding)
send(fd, body, 200, 0);      // Nagle HOLDS this — waiting for header's ACK
recv(fd, reply, ...);        // peer waits for the body → delayed ACK fires at
40ms
```

Two fixes, and you want **both**:

1. **Set TCP_NODELAY** on every latency-sensitive connection — RPC, games, interactive protocols, anything request/response.
2. **Coalesce writes**. One `send()` (or one `writenv()`) per logical message. This is better engineering regardless: it halves syscall count and removes the problem at the source, not just at the transport.

When to *keep* Nagle: bulk transfer with many small writes you can't easily batch, where throughput matters and latency doesn't.

TCP_CORK (Linux) / TCP_NOPUSH (BSD)

The deliberate inverse: hold data until you uncork or 200 ms elapses, so partial segments never go out.

```
int on = 1;
setsockopt(fd, IPPROTO_TCP, TCP_CORK, &on, sizeof on);

send_all(fd, http_headers, hlen);
sendfile(fd, filefd, NULL, filesize);    // zero-copy body

int off = 0;
setsockopt(fd, IPPROTO_TCP, TCP_CORK, &off, sizeof off);    // flush now
```

This is the standard header+file pattern (nginx does exactly this). It packs the header into the same segment as the start of the file instead of sending a small header segment alone.

`TCP_CORK` and `TCP_NODELAY` are not opposites in a usable sense — corking overrides. Modern Linux allows both set simultaneously; the cork wins while it's on. Prefer `writew()` where you can: it achieves the same coalescing with no state to un-set and no 200 ms failure mode if you forget to uncork.

`TCP_QUICKACK` (Linux)

```
int yes = 1;
setsockopt(fd, IPPROTO_TCP, TCP_QUICKACK, &yes, sizeof yes);
```

Disables delayed ACK — the *other* half of the Nagle pathology. Attacks the problem from the receiver side.

It's not sticky. The kernel resets it after the next ACK is sent, based on its own heuristics. You have to re-set it after each `recv()` if you want persistent behavior, which makes it awkward. Treat it as a niche tool;

`TCP_NODELAY` plus write coalescing solves the same problem more robustly.

Other TCP options worth knowing

```
// Don't wake accept() until actual data arrives (Linux) – saves a wakeup for
// protocols where the client speaks first (HTTP). Value is a timeout in seconds.
int secs = 30;
setsockopt(lfd, IPPROTO_TCP, TCP_DEFER_ACCEPT, &secs, sizeof secs);

// TCP Fast Open – send data in the SYN, saving one RTT on reconnect
int qlen = 5;
setsockopt(lfd, IPPROTO_TCP, TCP_FASTOPEN, &qlen, sizeof qlen); // server
// client: sendto(fd, buf, len, MSG_FASTOPEN, addr, addrlen)

// Read live connection statistics – RTT, retransmits, cwnd, state
struct tcp_info info; socklen_t ilen = sizeof info;
getsockopt(fd, IPPROTO_TCP, TCP_INFO, &info, &ilen);
// info.tcpi_rtt (µs), tcpi_retrans, tcpi_snd_cwnd, tcpi_ca_state
```

`TCP_INFO` is underused and excellent for diagnostics — it lets your application log the actual RTT and retransmission count per connection instead of guessing.

6. `IPV6_V6ONLY` and Dual-Stack

An `AF_INET6` socket can, on most systems, accept IPv4 connections too — they appear as **IPv4-mapped IPv6 addresses** (`::ffff:192.0.2.1`). One socket, both protocols.

```
int fd = socket(AF_INET6, SOCK_STREAM | SOCK_CLOEXEC, 0);
```

```
int no = 0;
setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no); // dual-stack
// or: int yes = 1; → IPv6 only

// ... bind to in6addr_any, listen
```

Always set it explicitly. The default varies: Linux follows `net.ipv6.bindv6only` (0 → dual-stack by default), FreeBSD and Windows default to v6-only. Code that doesn't set it behaves differently across platforms — a genuine portability bug.

Must be set before `bind()`.

Two viable server architectures:

Approach	Pros	Cons
One dual-stack <code>AF_INET6</code> socket	Simple, one accept loop	Mapped addresses complicate logging/ACLs; unavailable where v6only is forced (OpenBSD)
Two sockets (<code>AF_INET</code> + <code>AF_INET6</code> with <code>V6ONLY=1</code>)	Explicit, portable, clean addresses	Two listeners to multiplex

Production servers usually take the second path. Note that if you *do* run two sockets, the IPv6 one **must** have `IPV6_V6ONLY = 1`, or binding the IPv4 socket fails with `EADDRINUSE`.

Handling mapped addresses when logging:

```
if (ss.ss_family == AF_INET6) {
    struct sockaddr_in6 *s6 = (struct sockaddr_in6 *)&ss;
    if (IN6_IS_ADDR_V4MAPPED(&s6->sin6_addr)) {
        // Real client is IPv4; last 4 bytes of sin6_addr hold the v4 address
    }
}
```

This matters for ACLs: a rule matching `192.0.2.0/24` won't match `::ffff:192.0.2.5` unless you normalize first. Firewall bypass bugs live here.

7. `fcntl` — `O_NONBLOCK` and `FD_CLOEXEC`

These live on the file descriptor, not the socket layer, so they use `fcntl` rather than `setsockopt`.

`O_NONBLOCK`

```
int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags < 0) return -1;
    if (flags & O_NONBLOCK) return 0; // already set
}
```

```
    return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}
```

Always read-modify-write. `fcntl(fd, F_SETFL, O_NONBLOCK)` alone clears every other flag (`O_APPEND`, etc.) — a classic bug.

Once set, all operations return immediately:

- `recv()` with no data → `-1`, `EAGAIN/EWOULDBLOCK`
- `send()` with a full buffer → short write, or `-1 EAGAIN`
- `connect()` → `-1`, `EINPROGRESS`
- `accept()` with an empty queue → `-1`, `EAGAIN`

`EAGAIN` and `EWOULDBLOCK` are the same value on Linux but not guaranteed to be by POSIX — check both, or use a macro.

Prefer the atomic forms to avoid a window where the fd is blocking:

```
int fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK | SOCK_CLOEXEC, 0);
int cfd = accept4(lfd, NULL, NULL, SOCK_NONBLOCK | SOCK_CLOEXEC);
```

This is the whole foundation of section 6 — you cannot build an event loop on blocking sockets.

FD_CLOEXEC

Closes the fd automatically on `exec()`. Without it, every fd leaks into every child process you spawn.

```
int set_cloexec(int fd) {
    int flags = fcntl(fd, F_GETFD, 0);           // F_GETFD, not
    F_GETFL                                       F_GETFL
    if (flags < 0) return -1;
    return fcntl(fd, F_SETFD, flags | FD_CLOEXEC);
}
```

Note `F_GETFD/F_SETFD` (file *descriptor* flags) versus `F_GETFL/F_SETFL` (file *status* flags). Mixing them up is a frequent error.

Why it matters, concretely: a web server forks a CGI process. Without `FD_CLOEXEC`, that CGI inherits the listening socket and every other client's connection. It can read other users' traffic, and if it hangs around, it holds the listening port open so your server can't restart. This is a **security bug**, not just hygiene.

Set it on every socket you create. `SOCK_CLOEXEC` / `accept4()` do it atomically, which also closes a race in multithreaded programs where another thread could `fork()` between `socket()` and `fcntl()`.

Putting It Together

```
// Server listening socket
int make_listener(const char *port, int backlog) {
    int fd = socket(AF_INET6, SOCK_STREAM | SOCK_CLOEXEC | SOCK_NONBLOCK, 0);
    if (fd < 0) return -1;

    int yes = 1, no = 0;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes); // mandatory
    setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes); // multi-worker
    setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no); // explicit
    // ... bind, listen
    return fd;
}

// Per-connection setup, after accept4(..., SOCK_NONBLOCK | SOCK_CLOEXEC)
void setup_conn(int fd) {
    int yes = 1;
    setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &yes, sizeof yes); // latency

    setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &yes, sizeof yes);
    int idle = 60, intvl = 10, cnt = 3;
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPIIDLE, &idle, sizeof idle);
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &intvl, sizeof intvl);
    setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &cnt, sizeof cnt);

    unsigned int ut = 30000;
    setsockopt(fd, IPPROTO_TCP, TCP_USER_TIMEOUT, &ut, sizeof ut);
    // Leave SO_RCVBUF/SO_SNDBUF alone — let auto-tuning work
    // Leave SO_LINGER alone — use shutdown/drain/close
}
```

Note what's deliberately *not* set: buffers (auto-tuning is better), linger (default is correct), quickack (unreliable), cork (use `writetv`).

Pitfall Checklist

1. Passing `char` or `bool` where an `int` is required
2. Setting `SO_REUSEADDR` **after** `bind()` — no effect
3. Assuming `SO_REUSEADDR` on Windows means what it means on Unix
4. Not setting `SO_REUSEPORT` on the *first* socket of the group
5. Enabling `SO_KEEPALIVE` without tuning → 2-hour detection
6. `SO_LINGER` with `l_linger = 0` → data loss and RST
7. Expecting `getsockopt(SO_RCVBUF)` to return what you set (Linux doubles it)
8. Setting `SO_RCVBUF` and killing auto-tuning for no reason
9. Setting buffer sizes **after** `connect()/listen()` — window scale already negotiated
10. Expecting `SO_RCVTIMEO` to apply to `connect()`
11. Relying on `SO_RCVTIMEO` for slowloris defense (it resets on partial data)
12. Two writes then a read without `TCP_NODELAY` → 40 ms stall
13. Forgetting to uncork `TCP_CORK` → 200 ms delay

14. Expecting `TCP_QUICKACK` to persist
 15. Not setting `IPV6_V6ONLY` explicitly → different behavior per platform
 16. Running two listeners without `IPV6_V6ONLY=1` on the v6 socket → `EADDRINUSE`
 17. ACLs that don't normalize IPv4-mapped addresses
 18. `fcntl(fd, F_SETFL, O_NONBLOCK)` without preserving existing flags
 19. Confusing `F_GETFD/F_SETFD` with `F_GETFL/F_SETFL`
 20. No `FD_CLOEXEC` → fd leaks into children, security issue
 21. Assuming accepted sockets inherit `O_NONBLOCK` (they don't on Linux)
 22. Ignoring `setsockopt` return values
-

Exercises

1. **Prove `SO_REUSEADDR`.** Start a server, connect a client, kill the *server* first (so it enters `TIME_WAIT`), immediately restart. Without the option: `EADDRINUSE`. With it: instant. Then confirm with `ss -tan | grep TIME-WAIT`.
2. **Measure the Nagle/delayed-ACK stall.** Write a client that does two `send()` calls then a `recv()`, in a loop, timing each round trip. Without `TCP_NODELAY` you should see ~40 ms per iteration. Enable it → microseconds. Then remove `TCP_NODELAY` but merge the two sends into one `writew()` → also fast. This single experiment teaches more about TCP latency than any amount of reading.
3. **Linux buffer doubling.** Set `SO_RCVBUF` to 65536, immediately `getsockopt` it. Explain the number you get.
4. **`SO_REUSEPORT` load distribution.** Fork 4 workers, each with its own listener on the same port. Connect 100 clients and log which PID accepted each. Verify the spread. Then kill one worker mid-flight and observe what happens to connections hashed to it.
5. **Keepalive detection.** Two machines (or a VM). Establish an idle connection with `TCP_KEEPIDLE=5`, `TCP_KEEPINTVL=2`, `TCP_KEEPCNT=2`. Hard-kill the peer's network (`iptables -A INPUT -j DROP`, don't close cleanly). Time how long until `ETIMEDOUT`. Then repeat with default keepalive settings and give up waiting.
6. **`FD_CLOEXEC` leak.** Server with a listening socket forks and `execs /bin/sh -c 'sleep 300'`. Check `ls -l /proc/<child>/fd`. Kill the parent, then try to rebind the port — it fails, because the shell still holds it. Add `SOCK_CLOEXEC` and repeat.
7. **`TCP_INFO` telemetry.** Log `tcpi_rtt` and `tcpi_retrans` per connection while running under `tc qdisc ... netem delay 50ms loss 3%`. Confirm your application can observe the impairment you injected.

Exercise 2 is the one to do first — it's the clearest cause-and-effect demonstration in the entire subject.

Ready for **I/O Models & Multiplexing** when you are.

6. I/O Models & Multiplexing

- Blocking, non-blocking, I/O multiplexing, signal-driven, asynchronous
- `select()` — limits, `FD_SETSIZE`
- `poll()`

- `epoll` (Linux): `epoll_create1`, `epoll_ctl`, `epoll_wait`, level-triggered vs edge-triggered
- `kqueue` (BSD/macOS), IOCP (Windows) for portability awareness
- `io_uring` (modern Linux async I/O)
- Reactor and Proactor patterns
- Event loop design, timer wheels, timeouts
- The C10K / C10M problem

I/O Models & Multiplexing

This is the section that separates toy servers from real ones. Sections 4 and 5 gave you one thread per connection; this one gives you one thread for a hundred thousand.

The central shift: **you stop asking "read this data" and start asking "tell me when this fd is ready."**

1. The Five I/O Models

Stevens' taxonomy. Every I/O operation has two phases: **wait for data to arrive** and **copy it from kernel to userspace**. The models differ in who blocks on which phase.

Blocking I/O

```
recv(fd, buf, len, 0); // returns when data is available
```

Thread sleeps through both phases. Simple, correct, and the reason thread-per-connection exists. One thread per fd is a hard requirement.

Non-blocking I/O (polling)

```
set_nonblocking(fd);
while ((n = recv(fd, buf, len, 0)) < 0 && errno == EAGAIN)
    ; // spin
```

Returns immediately with `EAGAIN` when nothing is ready. **Naive polling like this is worthless** — it burns 100% CPU. Non-blocking mode is only useful *in combination with* readiness notification. That combination is the next model.

I/O Multiplexing

```
poll(fds, n, timeout); // block until ANY of n fds is ready
recv(ready_fd, buf, len, 0); // now guaranteed not to block (mostly)
```

One thread blocks on a readiness query covering many fds. This is `select/poll/epoll/kqueue`, and it's what 95% of high-performance servers use. Phase 1 blocks in the multiplexer; phase 2 is a fast memcpy.

Signal-driven I/O

```
fcntl(fd, F_SETOWN, getpid());
fcntl(fd, F_SETFL, flags | O_ASYNC); // SIGIO on readiness
```

Kernel sends **SIGIO** when the fd is ready. **Essentially obsolete** — signals don't queue (recall the **SIGCHLD** coalescing problem from section 4), so under load you lose notifications and can't tell which fd fired. Know it exists; never use it.

Asynchronous I/O

```
io_submit(...); // "read 4KB into this buffer, tell me when it's DONE"
```

The kernel performs *both* phases and notifies you on completion. You hand over a buffer, not receive one. POSIX AIO is famously broken for sockets (glibc emulates it with threads); Linux **io_uring** is the first genuinely good implementation on Linux. Windows IOCP has done this properly since the 1990s.

Model	Phase 1 (wait)	Phase 2 (copy)	Verdict
Blocking	blocked	blocked	Simple, doesn't scale
Non-blocking	polling	blocked	Only useful with multiplexing
Multiplexing	blocked in select/epoll	blocked	The standard
Signal-driven	not blocked	blocked	Obsolete
Asynchronous	not blocked	not blocked	io_uring , IOCP

The first four are all **synchronous** — you still call **recv()** yourself. Only the last is truly asynchronous.

2. **select()**

The oldest and most portable. Also the worst.

```
int select(int nfd, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);

FD_ZERO(&set); FD_SET(fd, &set); FD_ISSET(fd, &set); FD_CLR(fd, &set);
```

```
fd_set master, working;
FD_ZERO(&master);
FD_SET(lfd, &master);
int maxfd = lfd;
```

```

for (;;) {
    working = master;                                // select DESTROYS the set – copy
    it
    struct timeval tv = { .tv_sec = 1 };             // and destroys the timeout on
    Linux

    int n = select(maxfd + 1, &working, NULL, NULL, &tv);
    if (n < 0) { if (errno == EINTR) continue; break; }

    for (int fd = 0; fd <= maxfd; fd++) {             // O(maxfd) scan every iteration
        if (!FD_ISSET(fd, &working)) continue;
        if (fd == lfd) {
            int cfd = accept(lfd, NULL, NULL);
            FD_SET(cfd, &master);
            if (cfd > maxfd) maxfd = cfd;
        } else {
            handle_readable(fd, &master, &maxfd);
        }
    }
}

```

Four disqualifying problems:

1. **FD_SETSIZE — a hard limit of 1024 fds on Linux.** Not a performance limit, a *correctness* limit: `FD_SET(1500, &set)` writes past the end of the bitmask and corrupts your stack. There's no bounds check. Recompiling with a larger `FD_SETSIZE` is unreliable because libc may have been built with the original value.
2. **O(n) in both directions** — you rebuild the set each iteration, and the kernel scans all `nfds` bits.
3. **Destructive** — sets are modified in place; you must keep a master copy. On Linux the `timeout` is modified too, so it must be reinitialized every loop.
4. **`nfds` is `maxfd + 1`**, not a count. Off-by-one here silently drops your highest fd.

Only reason to use it: maximum portability, few fds, or a sub-millisecond timeout need (`select` takes microseconds; `poll/epoll_wait` take milliseconds — though `ppoll/epoll_pwait2` fix that).

3. `poll()`

Fixes `select`'s correctness problems, not its scaling problem.

```

struct pollfd {
    int fd;      // negative fd → ignored (handy for "disable without
removing")
    short events; // requested: POLLIN, POLLOUT, POLLPRI
    short revents; // returned: above + POLLERR, POLLHUP, POLLNVAL
};
int poll(struct pollfd *fds, nfds_t nfds, int timeout_ms);

```

```

struct pollfd *pfds = calloc(cap, sizeof *pfds);
pfds[0].fd = lfd; pfds[0].events = POLLIN;
nfds_t n = 1;

for (;;) {
    int r = poll(pfds, n, 1000);
    if (r < 0) { if (errno == EINTR) continue; break; }

    for (nfds_t i = 0; i < n; i++) {
        if (!pfds[i].revents) continue;

        // Check errors FIRST – you can't request these, but you always get them
        if (pfds[i].revents & (POLLERR | POLLNVAL)) { close_conn(i); continue; }
        if (pfds[i].revents & POLLHUP) { /* peer closed; may still have buffered
data */ }
        if (pfds[i].revents & POLLIN) { /* readable */ }
        if (pfds[i].revents & POLLOUT) { /* writable */ }
    }
}

```

Improvements over `select`: no fd limit (bounded only by `RLIMIT_NOFILE`), non-destructive (separate `events/revents`, no rebuild), distinguishes error conditions properly.

Still $O(n)$: the whole array crosses the user/kernel boundary on every call, and the kernel scans every entry. With 10,000 connections and 5 active, you copy 10,000 structs to learn about 5. That's the wall.

POLLHUP subtlety: it means the peer hung up, but there may still be buffered data to read. Don't close immediately — `recv()` until it returns 0. `POLLHUP` and `POLLIN` can both be set.

Practical ceiling: a few thousand fds. Use `poll` for moderate connection counts, portability across all Unix, or when you need `poll` on a handful of fds inside otherwise-blocking code.

4. `epoll` (Linux)

The fix: **register interest once, the kernel maintains the set, and `epoll_wait` returns only what's ready.** $O(1)$ with respect to total connections, $O(\text{active})$ per call.

```

int epoll_create1(int flags); // EPOLL_CLOEXEC
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *ev);
int epoll_wait(int epfd, struct epoll_event *evs, int maxevents, int timeout_ms);

struct epoll_event {
    uint32_t      events; // EPOLLIN, EPOLLOUT, EPOLLET, EPOLLONESHOT,
EPOLLRDHUP...
    epoll_data_t data;    // union { void *ptr; int fd; uint32_t u32; uint64_t
u64; }
};

```

Ops: `EPOLL_CTL_ADD`, `EPOLL_CTL_MOD`, `EPOLL_CTL_DEL`.

The `data` union is the key design feature — put a **pointer to your connection struct** there, not a bare fd. That's how you get from "this fd is ready" to "here is its state machine, its buffers, its parser" with zero lookup.

Level-triggered vs Edge-triggered

This is the concept people get wrong.

Level-triggered (default): "is there data *now*?" Reports readiness as long as the condition holds. If you read 100 of 500 buffered bytes, the next `epoll_wait` reports readable again. Behaves like `poll`, just faster. **Safe.**

Edge-triggered (EPOLLET): "did readiness *change*?" Reports only on transitions — when new data arrives. If you read 100 of 500 bytes and stop, `epoll_wait` **will not tell you again** until *more* data arrives. Those 400 bytes sit there forever. If the peer sent its final message and is waiting for a reply, you've deadlocked the connection.

Edge-triggered is only correct if you drain to `EAGAIN`:

```
// EDGE-TRIGGERED READ – the loop is MANDATORY
for (;;) {
    ssize_t n = recv(fd, buf, sizeof buf, 0);
    if (n > 0) { process(buf, n); continue; }           // keep going!
    if (n == 0) { close_conn(c); break; }               // peer closed
    if (errno == EINTR) continue;
    if (errno == EAGAIN || errno == EWOULDBLOCK) break; // NOW we're drained
    close_conn(c); break;                               // real error
}
```

Same for `accept()` — one `epoll_wait` wakeup may represent several queued connections:

```
for (;;) {
    int cfd = accept4(lfd, NULL, NULL, SOCK_NONBLOCK | SOCK_CLOEXEC);
    if (cfd < 0) {
        if (errno == EINTR || errno == ECONNABORTED) continue;
        if (errno == EAGAIN || errno == EWOULDBLOCK) break;
        break;
    }
    add_conn(cfd);
}
```

Edge-triggered requires non-blocking fds. With a blocking fd, the final drain attempt blocks forever and the entire event loop stops.

Level-triggered		Edge-triggered
Must drain fully	No	Yes

Level-triggered		Edge-triggered
Non-blocking required	Recommended	Mandatory
EPOLLOUT behavior	Fires constantly when writable → busy loop unless you MOD it off	Fires once per transition — natural fit
Failure mode	Extra wakeups	Silent hangs
Recommendation	Start here	Use when you've measured a need

The **EPOLLOUT** asymmetry is why people reach for ET. With LT, a socket is almost always writable, so leaving **EPOLLOUT** registered spins your loop at 100% CPU. Solution under LT: **only register EPOLLOUT when you have buffered data that wouldn't fit**, then **EPOLL_CTL_MOD** it off once flushed.

The honest advice: **use level-triggered unless profiling says otherwise**. ET saves a modest number of syscalls and costs you a class of bugs that only appear under load. nginx uses ET; it also has twenty years of hardening.

Other flags

- **EPOLLRDHUP** — peer closed its write side (FIN). Detects half-close without a **recv()** returning 0. Very useful for dropping connections early.
- **EPOLLONESHOT** — disable the fd after one event; you must **EPOLL_CTL_MOD** to rearm. **Essential for multithreaded epoll**: without it, two threads can be handed the same fd simultaneously and race on the connection state.
- **EPOLLEXCLUSIVE** (4.5+) — wake only one waiter for a shared listening fd; the thundering-herd fix for multiple threads on one listener. (**SO_REUSEPORT** from section 6 is the other approach.)
- **EPOLLERR** and **EPOLLHUP** are **always reported** whether or not you request them. Always check them first.

Minimal complete server

```
typedef struct conn {
    int      fd;
    uint8_t  rbuf[8192]; size_t rlen;
    uint8_t  *wbuf;      size_t wlen, wsent;
    bool     want_write;
} conn;

int main(void) {
    signal(SIGPIPE, SIG_IGN);
    int lfd = tcp_listen("8080", SOMAXCONN);      // from section 3
    set_nonblocking(lfd);

    int ep = epoll_create1(EPOLLCLOEXEC);
    struct epoll_event ev = { .events = EPOLLIN, .data.ptr = NULL }; // NULL ==
listener
    epoll_ctl(ep, EPOLL_CTL_ADD, lfd, &ev);
```

```

struct epoll_event events[256];
for (;;) {
    int n = epoll_wait(ep, events, 256, compute_next_timeout_ms());
    if (n < 0) { if (errno == EINTR) continue; break; }

    for (int i = 0; i < n; i++) {
        conn *c = events[i].data.ptr;
        uint32_t e = events[i].events;

        if (c == NULL) { // listener
            for (;;) {
                int cfd = accept4(lfd, NULL, NULL, SOCK_NONBLOCK |
SOCK_CLOEXEC);

                if (cfd < 0) break;
                int yes = 1;
                setsockopt(cfd, IPPROTO_TCP, TCP_NODELAY, &yes, sizeof yes);
                conn *nc = calloc(1, sizeof *nc);
                nc->fd = cfd;
                struct epoll_event cev = { .events = EPOLLIN | EPOLLRDHUP,
                                           .data.ptr = nc };
                epoll_ctl(ep, EPOLL_CTL_ADD, cfd, &cev);
            }
            continue;
        }

        if (e & (EPOLLERR | EPOLLHUP)) { close_conn(ep, c); continue; }
        if (e & EPOLLIN) { if (on_readable(ep, c) < 0) { close_conn(ep, c);
continue; } }
        if (e & EPOLLOUT) { if (on_writable(ep, c) < 0) { close_conn(ep, c);
continue; } }
    }
    expire_timers();
}
}

```

The write path — where event loops actually get hard

`send_all()` from section 4 **cannot be used here**. Looping until everything is sent blocks the entire loop on one slow client. Instead: send what you can, buffer the rest, register for writability.

```

int try_flush(int ep, conn *c) {
    while (c->wsent < c->wlen) {
        ssize_t n = send(c->fd, c->wbuf + c->wsent, c->wlen - c->wsent,
MSG_NOSIGNAL);
        if (n > 0) { c->wsent += (size_t)n; continue; }
        if (n < 0 && errno == EINTR) continue;
        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            if (!c->want_write) { // arm EPOLLOUT
                c->want_write = true;
                struct epoll_event ev = { .events = EPOLLIN | EPOLLOUT |

```



```

    EPOLLRDHUP,
                                .data.ptr = c };
    epoll_ctl(ep, EPOLL_CTL_MOD, c->fd, &ev);
}
    return 0; // partial – will resume
later
}
    return -1;
}
// Fully flushed – disarm EPOLLOUT or you'll spin (level-triggered)
if (c->want_write) {
    c->want_write = false;
    struct epoll_event ev = { .events = EPOLLIN | EPOLLRDHUP, .data.ptr = c };
    epoll_ctl(ep, EPOLL_CTL_MOD, c->fd, &ev);
}
    free(c->wbuf); c->wbuf = NULL; c->wlen = c->wsent = 0;
    return 0;
}

```

Backpressure: if `wlen` grows past some cap (say 4 MB), the client isn't reading. Stop producing, or disconnect. Otherwise a single slow client OOMs your server — this is the memory-side equivalent of the flow control TCP gave you for free in the blocking model.

5. `kqueue`, IOCP, and Portability

`kqueue` (BSD, macOS)

```

int kq = kqueue();
struct kevent ev;
EV_SET(&ev, fd, EVFILT_READ, EV_ADD | EV_ENABLE, 0, 0, conn_ptr);
kevent(kq, &ev, 1, NULL, 0, NULL); // register
int n = kevent(kq, NULL, 0, events, 64, &timeout); // wait

```

Arguably better designed than `epoll`:

- **One syscall** registers and waits (batched changelist + eventlist)
- **Not just fds** — `EVFILT_TIMER`, `EVFILT_SIGNAL`, `EVFILT_VNODE` (file changes), `EVFILT_PROC` (process exit) all flow through the same loop. On Linux you need `timerfd`, `signalfd`, and `inotify` to achieve the same thing.
- `udata` field serves the same role as `epoll`'s `data.ptr`
- `EV_CLEAR` = edge-triggered; `EV_ONESHOT`, `EV_DISPATCH` for the oneshot pattern

IOCP (Windows)

Completion-based, not readiness-based — the fundamental split. You submit an operation with a buffer; the kernel notifies you when it's *finished*.

```
WSARecv(sock, &wsabuf, 1, NULL, &flags, &overlapped, NULL); // start
GetQueuedCompletionStatus(iocp, &bytes, &key, &povl, timeout); // completed
```

Built-in thread pool, scales excellently. But the programming model is incompatible enough with readiness-based APIs that portable code needs a real abstraction layer — you can't just `#ifdef` your way across.

The portability answer

Don't write your own abstraction. **Use libuv, libevent, or Boost.Asio** (section 11). They exist precisely because `epoll/kqueue/IOCP` don't unify cleanly. Write your own event loop once as an exercise, then use a library in production.

6. `io_uring` (Linux 5.1+)

True async I/O on Linux, finally. Two shared ring buffers between userspace and kernel: a **submission queue** (SQ) and a **completion queue** (CQ). You write entries, the kernel consumes them, you read results — often with **zero syscalls** in the steady state.

```
#include <liburing.h>

struct io_uring ring;
io_uring_queue_init(256, &ring, 0);

struct io_uring_sq *sqe = io_uring_get_sqe(&ring);
io_uring_prep_recv(sqe, fd, buf, len, 0);
io_uring_sqe_set_data(sqe, conn);          // your pointer, same idea as epoll
io_uring_submit(&ring);

struct io_uring_cqe *cqe;
io_uring_wait_cqe(&ring, &cqe);
conn *c = io_uring_cqe_get_data(cqe);
int result = cqe->res;                     // bytes transferred, or -errno
io_uring_cqe_seen(&ring, cqe);
```

Advantages: batched submission (dozens of ops per syscall), works for *files* as well as sockets (`epoll` doesn't do regular file I/O usefully), supports `accept/connect/send/recv/openat/close/fsync`, linked operations, and `IORING_SETUP_SQPOLL` for a kernel polling thread that eliminates submission syscalls entirely.

Costs: Linux-only, needs a recent kernel, `liburing` dependency, a genuinely different programming model (you own the buffer until completion — lifetime bugs are easy), and it has had a rough security history that leads some environments to disable it via `seccomp`.

Use it when you're on modern Linux, chasing maximum performance, and mixing file and socket I/O.

Otherwise `epoll` is fine — it's not the bottleneck in most servers.

7. Reactor and Proactor

Reactor — readiness-based. "Tell me when I *can* read, then I read."

```
epoll_wait() → fd readable → your handler calls recv() → process
```

epoll, kqueue, libevent, libuv (on Unix), Netty. This is what you build with sections 1–6.

Proactor — completion-based. "Read into this buffer and tell me when it's *done*."

```
submit read + buffer → kernel does everything → completion handler gets filled  
buffer
```

IOCP, io_uring, Boost.Asio's Windows backend.

The practical difference is **buffer ownership**. Reactor: allocate a buffer when the event fires, use it, release it — so N connections need N buffers only at handling time. Proactor: the buffer must be allocated *before* submission and live until completion — so 100k pending reads means 100k live buffers. That's a real memory-design consequence, and it's why io_uring added provided-buffer rings.

Boost.Asio presents a proactor interface everywhere, emulating it over epoll on Linux. That's the standard way to get one API across both worlds.

8. Event Loop Design

Structure

```
loop:
    timeout = min(next_timer_expiry - now, MAX_WAIT)
    events = epoll_wait(ep, timeout)
    for each event: dispatch to connection handler
    expire_timers()
    process_deferred_closes()
```

Connection state machines

Handlers must never block. That means no blocking I/O, no `getaddrinfo` (section 3 — it blocks for seconds), no `sleep`, no unbounded loops, no synchronous disk reads. Anything blocking goes to a thread pool, with the result delivered back via an `eventfd` or a pipe registered in the loop.

Each connection is a state machine:

```
enum state { READING_HEADER, READING_BODY, PROCESSING, WRITING, CLOSING };
```

The handler advances one step per event and returns. This is why framing (section 4) matters so much here: you can't "read until you have a full message" — you accumulate into a buffer and check whether a complete message is present yet.

Deferred close — the bug everyone hits

Never `free()` a connection inside the event dispatch loop. `epoll_wait` returned an array; a later entry may reference the same connection you just freed. Use-after-free, intermittent, hard to reproduce.

```
// Mark, don't free
c->closing = true;
epoll_ctl(ep, EPOLL_CTL_DEL, c->fd, NULL);
list_add(&pending_close, c);
// ... after the dispatch loop completes:
process_deferred_closes();
```

Alternatively check `if (c->closing) continue;` at the top of each dispatch. Do both.

Timers

Every connection needs a timeout (idle, handshake, slowloris defense). Three implementations:

Structure	Add	Delete	Expire	Use when
Sorted list	O(n)	O(n)	O(1)	Toy
Binary heap	O(log n)	O(log n)	O(log n)	Default choice
Timer wheel	O(1)	O(1)	O(1) amortized	100k+ timers

A **timer wheel** is an array of buckets indexed by `(expiry_time / tick) % nbuckets` — a hash table keyed on time. Each tick, you process one bucket. Hierarchical wheels (like a clock's hour/minute/second hands) cascade between levels to cover long ranges. This is what the Linux kernel and nginx use.

```
#define WHEEL_SIZE 512
#define TICK_MS    100

typedef struct timer { struct timer *next, *prev; int64_t expire_ms;
                      void (*cb)(void *); void *arg; } timer;
timer *wheel[WHEEL_SIZE];
size_t cursor;

void timer_add(timer *t, int64_t delay_ms) {
    t->expire_ms = now_ms() + delay_ms;
    size_t slot = (cursor + (size_t)(delay_ms / TICK_MS)) % WHEEL_SIZE;
    list_insert(&wheel[slot], t);
}
```

Practical shortcut for the common case: **an idle-timeout LRU list**. Connections are naturally sorted by last-activity time, so touching a connection moves it to the tail, and expiring means walking from the head until you hit a non-expired entry. $O(1)$ for everything, no wheel needed. nginx does this.

Use `CLOCK_MONOTONIC` for all timer math — `CLOCK_REALTIME` jumps when NTP adjusts and your timeouts go haywire.

```
int64_t now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (int64_t)ts.tv_sec * 1000 + ts.tv_nsec / 1000000;
}
```

Signals in an event loop

Signal handlers can't safely touch loop state. Two clean solutions:

```
// signalfd — signals become readable events (Linux)
sigset_t mask;
sigemptyset(&mask); sigaddset(&mask, SIGTERM); sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, NULL); // must block them first
int sfd = signalfd(-1, &mask, SFD_NONBLOCK | SFD_CLOEXEC);
epoll_ctl(ep, EPOLL_CTL_ADD, sfd, &ev);

// self-pipe trick — portable equivalent
// handler does write(pipe_wfd, "x", 1); loop reads it
```

`eventfd` is the same idea for cross-thread wakeups — how a worker thread tells the loop "I finished your task."

Scaling to multiple cores

One event loop saturates one core. Options:

1. **SO_REUSEPORT** (section 6) — N processes/threads, each with its own listener and its own epoll. Kernel load-balances. Shared-nothing, no locks. **Simplest and usually best.**
2. **One acceptor, N loops** — acceptor distributes fds to worker loops round-robin. Needs cross-thread wakeup (`eventfd`).
3. **Shared epoll across threads** — all threads call `epoll_wait` on the same epfd. **Requires `EPOLLONESHOT`** or two threads race on one connection. Also needs `EPOLLEXCLUSIVE` to avoid thundering herd.

9. C10K and C10M

C10K (Dan Kegel, 1999): can one server handle 10,000 concurrent connections? At the time, no — thread-per-connection at 10k threads meant gigabytes of stacks and a scheduler in meltdown, and `select/poll` were

$O(n)$ per iteration on top of it.

The answer was this section: non-blocking I/O + $O(1)$ readiness notification ([epoll/kqueue](#)) + event loops. **C10K is solved.** A modest server does 10k connections without effort.

Practical requirements at 10k:

```
ulimit -n 100000                                # RLIMIT_NOFILE — the first wall
sysctl -w net.core.somaxconn=65535              # accept queue
sysctl -w net.ipv4.ip_local_port_range="1024 65535"
sysctl -w net.core.netdev_max_backlog=65535
```

Plus: don't allocate 8 KB buffers per idle connection ($10k \times 8 \text{ KB} \times 2 = 160 \text{ MB}$ of mostly-empty buffers — allocate on demand or use a shared pool).

C10M (10 million connections) is a different problem. The bottleneck moves from your code to the kernel itself: per-packet syscall overhead, the network stack's per-packet allocations, interrupt handling, cache misses walking connection tables.

Solutions live below the socket API:

- **Kernel bypass** — DPDK, netmap: userspace drives the NIC directly, kernel not involved
- **XDP/eBPF** — run filtering/routing code in the kernel at the driver level, before [sk_buff](#) allocation
- **Userspace TCP stacks** — mTCP, F-Stack, Seastar
- **RSS + CPU pinning** — NIC hashes flows to queues, each queue pinned to a core, shared-nothing all the way down

Realistically: C10M is for load balancers, DDoS scrubbers, and trading systems. If you're building a normal service, [epoll](#) and a thread per core gets you further than you probably need.

Choosing

Few connections, simple code, portability paramount	→ blocking + threads
Moderate (<1000), portable across all Unix	→ poll()
Many connections, Linux	→ epoll (level-triggered)
Many connections, BSD/macOS	→ kqueue
Cross-platform, don't want to write this	→ libuv / libevent / Asio
Maximum Linux performance, mixed file+socket I/O	→ io_uring
Millions of connections / packet processing	→ DPDK / XDP

Default recommendation for a Linux server you're writing today: **[epoll](#), level-triggered, one loop per core via [SO_REUSEPORT](#), heap or LRU timers, non-blocking everything.** Move to edge-triggered or [io_uring](#) only after profiling.

Pitfall Checklist

1. `FD_SET` with `fd ≥ FD_SETSIZE` → stack corruption, no warning
 2. Not copying the `fd_set` before `select` (it's destroyed)
 3. Not reinitializing `select`'s timeout on Linux
 4. Passing a count instead of `maxfd + 1` to `select`
 5. **Edge-triggered without draining to `EAGAIN`** → silent hangs
 6. Edge-triggered on a blocking fd → whole loop deadlocks
 7. Not looping `accept()` under ET → connections stuck in the queue
 8. Leaving `EPOLLOUT` registered under LT → 100% CPU spin
 9. Using `send_all()` inside an event loop → one slow client blocks everyone
 10. No write-buffer cap → slow client OOMs the server
 11. Freeing a connection inside the dispatch loop → use-after-free from a later event in the same batch
 12. Blocking calls in handlers (`getaddrinfo`, disk I/O, DNS, `sleep`)
 13. Not checking `EPOLLERR/EPOLLHUP` (delivered whether requested or not)
 14. Closing on `POLLHUP/EPOLLRDHUP` without draining remaining data
 15. Multithreaded epoll without `EPOLLONESHOT` → two threads, one connection
 16. `CLOCK_REALTIME` for timers → NTP jumps break timeouts
 17. Ignoring `EINTR` from `epoll_wait/poll` (they never auto-restart, even with `SA_RESTART`)
 18. Signal handlers touching loop state instead of `signalfd/self-pipe`
 19. Forgetting `ulimit -n` → `EMFILE` at 1024 connections
 20. Busy-looping on `EMFILE` instead of shedding load
-

Exercises

1. **Prove the `select` limit.** Open 1100 fds, `FD_SET` the highest, watch it corrupt memory (run under ASan — it will catch it immediately). This is the most visceral demonstration of why `select` is dead.
2. **Port the chat server** from section 4 through all three: `select` → `poll` → `epoll`. Same protocol, same framing. Notice the code gets *shorter*, not longer, with epoll — the `data.ptr` union removes an entire fd→connection lookup.
3. **Demonstrate the ET hang.** Edge-triggered echo server that does exactly one `recv()` of 64 bytes per event. Client sends 1 KB in one write. Server reads 64 bytes and hangs forever. Add the drain loop; watch it work. **Do this one** — it's the clearest lesson in the section.
4. **Demonstrate the LT `EPOLLOUT` spin.** Register `EPOLLIN | EPOLLOUT` permanently, run `top`. 100% CPU with zero traffic. Then implement arm/disarm.
5. **Slow-client backpressure.** Server sends a 100 MB response. Client connects, reads 1 KB, then sleeps forever. Watch your write buffer grow without bound. Add a 4 MB cap and a disconnect policy.
6. **10k connections.** Write a client that opens 10,000 connections from one process (you'll need multiple source ports and a raised `ulimit`). Measure your server's RSS and idle CPU. Then measure the thread-per-connection version from section 4 at the same load, and compare.
7. **Timer wheel.** Add a 30-second idle timeout to your epoll server, first with a linear scan over all connections each tick, then with an LRU list. At 10k connections, measure the difference.

8. **signalfd shutdown**. Make Ctrl-C drain connections gracefully through the event loop rather than dying mid-request.

Exercise 3 first, then 2. Everything after this section assumes you have a working event loop.

Ready for **Advanced I/O** when you are.

7. Advanced I/O

- Scatter/gather: `readv`, `writew`, `sendmsg`, `recvmsg`
- Zero-copy: `sendfile`, `splice`, `mmap`
- Ancillary data / `cmsg`, passing file descriptors over Unix sockets
- Unix domain sockets (`AF_UNIX`), abstract sockets
- Raw sockets (`SOCK_RAW`), packet sockets (`AF_PACKET`)
- Custom checksum computation, hand-crafting IP/TCP headers
- `libpcap` for packet capture and sniffing

Advanced I/O

Two themes here. The first half is about **doing more per syscall** — batching, avoiding copies, moving data without touching it. The second half is about **going below the socket abstraction** — building packets by hand and reading them off the wire.

1. Scatter/Gather I/O

`readv` / `writew`

```
struct iovec { void *iov_base; size_t iov_len; };

ssize_t readv(int fd, const struct iovec *iov, int iovcnt);
ssize_t writew(int fd, const struct iovec *iov, int iovcnt);
```

One syscall, multiple non-contiguous buffers. **Atomic with respect to other writers** — the segments go out contiguously, never interleaved with another thread's write.

This is the correct answer to the header+body problem from section 6:

```
// BAD: two syscalls, and Nagle holds the body waiting for the header's ACK
send(fd, hdr, 12, 0);
send(fd, body, len, 0);

// BAD: correct on the wire, but an allocation and a memcpy per message
uint8_t *tmp = malloc(12 + len);
memcpy(tmp, hdr, 12); memcpy(tmp + 12, body, len);
send(fd, tmp, 12 + len, 0);
free(tmp);
```



```
// GOOD: one syscall, one segment, zero copies
struct iovec iov[2] = {
    { .iov_base = hdr,          .iov_len = 12 },
    { .iov_base = (void *)body, .iov_len = len }
};
writev(fd, iov, 2);
```

That's the length-prefix framing from section 4, done right. It also makes `TCP_CORK` unnecessary for this pattern.

writev can still return short. Same rule as `send`. The iovec-advance loop from section 4:

```
ssize_t writev_all(int fd, struct iovec *iov, int cnt) {
    while (cnt > 0) {
        ssize_t n = writev(fd, iov, cnt);
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        while (cnt > 0 && (size_t)n >= iov->iov_len) {
            n -= (ssize_t)iov->iov_len;
            iov++; cnt--;
        }
        if (cnt > 0 && n > 0) {
            iov->iov_base = (char *)iov->iov_base + n;
            iov->iov_len -= (size_t)n;
        }
    }
    return 0;
}
```

Note it **mutates the caller's iovec array** — document that, or copy first. Bug source.

`IOV_MAX` (typically 1024) caps `iovcnt`; exceeding it gives `EINVAL`.

readv's classic use is a fixed header plus a body, read in one syscall:

```
struct iovec iov[2] = { {hdr, 12}, {body, sizeof body} };
ssize_t n = readv(fd, iov, 2);
```

Careful though — on a stream socket this can return fewer bytes than the header, so you still need the partial-read logic. `readv` saves syscalls; it doesn't repeal TCP.

sendmsg / recvmsg

The most general form. Everything else is a special case of these.

```
struct msghdr {
    void          *msg_name;          // sockaddr (UDP destination/source)
```

```

socklen_t    msg_namelen;
struct iovec *msg_iov;           // scatter/gather array
size_t       msg_iovlen;
void         *msg_control;       // ancillary data (section 3)
size_t       msg_controllen;
int          msg_flags;         // OUT: MSG_TRUNC, MSG_CTRUNC, MSG_EOR...
};

ssize_t sendmsg(int fd, const struct msghdr *msg, int flags);
ssize_t recvmsg(int fd, struct msghdr *msg, int flags);

```

Capabilities beyond `sendto/writev`:

- **Ancillary data** — file descriptors, credentials, packet metadata
- **msg_flags on receive** — the only way to detect `MSG_TRUNC` on UDP (section 5)
- Address + scatter/gather + control data in one call

```

// UDP receive with truncation detection
struct iovec iov = { .iov_base = buf, .iov_len = sizeof buf };
struct sockaddr_storage src;
struct msghdr msg = {
    .msg_name = &src, .msg_namelen = sizeof src,
    .msg_iov = &iov, .msg_iovlen = 1,
};
ssize_t n = recvmsg(fd, &msg, 0);
if (msg.msg_flags & MSG_TRUNC) { /* datagram was larger – data lost */ }

```

`sendmmsg / recvmmsg` (Linux) batch *multiple datagrams* per syscall — a large win for high-rate UDP:

```

struct mmsghdr msgs[64]; // each contains a struct msghdr + unsigned msg_len
int n = recvmmsg(fd, msgs, 64, MSG_WAITFORONE, NULL);
for (int i = 0; i < n; i++) handle(msgs[i].msg_hdr, msgs[i].msg_len);

```

Going from 1 datagram/syscall to 64 can double or triple throughput on a busy UDP server. `recvmmsg`'s timeout argument is famously broken (it's only checked between datagrams) — pass `NULL` and use the event loop's timeout instead.

2. Zero-Copy

The normal path for serving a file is four copies and two context switches:

```

disk → kernel page cache → user buffer → kernel socket buffer → NIC
  (read syscall)           (write syscall)

```

The two middle copies are pure waste — you never look at the bytes.

sendfile

```
ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);
```

Copies between file descriptors **entirely inside the kernel**.

```
off_t off = 0;
size_t remaining = filesize;
while (remaining > 0) {
    ssize_t n = sendfile(sock, filefd, &off, remaining);
    if (n < 0) {
        if (errno == EINTR) continue;
        if (errno == EAGAIN) { /* non-blocking: arm EPOLLOUT, resume later */
            break; }
        return -1;
    }
    if (n == 0) break;
    remaining -= (size_t)n;
}
```

Constraints on Linux: `in_fd` must support `mmap` (a regular file, not a socket or pipe); `out_fd` can be anything since 2.6.33 (was socket-only before). The `offset` pointer is updated in place; pass `NULL` to use and advance the file's own position.

The header+file pattern, combining sections 6 and this one:

```
int on = 1; setsockopt(fd, IPPROTO_TCP, TCP_CORK, &on, sizeof on);
writev_all(fd, hdr_iov, 1); // HTTP headers
sendfile(fd, filefd, &off, filesize); // body, zero-copy
int off_ = 0; setsockopt(fd, IPPROTO_TCP, TCP_CORK, &off_, sizeof off_);
```

That's nginx's static-file path, essentially verbatim.

When `sendfile` doesn't help: if you need to transform the data (compress, encrypt, template), you must have it in userspace. TLS breaks zero-copy for this reason — unless you use **kTLS** (`setsockopt(fd, SOL_TCP, TCP_ULP, "tls", 4)`), which moves encryption into the kernel and restores `sendfile`.

Portability: macOS and FreeBSD have `sendfile` with different signatures. Not POSIX.

splice / tee / vmsplice

```
ssize_t splice(int fd_in, off_t *off_in, int fd_out, off_t *off_out,
               size_t len, unsigned int flags);
```

Moves data between fds via a **pipe buffer**, passing page references rather than bytes. At least one of `fd_in/fd_out` must be a pipe — that's the constraint that shapes all `splice` code.

Socket-to-socket relay (a proxy) needs an intermediate pipe:

```
int pipefd[2];
pipe2(pipefd, O_NONBLOCK);

for (;;) {
    ssize_t n = splice(src, NULL, pipefd[1], NULL, 65536,
                      SPLICE_F_MOVE | SPLICE_F_MORE | SPLICE_F_NONBLOCK);
    if (n <= 0) break;
    while (n > 0) {
        ssize_t m = splice(pipefd[0], NULL, dst, NULL, (size_t)n,
                          SPLICE_F_MOVE | SPLICE_F_MORE | SPLICE_F_NONBLOCK);
        if (m <= 0) break;
        n -= m;
    }
}
```

This is how a TCP proxy moves bytes without ever touching them. Note the inner loop — the second `splice` can move less than the first delivered, and pipe capacity (64 KB default, adjustable with `F_SETPIPE_SZ`) bounds each round.

`tee` duplicates pipe data without consuming it (useful for tapping a stream). `vmsplice` maps user pages into a pipe — powerful and dangerous, since the kernel may hold references to your pages after the call returns; modifying them is a use-after-free class bug.

mmap

```
void *p = mmap(NULL, filesize, PROT_READ, MAP_PRIVATE, filefd, 0);
send(sock, p, filesize, 0);
munmap(p, filesize);
```

Eliminates the read copy but not the write copy. Rarely better than `sendfile` for serving files.

Real dangers: if the file is truncated while mapped, touching the removed pages raises `SIGBUS` and kills your process. Page faults are synchronous and invisible — a "memory access" can block on disk I/O for milliseconds inside your event loop, which is exactly the blocking-in-a-handler problem from section 7 with no syscall to point at. Setup cost (page table manipulation, TLB shootdowns across cores) makes it a loss for small files.

Use `mmap` for random access to large files, not for streaming.

Choosing

Goal	Tool
File → socket	<code>sendfile</code> (+ <code>TCP_CORK</code> for headers)
Socket → socket (proxy)	<code>splice</code> via a pipe
Header + body from memory	<code>writew</code>
Random access to a big file	<code>mmap</code>
Need to transform the data	Nothing — you need it in userspace
Encrypted file serving	kTLS + <code>sendfile</code>

Measure before reaching for these. Zero-copy wins big on large transfers (hundreds of KB up); below ~64 KB the setup overhead often exceeds the savings.

3. Ancillary Data and FD Passing

`msg_control` carries out-of-band metadata alongside the payload. Its most remarkable use: **sending an open file descriptor to another process.**

Not the *number* — the actual open file description. The receiver gets a working fd pointing at the same socket, file, or pipe, with an arbitrary (usually different) number. This is how privilege-separated servers work: a root process binds port 443 and hands the listening socket to an unprivileged worker.

The `cmsg` macros

Control data is a sequence of variable-length `cmsghdr` structures with alignment padding. **Always use the macros** — manual pointer arithmetic here is wrong on some architectures:

```

MSG_FIRSTHDR(msg)      MSG_NXTHDR(msg, cmsg)
MSG_DATA(cmsg)          // pointer to the payload
MSG_LEN(size)           // header + data, for cmsg_len
MSG_SPACE(size)         // header + data + padding, for buffer sizing

```

`MSG_LEN` goes in `cmsg_len`; `MSG_SPACE` sizes your buffer. Confusing them is the classic bug.

Sending an fd

```

int send_fd(int sock, int fd_to_send, const void *payload, size_t plen) {
    union {
        char buf[MSG_SPACE(sizeof(int))];
        struct cmsghdr align;
    } u = {0};

    struct iovec iov = { .iov_base = (void *)payload, .iov_len = plen };
    struct msghdr msg = {
        .msg_iov = &iov, .msg_iovlen = 1,
        .msg_control = u.buf, .msg_controllen = sizeof u.buf,

```

```

};

struct cmsghdr *cm = CMSG_FIRSTHDR(&msg);
cm->cmsg_level = SOL_SOCKET;
cm->cmsg_type = SCM_RIGHTS;
cm->cmsg_len = CMSG_LEN(sizeof(int));
memcpy(CMSG_DATA(cm), &fd_to_send, sizeof(int)); // memcpy, not a cast –
alignment

return sendmsg(sock, &msg, 0) < 0 ? -1 : 0;
}

```

Receiving an fd

```

int recv_fd(int sock, void *payload, size_t plen, ssize_t *out_n) {
    union { char buf[CMSG_SPACE(sizeof(int))]; struct cmsghdr align; } u;

    struct iovec iov = { .iov_base = payload, .iov_len = plen };
    struct msghdr msg = {
        .msg_iov = &iov, .msg_iovlen = 1,
        .msg_control = u.buf, .msg_controllen = sizeof u.buf,
    };

    ssize_t n = recvmsg(sock, &msg, MSG_CMSG_CLOEXEC); // atomic CLOEXEC
    if (n < 0) return -1;
    if (out_n) *out_n = n;

    if (msg.msg_flags & MSG_CTRUNC) return -1; // buffer too small → FD LEAKED

    for (struct cmsghdr *cm = CMSG_FIRSTHDR(&msg); cm; cm = CMSG_NXTHDR(&msg, cm))
    {
        if (cm->cmsg_level == SOL_SOCKET && cm->cmsg_type == SCM_RIGHTS &&
            cm->cmsg_len == CMSG_LEN(sizeof(int))) {
            int fd;
            memcpy(&fd, CMSG_DATA(cm), sizeof fd);
            return fd;
        }
    }
    errno = ENOENT;
    return -1;
}

```

Critical rules:

1. **Always send at least one byte of real payload.** A zero-length `sendmsg` may not deliver the ancillary data at all, and the receiver can't distinguish it from EOF.
2. **MSG_CTRUNC means you leaked an fd.** The kernel already installed the descriptor in your process; if your control buffer was too small, the number is discarded and the fd is unreachable and unclosable. Size the buffer correctly and always check the flag.

3. Use `MSG_CMSG_CLOEXEC` — sets `FD_CLOEXEC` atomically, closing the same race `SOCK_CLOEXEC` closes (section 6).
4. **Only works over `AF_UNIX`**. There's no way to do this over TCP — the fd is a kernel object, not data.
5. **Received fds are subject to your `RLIMIT_NOFILE`**. A malicious peer spamming `SCM_RIGHTS` can exhaust your fd table.

`SCM_CREDENTIALS` — peer authentication

```
int yes = 1;
setsockopt(sock, SOL_SOCKET, SO_PASSCRED, &yes, sizeof yes);
// then read a cmsg with cmsg_type == SCM_CREDENTIALS → struct ucred {pid, uid,
gid}
```

The kernel fills these in and **cannot be spoofed** — genuine authentication for local IPC, no passwords needed. Simpler alternative on Linux:

```
struct ucred cr; socklen_t len = sizeof cr;
getsockopt(sock, SOL_SOCKET, SO_PEERCREC, &cr, &len);
```

This is how systemd, D-Bus, and Docker's socket authenticate clients. (BSD/macOS: `LOCAL_PEERCREC` / `getpeereid`.)

Other ancillary uses

- `IP_PKTINFO` / `IPV6_PKTINFO` — which local address and interface a UDP datagram arrived on.
Essential for multi-homed UDP servers: without it, your reply may go out the wrong interface with the wrong source address, and the client drops it as unsolicited. DNS servers must handle this.
- `IP_RECVTTL`, `IP_RECVTOS` — per-packet metadata
- `SO_TIMESTAMPING` — hardware NIC timestamps, for latency measurement

4. Unix Domain Sockets

Same API, no network. Communication stays entirely within the kernel — no checksums, no protocol headers, no routing.

2–5× faster than loopback TCP, with filesystem permissions for access control and `SO_PEERCREC` for authentication. If both processes are on the same machine, this is almost always the right choice.

```
int uds_listen(const char *path) {
    int fd = socket(AF_UNIX, SOCK_STREAM | SOCK_CLOEXEC, 0);

    struct sockaddr_un addr = {0};
    addr.sun_family = AF_UNIX;
    if (strlen(path) >= sizeof addr.sun_path) { errno = ENAMETOOLONG; return -1; }
    strncpy(addr.sun_path, path, sizeof addr.sun_path - 1);
```

```
    unlink(path); // stale socket file blocks bind()
    if (bind(fd, (struct sockaddr *)&addr, sizeof addr) < 0) { close(fd); return
-1; }
    chmod(path, 0660); // THIS is your access control
    listen(fd, SOMAXCONN);
    return fd;
}
```

Key differences from TCP:

	Unix sockets	TCP loopback
Addressing	Filesystem path (108 bytes max)	IP:port
Access control	File permissions + <code>SO_PEERCRED</code>	Firewall only
FD passing	Yes	No
Speed	2–5× faster	Baseline
Cleanup	Manual <code>unlink</code>	Automatic
<code>SOCK_DGRAM</code>	Reliable and ordered	N/A

That last row surprises people: `AF_UNIX` + `SOCK_DGRAM` gives you message boundaries *with* reliability — no loss, no reordering, no duplication. It's the ideal local IPC primitive when you want datagrams without reimplementing section 5. `SOCK_SEQPACKET` gives you connection-oriented reliable datagrams.

Cleanup is your job. The socket file survives process death. Not `unlinking` before `bind` gives `EADDRINUSE` forever — the `SO_REUSEADDR` of Unix sockets is just `unlink()`.

Abstract sockets (Linux only)

```
struct sockaddr_un addr = {0};
addr.sun_family = AF_UNIX;
addr.sun_path[0] = '\0'; // leading NUL = abstract
memcpy(addr.sun_path + 1, "my-service", 10);
socklen_t len = offsetof(struct sockaddr_un, sun_path) + 1 + 10; // exact length!
bind(fd, (struct sockaddr *)&addr, len);
```

No filesystem entry, disappears automatically when the last reference closes, works in read-only filesystems. **But no file permissions** — anyone in the network namespace can connect, so you *must* use `SO_PEERCRED` for authorization. The `addrlen` must be computed exactly; passing `sizeof(addr)` creates a name padded with NULs.

5. Raw Sockets and Packet Sockets

Below the transport layer. **All of this requires root or `CAP_NET_RAW`.**

SOCK_RAW — you build the IP payload

```
int fd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP); // kernel writes the IP header

int one = 1;
setsockopt(fd, IPPROTO_IP, IP_HDRINCL, &one, sizeof one); // now YOU write it too
```

IP_HDRINCL off: you supply the transport payload, kernel adds the IP header. On: you supply everything from the IP header down.

Receiving raw IPv4 sockets **always include the IP header** regardless of **IP_HDRINCL**. Raw IPv6 sockets never do. Inconsistent, and a common source of off-by-20 parsing bugs.

AF_PACKET — layer 2 (Linux)

```
int fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL)); // full Ethernet frames
// SOCK_DGRAM instead of SOCK_RAW strips the link header for you
```

htons() on the protocol argument is required and universally forgotten.

For real capture volume, use **PACKET_MMAP** (**PACKET_RX_RING**) — a shared ring buffer between kernel and userspace, so you read packets without a syscall each. Or **PACKET_FANOUT** to spread capture across threads. At this point **libpcap** (section 7) does it better than you will.

Hand-crafting headers

```
struct ipv4_hdr {
    uint8_t ver_ihl;           // (4 << 4) | 5
    uint8_t tos;
    uint16_t tot_len;          // network order
    uint16_t id;
    uint16_t frag_off;
    uint8_t ttl;
    uint8_t proto;
    uint16_t check;
    uint32_t saddr, daddr;
} __attribute__((packed));
```

__attribute__((packed)) is acceptable *here* because it mirrors a fixed wire format — but reading multi-byte fields from a packed struct can fault on strict-alignment architectures. The byte-by-byte accessors from section 3 remain the portable answer.

The Internet checksum

One algorithm, used by IP, ICMP, TCP, and UDP: **one's complement of the one's complement sum of 16-bit words.**

```
uint16_t in_cksum(const void *data, size_t len) {
    const uint8_t *p = data;
    uint32_t sum = 0;
    while (len > 1) { sum += ((uint32_t)p[0] << 8) | p[1]; p += 2; len -= 2; }
    if (len) sum += (uint32_t)p[0] << 8; // odd byte, high half
    while (sum >> 16) sum = (sum & 0xFFFF) + (sum >> 16); // fold carries
    return (uint16_t)~sum;
}
```

Rules:

1. **Zero the checksum field before computing.** Forgetting this is the #1 raw-socket bug.
2. Verification works by computing over the packet *including* the checksum — a valid packet sums to zero.
3. **TCP and UDP checksums cover a pseudo-header** (src IP, dst IP, zero, protocol, transport length) that isn't transmitted. Build a temporary buffer of pseudo-header + segment, checksum that, then discard the pseudo-header. Omitting it produces packets every stack silently drops.

```
struct pseudo_hdr {
    uint32_t saddr, daddr;
    uint8_t zero, proto;
    uint16_t len; // TCP/UDP header + data length
} __attribute__((packed));
```

Byte order: `saddr/daddr` stay in network order (they're already `in_addr_t`); `tot_len` and `id` need `htons`. On old Linux the kernel expected `tot_len` and `frag_off` in *host* order with `IP_HDRINCL` — modern kernels want network order. If your packets vanish, that's a candidate.

Legitimate uses

`ping/traceroute`, `nmap`-style scanners, custom transport protocols above IP, network monitoring, VPN/tunnel software, packet injection for testing.

Be careful. Spoofing source addresses is how amplification attacks work (section 5), scanning networks you don't own is often illegal regardless of intent, and injecting into a network you share affects other people. Test on your own hosts or in a namespace:

```
sudo ip netns add test && sudo ip netns exec test bash
```

6. libpcap

Portable capture that abstracts `AF_PACKET/BPF/PACKET_MMAP` and gives you kernel-level filtering for free.

```

#include <pcap.h>

char errbuf[PCAP_ERRBUF_SIZE];
pcap_t *h = pcap_open_live("eth0", 65535, /*promisc=*/1, /*to_ms=*/1000, errbuf);
if (!h) { fprintf(stderr, "%s\n", errbuf); return 1; }

// Compile and install a BPF filter – runs IN THE KERNEL
struct bpf_program fp;
bpf_u_int32 net = 0, mask = 0;
pcap_lookupnet("eth0", &net, &mask, errbuf);
if (pcap_compile(h, &fp, "tcp port 80", /*optimize=*/1, mask) < 0) {
    fprintf(stderr, "%s\n", pcap_geterr(h)); return 1;
}
pcap_setfilter(h, &fp);

pcap_loop(h, /*count=*/-1, packet_handler, NULL);
pcap_freecode(&fp);
pcap_close(h);

```

```

void packet_handler(u_char *user, const struct pcap_pkthdr *hdr,
                   const u_char *pkt) {
    // hdr->caplen – bytes actually captured (may be < len if snaplen is small)
    // hdr->len    – original wire length
    // hdr->ts     – timestamp

    if (hdr->caplen < 14) return; // ALWAYS bounds-check
    uint16_t ethertype = (pkt[12] << 8) | pkt[13];
    if (ethertype != 0x0800) return;

    if (hdr->caplen < 14 + 20) return;
    const u_char *ip = pkt + 14;
    unsigned ihl = (ip[0] & 0x0F) * 4;
    if (ihl < 20 || hdr->caplen < 14 + ihl) return; // check the CLAIMED length
    too
    // ... parse transport
}

```

Never trust length fields in captured packets. They come from the network, possibly from an attacker, and a parser that indexes past `caplen` is a remote code execution bug in a program running as root. This is the single most important thing about writing capture code. Historically, several pcap-based tools have had exactly this vulnerability.

Key points:

- `pcap_compile` filters run **in the kernel** via BPF — vastly cheaper than filtering in your callback. Always filter as narrowly as you can.
- **Check the datalink type** with `pcap_datalink()`. `DLT_EN10MB` = 14-byte Ethernet header, `DLT_LINUX_SLL` (the `any` pseudo-interface) = 16 bytes, `DLT_RAW` = 0. Hardcoding 14 breaks the moment someone captures on `any` or a tunnel.

- `pcap_open_offline()` reads `.pcap` files — same parsing code, reproducible tests.
- `pcap_dump_open()` / `pcap_dump()` writes them, viewable in Wireshark.
- `pcap_next_ex()` gives you a non-callback loop; combine with `pcap_get_selectable_fd()` to fold capture into the epoll loop from section 7.
- Drop privileges immediately after `pcap_open_live()` succeeds — you don't need root to parse.

Link with `-lpcap`.

Pitfall Checklist

1. `writerv_all` mutating the caller's iovec array without documenting it
 2. Ignoring short `writerv` returns
 3. Exceeding `IOV_MAX`
 4. Confusing `CMSG_LEN` and `CMSG_SPACE`
 5. Manual pointer math instead of `CMSG_*` macros
 6. Zero-length payload with `SCM_RIGHTS` → ancillary data may not arrive
 7. Not checking `MSG_CTRUNC` → silently leaked, unclosable fds
 8. Casting `CMSG_DATA` to `int*` instead of `memcpy` → alignment fault
 9. Forgetting `MSG_CMSG_CLOEXEC` on received fds
 10. Not `unlinking` a stale Unix socket path before `bind`
 11. Wrong `addrlen` for abstract sockets (`sizeof(addr)` pads with NULs)
 12. Abstract sockets without `SO_PEERCRED` → no access control at all
 13. `mmap` + file truncation → `SIGBUS`, process death
 14. `mmap` page faults blocking an event loop invisibly
 15. Not zeroing the checksum field before computing it
 16. Omitting the TCP/UDP pseudo-header → packets silently dropped everywhere
 17. Forgetting `htons(ETH_P_ALL)` on `AF_PACKET`
 18. Assuming raw IPv6 sockets include the IP header (they don't)
 19. Hardcoding a 14-byte link header instead of checking `pcap_datalink()`
 20. Parsing past `caplen` → remote crash or RCE in a root process
 21. Trusting length fields inside captured packets
 22. Reaching for zero-copy on small transfers where it's a net loss
-

Exercises

1. **writerv vs two sends.** Send a 12-byte header + 200-byte body, 10,000 times, with `TCP_NODELAY` **off**. Time both versions. The `writerv` one should be dramatically faster — this is section 6's Nagle lesson from the other direction. Confirm with `strace -c` that syscall count halved.
2. **sendfile benchmark.** Serve a 1 GB file three ways: `read+write` loop, `mmap+write`, and `sendfile`. Measure wall time and CPU time with `/usr/bin/time -v`. The CPU-time difference is the point — throughput may be network-bound, but `sendfile` uses a fraction of the CPU.
3. **FD passing.** Parent binds port 8080, forks a child over a `socketpair()`, sends the listening fd via `SCM_RIGHTS`, then closes its own copy and drops privileges. Child accepts connections on the inherited fd. Verify with `ls -l /proc/<child>/fd`. This is the core of privilege separation.

4. **Break it deliberately.** In exercise 3, size the control buffer as `MSG_LEN(sizeof(int))` instead of `MSG_SPACE(...)`. Observe `MSG_CTRUNC` and confirm via `/proc/<pid>/fd` that the fd count grows with each leaked descriptor.
5. **Unix vs TCP throughput.** Same echo benchmark over `AF_UNIX` and TCP loopback. Measure requests/sec. Then add `SO_PEERCREC` authentication to the Unix version and reject any UID but your own.
6. **ping from scratch.** Raw ICMP socket, hand-built echo request with a correct checksum, parse the reply, compute RTT. Then deliberately corrupt one byte of the checksum and watch the target silently ignore your packet — that's the checksum earning its keep.
7. **Sniffer.** libpcap, filter `"tcp port 80"`, print src/dst IP:port and payload length for each packet. Then feed it a malformed pcap file (truncate one with `dd`) and confirm your bounds checks hold instead of segfaulting.
8. **TCP proxy with splice.** Relay between two sockets through a pipe. Compare CPU usage against a `read/write` version at 1 Gbps.

Exercise 3 is the most valuable — fd passing is the trick that makes privilege separation, socket activation, and zero-downtime restarts possible, and almost nobody knows it works.

Ready for **Concurrency & Architecture** when you are.

8. Concurrency & Architecture

- Thread pools, work queues, lock-free queues
- Per-core accept with `SO_REUSEPORT`
- Multi-process (pre-fork) vs multi-threaded servers
- Connection state machines, buffer management, ring buffers
- Backpressure, slow-client handling, connection limits
- Graceful shutdown and connection draining

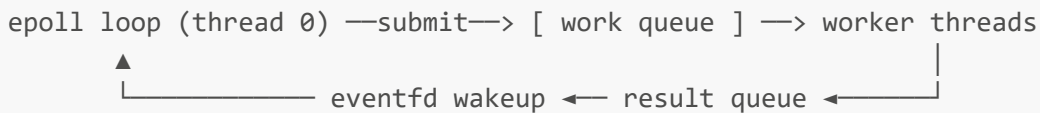
Concurrency & Architecture

Section 7 gave you one event loop that handles many connections. This section is about the shape of the whole server: how you use multiple cores, where memory goes, and what happens when things go wrong.

The theme: **an event loop scales connections; it doesn't scale CPU**. Everything here is about the second problem, plus the resource discipline that keeps a server alive under stress.

1. Thread Pools and Work Queues

Section 4 gave you a thread pool where each worker owned a connection for its lifetime. That's the wrong shape once you have an event loop. The right shape: **the loop owns the connection; the pool executes discrete blocking tasks**.



What belongs in the pool: `getaddrinfo` (blocks for seconds — section 3), disk reads, database calls, compression, TLS handshake crypto, any computation over $\sim 100\ \mu\text{s}$.

Returning results to the loop

This is the part people get wrong. A worker cannot touch connection state — the loop thread owns it. The worker puts a completed task on a result queue and pokes the loop:

```

int notify_fd = eventfd(0, EFD_NONBLOCK | EFD_CLOEXEC);
epoll_ctl(ep, EPOLL_CTL_ADD, notify_fd, &(struct epoll_event){
    .events = EPOLLIN, .data.ptr = &notify_tag });

// Worker, on completion:
result_queue_push(&rq, task);
uint64_t one = 1;
write(notify_fd, &one, sizeof one);           // wakes epoll_wait

// Loop, when notify_fd is readable:
uint64_t drain;
read(notify_fd, &drain, sizeof drain);        // MUST read or it stays readable
(LT)
task *t;
while ((t = result_queue_pop(&rq)))
    t->conn->state = DONE, try_flush(ep, t->conn);
  
```

`eventfd` counts, so `N` writes coalesce into one wakeup with a counter value of `N` — exactly the behavior you want. Forgetting the `read()` under level-triggered `epoll` gives you the 100% CPU spin from section 7.

Queue design

The bounded blocking queue from section 4 is correct and sufficient for most servers. Two upgrades worth knowing:

Per-worker queues with work stealing. A single shared queue is a contention point at high task rates. Give each worker its own deque; idle workers steal from the tail of a busy worker's deque. This is what Go's scheduler, Rust's Tokio, and Intel TBB do.

Sharding. Route tasks to a specific worker by hashing a key (connection ID, user ID). Removes contention *and* gives you single-threaded access to per-key state — no locks on the state itself. Often better than making a queue faster.

Lock-free queues

The honest framing: **a mutex + condvar queue is fast**. An uncontended `pthread_mutex_lock` is ~20 ns. Reach for lock-free only when profiling shows contention, and know what you're signing up for.

A single-producer/single-consumer ring buffer is the one case that's genuinely simple and worth writing:

```
typedef struct {
    void    *slots[CAP];           // CAP must be a power of 2
    _Atomic size_t head;           // consumer writes
    _Atomic size_t tail;           // producer writes
} spsc_queue;

bool spsc_push(spsc_queue *q, void *item) {
    size_t t = atomic_load_explicit(&q->tail, memory_order_relaxed);
    size_t h = atomic_load_explicit(&q->head, memory_order_acquire);
    if (((t + 1) & (CAP - 1)) == (h & (CAP - 1))) return false; // full
    q->slots[t & (CAP - 1)] = item;
    atomic_store_explicit(&q->tail, t + 1, memory_order_release); // publishes
the slot
    return true;
}

bool spsc_pop(spsc_queue *q, void **out) {
    size_t h = atomic_load_explicit(&q->head, memory_order_relaxed);
    size_t t = atomic_load_explicit(&q->tail, memory_order_acquire);
    if (h == t) return false; // empty
    *out = q->slots[h & (CAP - 1)];
    atomic_store_explicit(&q->head, h + 1, memory_order_release);
    return true;
}
```

The acquire/release pairing is the whole correctness argument: the release store on `tail` guarantees the slot write is visible to any thread that acquire-loads that `tail`. Use `memory_order_seq_cst` everywhere if you're unsure — it's slower but you won't be subtly wrong.

Pad head and tail to separate cache lines. Without it, the producer's store to `tail` invalidates the consumer's cache line holding `head` on every push — **false sharing**, and it can be slower than a mutex:

```
_Atomic size_t head;
char _pad[64 - sizeof(size_t)];
_Atomic size_t tail;
```

MPMC lock-free queues (Michael-Scott, Vyukov's bounded MPMC) are a different tier of difficulty — ABA problems, memory reclamation requiring hazard pointers or epoch-based schemes. **Use a library** (`liblfs`, `moodycamel::ConcurrentQueue`, `boost::lockfree`) rather than writing one.

Also note "lock-free" is not "wait-free" and not "fast." It guarantees system-wide progress, not per-thread progress, and under contention it can be slower than a mutex because of retry loops.

2. Per-Core Accept with `SO_REUSEPORT`

The default architecture for a modern Linux server. From section 6: N sockets bind the same port, kernel load-balances by 4-tuple hash.

```
typedef struct { int lfd, ep; pthread_t tid; int id; } worker;

static int make_listener(const char *port) {
    int fd = socket(AF_INET6, SOCK_STREAM | SOCK_NONBLOCK | SOCK_CLOEXEC, 0);
    int yes = 1, no = 0;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
    setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes); // every socket
    setsockopt(fd, IPPROTO_IPV6, IPV6_V6ONLY, &no, sizeof no);
    // ... bind + listen
    return fd;
}

static void *worker_main(void *arg) {
    worker *w = arg;

    cpu_set_t set; // pin to a core
    CPU_ZERO(&set); CPU_SET(w->id, &set);
    pthread_setaffinity_np(pthread_self(), sizeof set, &set);

    w->lfd = make_listener("8080"); // OWN listener, own accept
    queue
    w->ep = epoll_create1(EPOOL_CLOEXEC);
    // ... register lfd, run the event loop from section 7
    return NULL;
}
```

Why this beats one-acceptor-N-loops:

- **Shared-nothing.** No queue, no lock, no cross-thread fd handoff. Each connection lives entirely on one core.
- **No thundering herd.** Each worker's `epoll_wait` only sees its own listener.
- **Cache locality.** Connection state stays in one core's L1/L2. With CPU pinning and NIC RSS aligned, the packet, the interrupt, and the handler can all land on the same core.

Where it hurts:

- **Load imbalance.** The hash is on the 4-tuple, not on load. One worker can get all the long-lived expensive connections while another idles. Usually fine at scale; visible with few clients.
- **Lost queued connections on worker exit.** In-flight SYNs hashed to a dying worker are dropped. `SO_ATTACH_REUSEPORT_EBPF` lets you steer connections away from a draining worker; without it, drain carefully.
- **Not portable.** Linux 3.9+ and recent BSDs only.

Process vs thread for the workers

	Multi-process (pre-fork)	Multi-threaded
Isolation	Crash kills one worker	Crash kills everything
Memory	Separate; no data races possible	Shared; races possible
Shared state	Needs shm/pipes	Trivial
Restart on crash	Supervisor respawns	Whole process dies
Memory use	Higher (though COW helps)	Lower
Debugging	Harder (attach per-PID)	Race conditions
Used by	nginx, Apache, PostgreSQL	Envoy, HAProxy (threads mode), most Rust/Go servers

nginx's model is worth internalizing: a master process (root, binds ports, reads config, supervises) forks N unprivileged workers, each running its own event loop. Master never handles requests. Workers are respawned on crash. Config reload spawns new workers with the new config and drains the old ones — zero downtime, and it works because the master holds the listening fds and can pass them (section 8's [SCM_RIGHTS](#)).

Recommendation: pre-fork if you're serving untrusted input in C/C++ and a memory-safety bug shouldn't take down the whole server. Threads if you need shared state (a cache, connection pool, shared counters) and can accept the blast radius. Many production servers do both — processes for isolation, threads within each for I/O.

Worker count: start at `sysconf(_SC_NPROCESSORS_ONLN)`. More than core count only helps if workers block, and if they block you should have fixed that instead.

3. Connection State Machines

In an event loop you cannot write `read_request(); process(); write_response();` — every step must be resumable. The connection is the stack.

```
typedef enum {
    ST_READ_HEADER,    // accumulating the 4-byte length prefix
    ST_READ_BODY,      // accumulating `need` payload bytes
    ST_PROCESSING,      // handed to the thread pool
    ST_WRITE,           // flushing a response
    ST_DRAINING,        // sent FIN, reading remaining input
    ST_CLOSING
} conn_state;

typedef struct conn {
    int      fd;
    conn_state state;
    uint32_t need;      // bytes still required for the current phase
}
```

```

int64_t    last_active_ms;    // for idle timeout
struct conn *lru_prev, *lru_next;
ringbuf    rx, tx;
bool       want_write, closing;
} conn;

```

The handler advances the machine as far as available data allows, then returns:

```

static int on_readable(int ep, conn *c) {
    if (rx_fill(c) < 0) return -1;           // one recv() into the ring
    buffer
    c->last_active_ms = now_ms();

    for (;;) {                               // may complete several messages
        switch (c->state) {
            case ST_READ_HEADER:
                if (rb_used(&c->rx) < 4) return 0;    // need more – yield to the loop
                c->need = rb_peek_u32(&c->rx);
                if (c->need > MAX_MSG) return -1;    // section 4's bound. Still
mandatory.
                rb_consume(&c->rx, 4);
                c->state = ST_READ_BODY;
                continue;

            case ST_READ_BODY:
                if (rb_used(&c->rx) < c->need) return 0;
                submit_to_pool(c);                // or handle inline if cheap
                c->state = ST_PROCESSING;
                return 0;

            default:
                return 0;                          // not expecting input right now
        }
    }
}

```

Two things this shape gets right: **partial data yields instead of blocking**, and the `for(;;)` loop drains multiple pipelined messages from one `recv()` — without it, a client that pipelines three requests gets one response and hangs (under edge-triggered epoll, permanently).

Buffer management

Per-connection buffers dominate memory at scale. 10k connections × 8 KB rx + 8 KB tx = 160 MB, mostly idle.

Strategies, roughly in order of preference:

1. **Allocate on demand, release when idle.** Most connections are idle most of the time.
2. **Buffer pool.** Free-list of fixed-size blocks; `pop` on demand, `push` on release. Avoids malloc contention and fragmentation.
3. **Start small, grow.** 4 KB initial, double up to a cap. Most requests are small.

4. Cap the tx buffer and treat overflow as backpressure (section 5 below).

Ring buffers

The right structure for a stream: no memmove on consume, natural producer/consumer split.

```
typedef struct {
    uint8_t *buf;
    size_t   cap;           // power of two
    size_t   r, w;          // monotonically increasing; mask on access
} ringbuf;

static inline size_t rb_used(const ringbuf *rb) { return rb->w - rb->r; }
static inline size_t rb_free(const ringbuf *rb) { return rb->cap - rb_used(rb); }
```

The trick that makes them pleasant: **let `r` and `w` grow without bound and mask only when indexing.** `w - r` is then always the used count with no ambiguity between full and empty, and unsigned wraparound is well-defined and self-correcting.

Reading directly into the ring with `readv` avoids the wrap-splitting problem entirely:

```
static int rx_fill(conn *c) {
    ringbuf *rb = &c->rx;
    size_t space = rb_free(rb);
    if (space == 0) return 0;           // buffer full: apply
    backpressure

    size_t wi = rb->w & (rb->cap - 1);
    size_t first = rb->cap - wi;        // to end of buffer
    if (first > space) first = space;

    struct iovec iov[2] = {
        { rb->buf + wi, first },
        { rb->buf,         space - first } // wrapped portion
    };
    ssize_t n = readv(c->fd, iov, (space > first) ? 2 : 1);
    if (n > 0) { rb->w += (size_t)n; return 1; }
    if (n == 0) return -1;              // EOF
    if (errno == EAGAIN || errno == EWOULDBLOCK) return 0;
    if (errno == EINTR) return 0;
    return -1;
}
```

This is exactly what section 8's scatter/gather is for. One syscall fills both halves of a wrapped buffer.

The mirrored/"magic" ring buffer — `mmap` the same physical pages twice consecutively so the buffer appears contiguous even across the wrap — eliminates the two-iovec case entirely. Elegant, but it costs a page-aligned minimum size and setup complexity. Worth it only for very high-rate paths.

4. Backpressure, Slow Clients, Connection Limits

In the **blocking model**, **TCP flow control gave you backpressure for free** (section 1). Your thread blocked in `send()`, the receive window closed, the peer slowed down. In an event loop you never block — so unbounded queuing is now *your* problem, and by default your server will happily allocate memory until it dies.

This is the single most common way event-driven servers fail in production.

Write-side backpressure

```
#define TX_HIGH_WATER (4u << 20)    // 4 MB
#define TX_LOW_WATER  (1u << 20)

int queue_response(int ep, conn *c, const void *data, size_t len) {
    if (rb_used(&c->tx) + len > TX_HIGH_WATER) {
        // Client isn't reading. Do NOT keep buffering.
        log_warn("slow client, dropping");
        return -1;                                // disconnect
    }
    rb_write(&c->tx, data, len);
    return try_flush(ep, c);                      // section 7
}
```

For a proxy or any pipeline, disconnecting is too blunt — instead **stop reading from the source** until the destination drains:

```
void on_tx_progress(int ep, conn *c) {
    if (rb_used(&c->tx) > TX_HIGH_WATER && c->reading) {
        c->reading = false;                        // stop consuming input
        epoll_mod(ep, c, EPOLLOUT);               // drop EPOLLIN
    } else if (rb_used(&c->tx) < TX_LOW_WATER && !c->reading) {
        c->reading = true;
        epoll_mod(ep, c, EPOLLIN | EPOLLOUT);
    }
}
```

Removing `EPOLLIN` stops you draining the kernel receive buffer, which closes the TCP window, which throttles the upstream sender. **You've reconnected application backpressure to TCP flow control** — that's the whole trick. The hysteresis (high/low water marks) prevents flapping.

Read-side and slowloris

The mirror-image attack: open thousands of connections and send one byte per minute. Each costs you a connection slot and a buffer.

```
if (rb_free(&c->rx) == 0 && !message_complete(c))
    return -1; // header/message exceeds buffer → reject
```

Plus timeouts:

Timer	Typical	Defends against
Handshake/first-byte	5–10 s	Idle connection floods
Header complete	10–20 s	Slowloris
Idle (keep-alive)	60–300 s	Resource leak
Total request	30–60 s	Slow POST
Minimum throughput	e.g. 100 B/s	Slow drip that resets per-op timers

Recall from section 6 that `SO_RCVTIMEO` **does not** defend against slowloris — it resets on every partial read. You need an absolute deadline tracked in your own timer structure.

The LRU-list idle timeout from section 7 handles most of this in $O(1)$:

```
void touch(conn *c) { lru_remove(c); c->last_active_ms = now_ms();
lru_push_tail(c); }

void expire_idle(int64_t now) {
    conn *c;
    while ((c = lru_head()) && now - c->last_active_ms > IDLE_MS)
        close_conn(c); // list is sorted by activity; stop at first
live one
}
```

Connection limits and `EMFILE`

Running out of file descriptors is the classic outage. Under level-triggered epoll, `accept()` fails with `EMFILE`, the listener **stays readable**, and your loop spins at 100% CPU forever, accepting nothing.

Three defenses:

```
// 1. Hard cap, checked before accept
if (n_conns >= MAX_CONNS) {
    int cfd = accept4(lfd, NULL, NULL, SOCK_CLOEXEC);
    if (cfd >= 0) close(cfd); // accept and immediately close: fast
reject
    return;
}

// 2. Reserve an fd so you can always accept-and-close under EMFILE
static int spare_fd; // open("/dev/null") at startup
```

```

if (errno == EMFILE) {
    close(spare_fd);                // free one slot
    int cfd = accept(lfd, NULL, NULL);
    if (cfd >= 0) close(cfd);        // drain the queue entry
    spare_fd = open("/dev/null", O_RDONLY | O_CLOEXEC);
    return;
}

// 3. Failing that, disarm the listener briefly
epoll_ctl(ep, EPOLL_CTL_DEL, lfd, NULL);
arm_timer(100 /*ms*/, rearm_listener);

```

Trick 2 is nginx's. It's ugly and it's correct — without something like it, `EMFILE` turns a capacity problem into a total outage.

Raise the limit too, of course:

```

struct rlimit rl;
getrlimit(RLIMIT_NOFILE, &rl);
rl.rlim_cur = rl.rlim_max;           // raise soft limit to hard limit
setrlimit(RLIMIT_NOFILE, &rl);

```

Load shedding beats queuing. A fast rejection (RST, HTTP 503, "server busy") preserves capacity for the requests you *can* serve. Accepting everything and timing out serves nobody and wastes all your resources doing it.

5. Graceful Shutdown and Draining

Shutdown has to satisfy three things at once: stop accepting new work, let in-flight work finish, and bound how long you'll wait.

The sequence

1. Receive SIGTERM
2. Stop accepting → close/deregister listeners
3. (Optional) fail health checks so the LB stops routing to you
4. Signal "closing" on idle keep-alive connections
5. Let in-flight requests complete, refuse new pipelined ones
6. Wait, with a deadline
7. Force-close whatever remains
8. Exit

Step 3 matters more than it looks: if a load balancer is still sending you traffic when you stop accepting, clients get connection errors. Fail readiness checks first, wait for the LB's detection interval, *then* stop accepting.

Implementation

Signals must go through the loop, not a handler touching loop state (section 7):

```
static void on_signal(int ep, int sfd) {
    struct signalfd_siginfo si;
    read(sfd, &si, sizeof si);
    if (si.ssi_signo != SIGTERM && si.ssi_signo != SIGINT) return;

    draining = true;
    drain_deadline = now_ms() + 30000;

    for (int i = 0; i < n_listeners; i++) {           // stop accepting
        epoll_ctl(ep, EPOLL_CTL_DEL, listeners[i], NULL);
        close(listeners[i]);
    }
    for (conn *c = conns_head; c; c = c->next)
        if (c->state == ST_IDLE)                     // idle keep-alives: close
            begin_close(ep, c);
    now
    else
        c->close_after_response = true;              // busy: finish first
}

// In the loop, each iteration:
if (draining) {
    if (n_conns == 0) break;                          // clean exit
    if (now_ms() > drain_deadline) {                  // deadline: force it
        for (conn *c = conns_head; c; c = c->next) force_close(c);
        break;
    }
}
```

Per-connection graceful close is the section 4 pattern, adapted to be non-blocking:

```
void begin_close(int ep, conn *c) {
    if (rb_used(&c->tx) > 0) { c->close_after_flush = true; return; } // flush
    first
    shutdown(c->fd, SHUT_WR);                                     // FIN
    c->state = ST_DRAINING;                                       // keep reading until peer's
    FIN
    c->drain_deadline = now_ms() + 5000;
}
```

Recall why the drain matters: `close()` with unread data in the receive buffer sends RST, and the peer may lose the response you just sent.

Zero-downtime restart

Two mechanisms, both from earlier sections:

SO_REUSEPORT — start the new binary, it binds the same port, both serve concurrently, then **SIGTERM** the old one. Simplest. Risk: connections hashed to the draining worker's queue are lost.

FD passing (**SCM_RIGHTS**, section 8) — the old process hands its listening fds to the new one over a Unix socket, so there's exactly one accept queue throughout and nothing is dropped. This is what nginx, HAProxy, and systemd socket activation do. It's more work and it's the correct answer when dropped connections are unacceptable.

nginx's reload is the reference implementation: master forks new workers with the new config, sends old workers a shutdown signal, old workers stop accepting but finish in-flight requests and exit when done. The listening sockets never close because the *master* holds them.

Deferred close, again

From section 7, restated because it's the bug that survives into production: **never `free()` a connection inside the event dispatch loop**. A later entry in the same `epoll_wait` batch may reference it. Mark and sweep:

```
c->closing = true;
epoll_ctl(ep, EPOLL_CTL_DEL, c->fd, NULL);
list_add(&pending_close, c);
// ... after the dispatch loop:
process_deferred_closes();
```

And check `if (c->closing) continue;` at the top of each dispatch. Both, not either.

Reference Architecture

Putting sections 1–9 together, the shape most production Linux servers converge on:

```
Master process (root)
├─ reads config, binds nothing itself
├─ forks N workers, one per core, respawns on crash
└─ holds listening fds for zero-downtime reload

Worker i (unprivileged, pinned to core i)
├─ own listener via SO_REUSEPORT (or inherited fd)
├─ own epoll instance, level-triggered
├─ non-blocking sockets, TCP_NODELAY, FD_CLOEXEC
├─ per-connection state machine + ring buffers from a pool
├─ LRU idle-timeout list, deferred-close list
├─ tx high/low water marks → EPOLLIN arm/disarm for backpressure
├─ eventfd for thread-pool completions
└─ signalfd for shutdown
    └─ small thread pool for blocking work (DNS, disk, crypto)
```

Nothing here is exotic. Every piece appeared in an earlier section; this is just the assembly.

Pitfall Checklist

1. Worker threads touching connection state owned by the loop
2. Not `read()`ing the `eventfd` → level-triggered spin
3. False sharing between producer and consumer indices
4. Reaching for lock-free before measuring mutex contention
5. Hand-rolling an MPMC lock-free queue
6. `SO_REUSEPORT` without setting it on the *first* socket
7. Assuming `SO_REUSEPORT` balances by load rather than by hash
8. Killing a `SO_REUSEPORT` worker with connections queued to it
9. State machine that handles one message per `recv()` → pipelining hangs
10. `memmove` on every buffer consume instead of using a ring
11. Masking ring indices before subtracting (breaks full/empty distinction)
12. **No tx buffer cap → one slow client OOMs the server**
13. Backpressure without hysteresis → arm/disarm flapping
14. Relying on `SO_RCVTIMEO` for slowloris defense
15. No first-byte or header-complete timeout
16. Busy-looping on `EMFILE` → total outage from a capacity problem
17. No reserved fd for accept-and-close under `EMFILE`
18. Stopping accept before failing health checks → LB sends to a closed port
19. Shutdown with no deadline → hangs forever on one stuck connection
20. `close()` without draining → RST truncates the response you just sent
21. Freeing a connection inside the dispatch loop
22. Per-connection buffers allocated eagerly for idle connections

Exercises

1. **Pool + eventfd.** Add a 4-thread pool to your epoll server for a deliberately slow operation (`usleep(50000)`). Verify the loop still serves other connections during it. Then remove the `read()` on the `eventfd` and watch `top` hit 100%.
2. **`SO_REUSEPORT` distribution.** 4 workers, each logging its ID on accept. Open 1000 connections; plot the distribution. Then open 4 connections from the same source IP with sequential source ports and observe the hash isn't as uniform as you'd hope at small N.
3. **Pin and measure.** Same server with and without `pthread_setaffinity_np`. Benchmark with `wrk`. Then check `perf stat -e cache-misses` for both.
4. **Ring buffer with `readv`.** Implement `rx_fill` above. Test with a buffer sized to force wraps mid-message (make it 64 bytes and send 100-byte messages). Verify no data is lost or duplicated across the wrap.
5. **Pipelining.** Client sends 5 length-prefixed messages in one `write()`. Server must reply 5 times. Remove the `for(;;)` from the state machine and confirm it replies once and hangs.
6. **OOM a server.** Server sends a 500 MB response. Client connects, reads 1 KB, sleeps forever. Watch RSS climb in `top` until the OOM killer fires. Add the high-water mark and confirm it disconnects instead.

7. **Proxy backpressure.** Proxy a fast source to a slow sink. Without `EPOLLIN` disarming, memory grows without bound. With it, `ss -tmi` should show the receive window closing on the source connection — the moment application backpressure becomes TCP backpressure.
8. **EMFILE survival.** `ulimit -n 64`, then open 200 connections. Observe the spin. Add the reserved-fd trick and confirm the server keeps serving its existing 60 connections while rejecting new ones.
9. **Graceful drain.** Client mid-request when you send `SIGTERM`. The request must complete; new connections must be refused. Then hold one connection open past the deadline and confirm the forced exit.
10. **Zero-downtime restart.** Run a client issuing continuous requests. Restart the server via `SO_REUSEPORT` overlap, then again via `SCM_RIGHTS` fd passing. Count dropped requests in each. The difference is the argument for fd passing.

Exercise 6 first — watching your own server get OOM-killed by a client that does nothing but sleep is the fastest way to internalize why backpressure is non-negotiable.

Ready for **Application Protocols** when you are.

9. Application Protocols (implement at least two)

- HTTP/1.1: request parsing, headers, chunked encoding, keep-alive
- HTTP/2 basics (framing, streams, HPACK), HTTP/3 & QUIC overview
- WebSocket: handshake, frame format, masking, ping/pong
- DNS message format and a resolver client
- SMTP, FTP, Telnet as protocol-design exercises
- MQTT, gRPC/Protobuf, custom binary protocols
- Serialization: Protobuf, FlatBuffers, MessagePack, JSON

Application Protocols

Sections 1–9 built the transport machinery. This section is about what you put on top of it — and, more importantly, about the recurring design decisions every protocol makes. Once you've implemented two, the third is mostly a spec-reading exercise.

Security framing for this whole section: a protocol parser is the part of your program that untrusted bytes reach first. Every length field is attacker-controlled. Every bounds check is load-bearing. The CVE history of HTTP parsers is long and instructive.

1. HTTP/1.1

The best first protocol. Text-based, debuggable with `telnet`, and it exercises delimiter framing, length framing, *and* chunked framing in one spec.

Request format

```
GET /path?query HTTP/1.1\r\n
Host: example.com\r\n
Content-Length: 27\r\n
\r\n
<body>
```

Response:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 1234\r\n
\r\n
<body>
```

Headers end at `\r\n\r\n` — that's the delimiter framing from section 4. The body length then comes from `Content-Length` (length framing) or `Transfer-Encoding: chunked`.

Parsing, incrementally

In an event loop you can't "read until headers end" — you accumulate and re-check (section 9's state machine):

```
typedef enum { P_REQUEST_LINE, P_HEADERS, P_BODY, P_CHUNK_SIZE,
              P_CHUNK_DATA, P_COMPLETE } parse_state;

typedef struct {
    parse_state state;
    char        method[16], path[2048], version[16];
    header      hdrs[MAX_HEADERS];    // bounded! 64 or 100
    size_t      nhdrs;
    int64_t     content_length;        // -1 = unset
    bool        chunked, keep_alive;
    size_t      chunk_remaining;
    size_t      header_bytes;          // for the size cap
} http_req;
```

```
// Returns: 1 = need more data, 0 = complete, -1 = protocol error
int parse_headers(http_req *r, ringbuf *rb) {
    for (;;) {
        char *line; size_t len;
        if (!rb_find_crlf(rb, &line, &len)) {
            if (rb_used(rb) > MAX_HEADER_BYTES) return -1; // slowloris / DoS
            bound
                return 1; // need more
        }
        if (len == 0) { // blank line:
```

```

headers done
    rb_consume(rb, 2);
    return finalize_headers(r);
}
if (r->nhdrs >= MAX_HEADERS) return -1;           // bound the count
too

char *colon = memchr(line, ':', len);
if (!colon) return -1;

size_t nlen = (size_t)(colon - line);
char *val = colon + 1;
size_t vlen = len - nlen - 1;
while (vlen && (*val == ' ' || *val == '\t')) val++, vlen--; // OWS

if (nlen == 0 || memchr(line, ' ', nlen)) return -1; // space before
colon: smuggling

store_header(r, line, nlen, val, vlen);           // name compare is
case-insensitive
rb_consume(rb, len + 2);
}
}

```

Header names are **case-insensitive** — `strncasecmp`, always. Values are not.

Body framing — where the bugs live

Three ways to determine body length, and the precedence between them is security-critical:

1. **Transfer-Encoding: chunked** → chunked framing
2. **Content-Length: N** → exactly N bytes
3. Neither → no body (request), or read-until-close (HTTP/1.0 response)

```

static int finalize_headers(http_req *r) {
    const char *te = find_header(r, "Transfer-Encoding");
    const char *cl = find_header(r, "Content-Length");

    // REQUEST SMUGGLING: if both are present, front-end and back-end may
    // disagree about which wins, letting an attacker inject a second request.
    if (te && cl) return -1;           // reject. Do not "pick one".

    if (te) {
        if (strncasecmp(te, "chunked") != 0) return -1; // only chunked is valid
last
        r->chunked = true;
        r->state = P_CHUNK_SIZE;
    } else if (cl) {
        if (find_header_dup(r, "Content-Length")) return -1; // duplicate CL:
reject
        if (!parse_i64_strict(cl, &r->content_length)) return -1; // no "+5", no

```

```

"5abc"
    if (r->content_length < 0 || r->content_length > MAX_BODY) return -1;
    r->state = P_BODY;
} else {
    r->state = P_COMPLETE;
}
return 0;
}

```

HTTP request smuggling is the class of attack this guards against: a proxy uses **Content-Length** while the origin uses **Transfer-Encoding** (or vice versa), so bytes the proxy thinks are a body get parsed by the origin as a new request — bypassing authentication, poisoning caches. Reject ambiguity rather than resolving it. Same for duplicate **Content-Length** headers, header names with trailing spaces, and **Transfer-Encoding: chunked, chunked**.

Chunked encoding

```

1a\r\n
<26 bytes of data>\r\n
0\r\n
\r\n

```

Size is **hexadecimal**, optionally followed by **extension**. Terminated by a zero-size chunk plus optional trailers.

```

int parse_chunk_size(http_req *r, ringbuf *rb) {
    char *line; size_t len;
    if (!rb_find_crlf(rb, &line, &len)) return 1;

    char *semi = memchr(line, ';', len);          // chunk extensions - ignore,
    don't reject
    size_t hexlen = semi ? (size_t)(semi - line) : len;
    if (hexlen == 0 || hexlen > 16) return -1;    // 16 hex digits = 64 bits, cap
    it

    uint64_t sz = 0;
    for (size_t i = 0; i < hexlen; i++) {
        int d = hexval(line[i]);
        if (d < 0) return -1;                    // strict: no whitespace, no sign
        if (sz > (UINT64_MAX >> 4)) return -1;    // OVERFLOW CHECK
        sz = (sz << 4) | (uint64_t)d;
    }
    if (sz > MAX_BODY) return -1;

    rb_consume(rb, len + 2);
    r->chunk_remaining = (size_t)sz;
    r->state = sz == 0 ? P_COMPLETE : P_CHUNK_DATA;
}

```

```
    return 0;
}
```

The overflow check is the one people skip. A chunk size of `FFFFFFFFFFFFFFFF` that wraps to a small number, followed by a large body, is a heap overflow.

Writing chunked output — this is how you stream a response of unknown length:

```
void write_chunk(conn *c, const void *data, size_t len) {
    char hdr[32];
    int n = snprintf(hdr, sizeof hdr, "%zx\r\n", len);
    struct iovec iov[3] = { {hdr, n}, {(void*)data, len}, {"\r\n", 2} };
    writev_all(c->fd, iov, 3); // section 8: one syscall
}
void end_chunked(conn *c) { send_all(c->fd, "0\r\n\r\n", 5); }
```

Keep-alive

HTTP/1.1 defaults to persistent connections. Recall from section 1 why this matters: every new TCP connection pays a handshake RTT *plus* slow-start ramp.

```
r->keep_alive = true; // 1.1 default
const char *conn_hdr = find_header(r, "Connection");
if (conn_hdr && strcasecmp(conn_hdr, "close") == 0) r->keep_alive = false;
if (strcmp(r->version, "HTTP/1.0") == 0)
    r->keep_alive = conn_hdr && strcasecmp(conn_hdr, "keep-alive") == 0;
```

The rule: you may only keep the connection alive if you consumed the body exactly. Get the length wrong by one byte and the next request parse starts mid-body — desync, and potentially smuggling. When in doubt, send `Connection: close` and close.

Pipelining (multiple requests in flight) requires the `for(;;)` drain loop from section 9 and responses in strict request order. Most servers support it; most clients don't use it. HTTP/2 replaced it.

Head-of-line blocking is HTTP/1.1's core limitation: one slow response stalls everything behind it on that connection. Browsers work around it with 6 parallel connections per host. That's the problem HTTP/2 exists to solve.

2. HTTP/2

Same semantics, completely different wire format: binary, multiplexed, header-compressed.

Framing

```

+-----+
| Length (24 bits) |
+-----+-----+-----+
| Type (8)      | Flags (8) |
+-----+-----+-----+
|R| Stream Identifier (31) |
+=====+
| Frame Payload |
+-----+

```

9-byte header. Types: **DATA**, **HEADERS**, **PRIORITY**, **RST_STREAM**, **SETTINGS**, **PUSH_PROMISE**, **PING**, **GOAWAY**, **WINDOW_UPDATE**, **CONTINUATION**.

Connection opens with the preface **PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n** followed by a **SETTINGS** frame. That odd string is deliberately chosen to make HTTP/1.1 servers fail immediately rather than misinterpret it.

Streams

Independent request/response pairs multiplexed over one connection, each with an ID (odd = client-initiated, even = server). Interleaved **DATA** frames mean a slow response no longer blocks others — **HTTP-level head-of-line blocking is solved**. TCP-level HOL blocking remains, because a lost TCP segment stalls every stream. That's what QUIC fixes.

HPACK

Header compression using a static table (61 predefined common headers), a dynamic table (recently seen headers), and Huffman coding. **:method: GET** becomes a single byte.

Security note: HPACK's dynamic table shares state between requests. **CRIME/BREACH**-style attacks exploit compression side channels — this is why HPACK deliberately restricts what can be compressed together, and why implementing HPACK yourself is a bad idea.

Flow control

Per-stream *and* per-connection **WINDOW_UPDATE** frames — application-level flow control layered on TCP's. If you don't implement it correctly, you get exactly the unbounded-buffering failure from section 9, one level up.

Practical advice

Don't implement HTTP/2 from scratch. Use **nghttp2** (C, the reference implementation) or **libcurl** for clients. HPACK alone is a substantial project, and the failure modes are subtle. The **HTTP/2 Rapid Reset** CVE (2023) — flooding streams and immediately resetting them — hit essentially every implementation simultaneously.

Read the frame layout, implement a frame parser as an exercise, then use a library.

3. HTTP/3 and QUIC

QUIC is a **reliable, multiplexed, encrypted transport over UDP** — everything from section 5 that you were told not to build yourself, built properly by people who spent years on it.

Why UDP: TCP is implemented in kernels and middleboxes, so it can't evolve. Building on UDP puts the transport in userspace where it can ship with the application.

What it provides:

- **Streams without TCP head-of-line blocking** — a lost packet stalls only its stream
- **0-RTT / 1-RTT handshake** — TLS 1.3 integrated into the transport, not layered above it
- **Connection migration** — a connection ID rather than a 4-tuple, so switching WiFi→cellular doesn't break connections
- **Always encrypted**, including most of the transport header
- Congestion control in userspace, upgradeable

Costs: higher CPU than TCP (userspace, per-packet crypto), UDP blocked or deprioritized on some networks, complex.

Note how many section-5 concerns it addresses explicitly: address validation tokens against amplification, a 1200-byte minimum MTU assumption, sequence numbers that never repeat (packet numbers are distinct from stream offsets).

Implementations: [ngtcp2](#), [quiche](#) (Cloudflare), [msquic](#) (Microsoft), [lsquic](#) (LiteSpeed). HTTP/3 = HTTP semantics + QPACK (HPACK adapted for out-of-order delivery) over QUIC.

4. WebSocket

Excellent second protocol: HTTP handshake, then a compact binary frame format. Small enough to implement fully in an afternoon.

Handshake

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Server computes the accept value: SHA-1 of the client key concatenated with a fixed GUID, base64-encoded.

```
#define WS_GUID "258EAF55-E914-47DA-95CA-C5AB0DC85B11"

void ws_accept_key(const char *client_key, char out[32]) {
    char concat[128];
    int n = snprintf(concat, sizeof concat, "%s%s", client_key, WS_GUID);
    unsigned char sha[20];
    SHA1((unsigned char *)concat, (size_t)n, sha);
```



```
base64_encode(sha, 20, out);
}
```

This isn't security — it's proof that the server understands WebSocket, preventing cache-poisoning attacks against intermediaries that might otherwise treat the upgrade as a normal HTTP response.

Response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

Check Origin for browser clients — WebSocket is not subject to the same-origin policy, so any page can connect to your server. This is Cross-Site WebSocket Hijacking, and it's easy to miss.

Frame format

0										1										2										3											
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1										
+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+											
F R R R				opcode M						Payload len										Extended payload length																					
I S S S				(4) A						(7)										(16/64)																					
N V V V				S																																					
1 2 3				K																																					
+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+											
										Masking-key (0 or 4 bytes)																				Payload Data											
+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+										+---+---+---+---+---+---+---+---+---+---+											

Opcodes: **0x0** continuation, **0x1** text, **0x2** binary, **0x8** close, **0x9** ping, **0xA** pong. Control frames (0x8–0xA) must have payload ≤ 125 bytes and must not be fragmented.

Payload length is **variable-width**: 0–125 is the literal length; 126 means the next 2 bytes are the length; 127 means the next 8 bytes are.

```
// Returns 1 = need more, 0 = frame parsed, -1 = protocol error
int ws_parse_frame(ringbuf *rb, ws_frame *f) {
    if (rb_used(rb) < 2) return 1;
    uint8_t h[2]; rb_peek(rb, h, 2);

    f->fin    = h[0] & 0x80;
    f->opcode = h[0] & 0x0F;
    bool mask = h[1] & 0x80;
    uint64_t len = h[1] & 0x7F;
    size_t off = 2;

    if (h[0] & 0x70) return -1; // RSV bits set, no extension
```

```

negotiated

    if (len == 126) {
        if (rb_used(rb) < off + 2) return 1;
        uint8_t b[2]; rb_peek_at(rb, off, b, 2);
        len = ((uint64_t)b[0] << 8) | b[1];
        if (len < 126) return -1; // must use minimal encoding
        off += 2;
    } else if (len == 127) {
        if (rb_used(rb) < off + 8) return 1;
        uint8_t b[8]; rb_peek_at(rb, off, b, 8);
        len = 0;
        for (int i = 0; i < 8; i++) len = (len << 8) | b[i];
        if (len < 65536 || (len >> 63)) return -1; // minimal encoding; MSB must
be 0
        off += 8;
    }

    if (len > MAX_WS_FRAME) return -1; // ALWAYS cap
    if (f->opcode >= 0x8 && (len > 125 || !f->fin)) return -1; // control frame
rules

    uint8_t key[4] = {0};
    if (mask) {
        if (rb_used(rb) < off + 4) return 1;
        rb_peek_at(rb, off, key, 4);
        off += 4;
    } else if (f->is_server_side) {
        return -1; // clients MUST mask; reject
if not
    }

    if (rb_used(rb) < off + len) return 1; // wait for the full payload
    rb_consume(rb, off);
    rb_read(rb, f->payload, (size_t)len);
    if (mask)
        for (uint64_t i = 0; i < len; i++)
            f->payload[i] ^= key[i & 3]; // unmask
    f->len = len;
    return 0;
}

```

Masking

Client→server frames must be masked with a random 4-byte key, XORed over the payload. Server→client frames must not be.

The reason is not confidentiality — the key is sent in the clear. It's to prevent **cache poisoning of intercepting proxies**: without masking, an attacker could craft WebSocket payloads that a naive transparent proxy parses as a separate HTTP request. Masking makes the bytes on the wire unpredictable, so they can't be steered.

A server that accepts unmasked client frames is non-compliant and creates that exposure. Reject them.

Ping/pong and close

Ping/pong is the application-level heartbeat recommended throughout sections 4 and 6. **A pong must echo the ping's payload exactly.** Unsolicited pongs are legal as one-way keepalives.

Close is a handshake: send a close frame with a 2-byte big-endian status code (1000 = normal, 1001 = going away, 1002 = protocol error, 1009 = message too big), wait for the peer's close frame, *then* close the TCP connection. That's section 4's graceful shutdown, at the application layer.

Fragmentation: opcode `0x1/0x2` with `FIN=0`, then `0x0` continuations, final with `FIN=1`. **Control frames may be interleaved between fragments** — your state machine must handle a ping arriving mid-message.

5. DNS

The best UDP protocol to implement (section 5), and small enough to do completely.

Message format

```
+-----+
|      Header      | 12 bytes
+-----+
|      Question    | QNAME, QTYPE, QCLASS
+-----+
|      Answer      | RRs
+-----+
|      Authority   |
+-----+
|      Additional  |
+-----+
```

Header:

```
struct dns_header {
    uint16_t id;
    uint16_t flags;      // QR(1) OPCODE(4) AA TC RD RA Z(3) RCODE(4)
    uint16_t qdcount, ancount, nscount, arcount;
}; // all network byte order
```

Name encoding

`www.example.com` becomes `\3www\7example\3com\0` — each label prefixed by its length, terminated by a zero byte.

```

size_t dns_encode_name(uint8_t *out, size_t cap, const char *name) {
    size_t o = 0;
    const char *p = name;
    while (*p) {
        const char *dot = strchr(p, '.');
        size_t llen = dot ? (size_t)(dot - p) : strlen(p);
        if (llen == 0 || llen > 63) return 0;          // label limit
        if (o + 1 + llen + 1 > cap) return 0;
        out[o++] = (uint8_t)llen;
        memcpy(out + o, p, llen);
        o += llen;
        p = dot ? dot + 1 : p + llen;
    }
    if (o + 1 > cap || o + 1 > 255) return 0;        // total name limit
    out[o++] = 0;
    return o;
}

```

Name compression — the dangerous part

A label whose top two bits are **11** is a **pointer** to an offset earlier in the message. This saves space and creates a classic vulnerability: **a pointer that points to itself, or two pointers that point at each other, makes a naive parser loop forever.**

```

int dns_decode_name(const uint8_t *msg, size_t msglen, size_t off,
                    char *out, size_t cap) {
    size_t o = 0, jumps = 0;
    bool jumped = false;
    size_t consumed_end = 0;

    for (;;) {
        if (off >= msglen) return -1;
        uint8_t len = msg[off];

        if ((len & 0xC0) == 0xC0) {                    // pointer
            if (off + 1 >= msglen) return -1;
            if (++jumps > 20) return -1;              // ← THE CHECK. Bounds the
loop.
            size_t ptr = (((size_t)(len & 0x3F)) << 8) | msg[off + 1];
            if (ptr >= off) return -1;                // must point BACKWARD only
            if (!jumped) { consumed_end = off + 2; jumped = true; }
            off = ptr;
            continue;
        }
        if (len & 0xC0) return -1;                    // reserved bits
        if (len == 0) { off++; break; }
        if (off + 1 + len > msglen) return -1;
        if (o + len + 1 >= cap) return -1;

        memcpy(out + o, msg + off + 1, len);
    }
}

```

```

        o += len; out[o++] = '.';
        off += 1 + len;
    }
    if (o) out[o - 1] = '\\0'; else out[0] = '\\0';
    return (int)(jumped ? consumed_end : off);
}

```

Two defenses: a jump counter, and requiring pointers to go strictly backward. Either alone is sufficient; use both.

Resolver client

```

int dns_query(const char *server, const char *name, uint16_t qtype) {
    uint8_t pkt[512]; // 512 = the classic UDP
    limit
    struct dns_header h = {0};
    h.id = (uint16_t)(random() & 0xFFFF); // MUST be unpredictable
    h.flags = htons(0x0100); // RD = recursion desired
    h.qdcount = htons(1);
    memcpy(pkt, &h, 12);

    size_t n = 12 + dns_encode_name(pkt + 12, sizeof pkt - 12, name);
    put_u16(pkt + n, qtype); n += 2; // A=1, AAAA=28, MX=15,
    TXT=16
    put_u16(pkt + n, 1); n += 2; // IN

    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    connect(fd, server_addr, addrlen); // section 5: enables
    ECONNREFUSED
    send(fd, pkt, n, 0);

    // Retransmit with backoff, section 5's stop-and-wait
    uint8_t rsp[4096];
    ssize_t rn = recv_with_timeout(fd, rsp, sizeof rsp, 2000);
    if (rn < 12) return -1;
    if (memcmp(rsp, &h.id, 2) != 0) return -1; // ID must match – anti-
    spoofing
    // Also verify the question section echoes what you asked.
    if (ntohs(*(uint16_t*)(rsp+2)) & 0x0200) { /* TC set → retry over TCP */ }
    return parse_response(rsp, (size_t)rn);
}

```

Security essentials: a random query ID *and* a random source port (section 3: don't **bind**, let the kernel pick) are what make off-path spoofing hard. Kaminsky's 2008 attack worked because resolvers used predictable IDs and fixed ports. Always verify the ID and the echoed question before trusting a response.

Truncation: if the **TC** flag is set, the response didn't fit — retry over TCP (same message format, prefixed with a 2-byte big-endian length; length framing from section 4). EDNS0 negotiates larger UDP responses, but recall from section 5 that large UDP invites fragmentation.

6. SMTP, FTP, Telnet as Design Exercises

Worth reading not to implement, but for what they teach about protocol design.

SMTP — line-based commands, 3-digit status codes with a class in the first digit (2xx ok, 4xx transient, 5xx permanent). The **DATA** command ends with a lone **.** on a line, requiring **dot-stuffing**: any line starting with **.** gets an extra one prepended. That's what in-band delimiters cost you, and it's the argument for length prefixing.

FTP — separate control and data connections. Active mode has the *server* connect back to the client, which NAT (section 1) broke completely, forcing passive mode. The lesson: protocols that embed IP addresses in their payload don't survive address translation. SIP has the same problem.

Telnet — in-band signaling via **IAC** (0xFF) escape sequences, so any literal 0xFF in the data must be doubled. Same lesson as dot-stuffing.

The common thread: **in-band signaling always costs you escaping**. Length-prefixed binary framing avoids the entire category.

7. MQTT, gRPC, and Custom Binary Protocols

MQTT

A clean, small binary protocol — an excellent implementation target after WebSocket.

Fixed header: 1 control byte (type + flags) plus a **variable-length integer** for remaining length — 7 bits of payload per byte, high bit as continuation, 1–4 bytes covering up to 256 MB.

```
int mqtt_decode_varint(const uint8_t *p, size_t avail, uint32_t *out) {
    uint32_t v = 0, mult = 1;
    for (size_t i = 0; i < 4; i++) {                // MAX 4 BYTES
        if (i >= avail) return 0;                    // need more
        v += (uint32_t)(p[i] & 0x7F) * mult;
        if (!(p[i] & 0x80)) { *out = v; return (int)i + 1; }
        mult *= 128;
    }
    return -1;                                       // malformed: 5th
    continuation byte
}
```

That 4-byte cap is the bounds check — without it, a stream of 0xFF bytes loops indefinitely.

QoS levels are a clean case study in the reliability spectrum from section 5: QoS 0 = fire and forget (raw UDP semantics), QoS 1 = at least once (ACK + retransmit, duplicates possible), QoS 2 = exactly once (four-way handshake). Each level costs a round trip.

gRPC / Protobuf

HTTP/2 + Protobuf. Message framing on the wire: 1 compression byte + 4-byte big-endian length + payload — length prefixing again, inside HTTP/2 **DATA** frames.

Protobuf itself: tag-length-value with varint encoding, field numbers rather than names, unknown fields skipped. That last property is what gives it forward and backward compatibility — the rule being **never reuse a field number, never change a field's type**.

Designing your own binary protocol

If you build one, include these:

```
// Header – 16 bytes, fixed, explicitly serialized
// magic(4) | version(1) | type(1) | flags(2) | request_id(4) | length(4)
```

Element	Why
Magic number	Detect desync and wrong-protocol connections immediately
Version	You will need to change the format
Fixed-size header	Read a known amount, then the body
Length field	Framing (section 4)
Request ID	Lets you multiplex and pipeline; without it you're serialized
Explicit byte order	Section 3. Never send a struct.
Reserved/flag bits	Extension without a version bump
A hard max length	Section 4. Non-negotiable.

Extensibility rules: length-prefix everything so old readers can skip unknown pieces; add fields, never repurpose them; make new fields optional with sane defaults.

8. Serialization

Format	Size	Speed	Schema	Human-readable	Zero-copy
JSON	Large	Slow	No	Yes	No
MessagePack	Small	Fast	No	No	No
Protobuf	Small	Fast	Yes	No	No
FlatBuffers	Medium	Fastest read	Yes	No	Yes
Cap'n Proto	Medium	Fastest read	Yes	No	Yes

JSON — the default for public APIs and debuggability. In C, use **jansson**, **cJSON**, or **yyjson** (fastest). Never hand-roll a JSON parser for untrusted input; deeply nested arrays are a stack-exhaustion DoS, and every hand-rolled parser has gotten string escapes wrong.

MessagePack — JSON's data model in a compact binary encoding. Drop-in replacement when you control both ends and want the bytes smaller. `msgpack-c`.

Protobuf — schema-driven, generated code, excellent evolution story. `protobuf-c` for plain C; the C++ library for C++. Costs a build step and a `.proto` file.

FlatBuffers / Cap'n Proto — the buffer *is* the object graph. No parsing step: you access fields directly in the received bytes via offsets. Ideal for large messages where you read a few fields, and for mmap'd data. Costs: larger encoding, more awkward construction API.

Choosing: public API → JSON. Internal RPC with schema evolution → Protobuf. Latency-critical with large messages → FlatBuffers. Bandwidth-constrained, no schema → MessagePack. Fully in-house, fixed format → hand-rolled binary with the header above.

The rule for all of them: cap the maximum decoded size, cap nesting depth, and never trust a length or count field from the wire. This is the same sentence as section 4's `len > MAX_MSG` check, and it's the last time it needs saying.

Pitfall Checklist

1. Accepting both `Content-Length` and `Transfer-Encoding` → request smuggling
2. Accepting duplicate `Content-Length` headers
3. Case-sensitive header name comparison
4. No cap on header count, header size, or body size
5. Chunk-size parsing without an overflow check
6. Keeping a connection alive after an inexact body read → desync
7. Implementing HPACK or QUIC yourself
8. WebSocket server accepting unmasked client frames
9. Not checking `Origin` on WebSocket upgrade
10. Accepting non-minimal WebSocket length encodings
11. Control frames larger than 125 bytes, or fragmented
12. Not handling control frames interleaved between fragments
13. DNS name decompression without a jump limit → infinite loop
14. DNS pointers allowed to point forward
15. Predictable DNS query IDs or fixed source port → spoofing
16. Not verifying the DNS response ID and question section
17. Ignoring the DNS `TC` flag instead of retrying over TCP
18. MQTT varint without a 4-byte cap
19. Custom protocol with no version field
20. Custom protocol with no request ID → no multiplexing possible
21. Sending raw structs (section 3, still true)
22. Unbounded nesting depth in JSON/MessagePack parsers

Exercises

Pick **two**. The recommended pair is HTTP/1.1 and WebSocket, since the second builds on the first's handshake.

1. **HTTP/1.1 static file server.** Incremental parser on the epoll loop from section 7, keep-alive, **Content-Length** and chunked responses, **sendfile** for bodies (section 8), 404/400/413/501 handling, request timeouts. Test with **curl -v**, then **wrk -c 100 -d 30s**. Then feed it deliberately malformed requests: both **Content-Length** and **Transfer-Encoding**, a chunk size of **FFFFFFFFFFFFFFFF**, 10,000 headers, a header line with no newline. It must reject all of them without crashing.
2. **WebSocket echo server.** Upgrade from your HTTP server, full frame parsing including 64-bit lengths, unmasking, fragmentation, ping/pong, close handshake. Test against the **Autobahn Testsuite** — 500+ conformance cases, and it will find bugs you didn't imagine. Passing Autobahn is a real accomplishment.
3. **DNS resolver.** **dig**-like supporting A, AAAA, MX, TXT, CNAME. Random ID and source port, response validation, name decompression with the jump limit, **TC** → TCP fallback, retry with backoff. Then craft a malicious response with a self-referential compression pointer and confirm your parser rejects it rather than hanging.
4. **MQTT client.** CONNECT/CONNACK, SUBSCRIBE, PUBLISH at QoS 0 and 1, PINGREQ keepalive. Test against Mosquitto.
5. **Custom RPC protocol.** Design the 16-byte header above, implement request/response multiplexing over one connection with request IDs, and benchmark against HTTP/1.1 doing the same work. Then bump the version field and prove an old client and new server interoperate.
6. **Fuzz your parser.** Build with **-fsanitize=address,undefined** and run **AFL++** or **libFuzzer** against your HTTP parser for an hour. Assume it will find something; parser bugs are found by fuzzers, not by reading. This is the highest-value hour in the entire exercise list.

Do exercise 6 no matter which protocols you pick.

Ready for **Security** when you are.

10. Security

- TLS/SSL concepts: handshake, certificates, cipher suites, PKI
- OpenSSL API: **SSL_CTX**, **SSL_new**, **SSL_connect**, **SSL_accept**, **SSL_read/write**
- Certificate verification, hostname validation, SNI
- mbedTLS / BoringSSL as alternatives
- Common vulnerabilities: buffer overflows, integer overflow in length fields, format strings, unvalidated input, DoS via slowloris/resource exhaustion
- Secure coding practices for parsers

Security

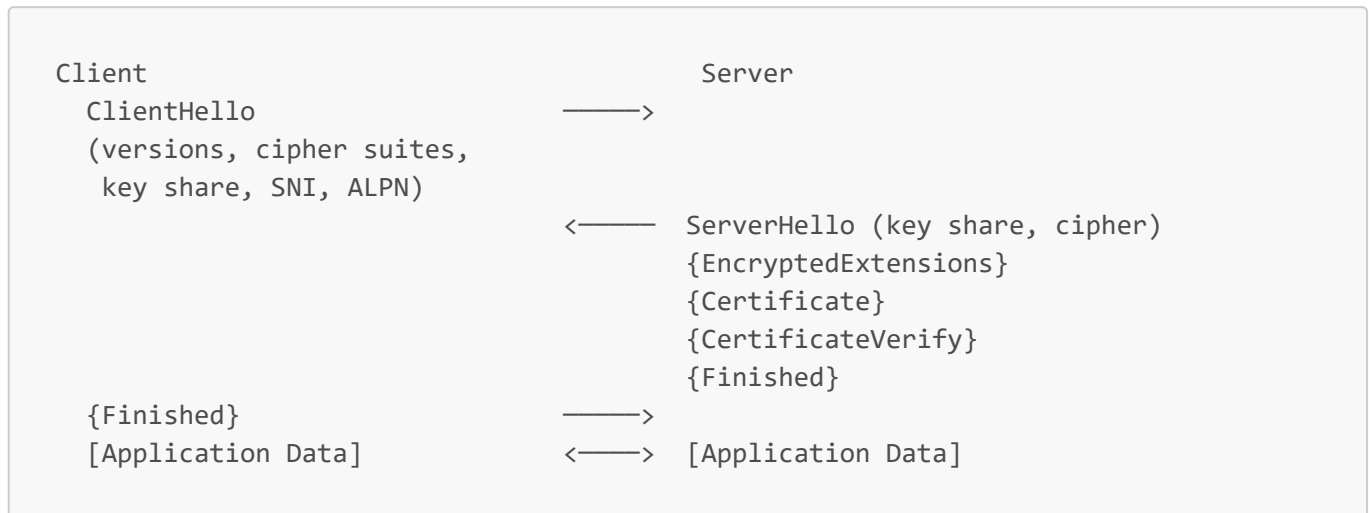
Two halves. The first is **transport security** — using TLS correctly, which mostly means using a library correctly rather than implementing anything. The second is **memory and resource safety** — the vulnerability classes that C and C++ network code produces, and the discipline that prevents them.

The framing that matters: everything you've built in sections 1–10 processes bytes from an untrusted source, often as a long-running privileged process. That's the highest-value attack surface in most systems.

1. TLS Concepts

TLS provides three things: **confidentiality** (encryption), **integrity** (tamper detection), and **authentication** (you're talking to who you think). The third is the one applications get wrong.

Handshake — TLS 1.3



`{ }` = encrypted with handshake keys, `[ ]` = with application keys.

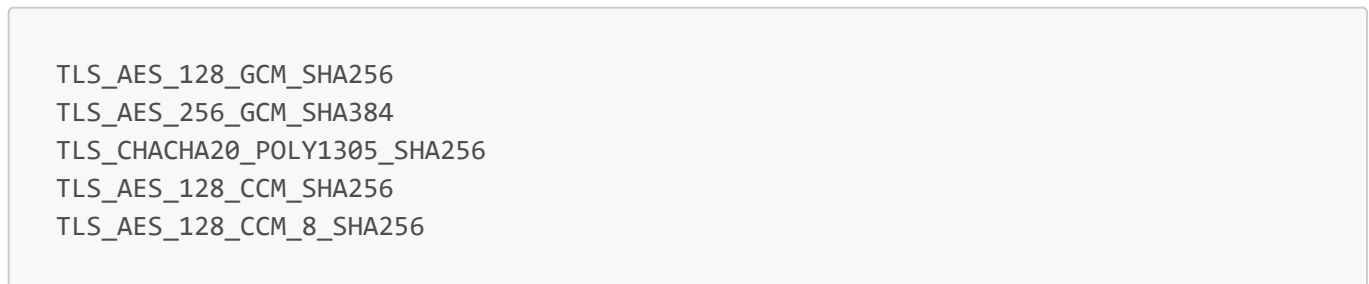
One round trip. TLS 1.2 needed two, so every connection paid an extra RTT on top of the TCP handshake — recall from section 1 why that plus slow start makes connection reuse so valuable.

TLS 1.3 also removed the broken parts: static RSA key exchange (no forward secrecy), CBC mode, RC4, compression (CRIME), renegotiation, and custom DH groups. **It is much harder to configure insecurely**, which is the main practical argument for requiring it.

0-RTT sends application data in the first flight using a pre-shared key from a prior session. It is **replayable by design** — an attacker can capture and resend that data. Only use it for idempotent requests, never for anything that mutates state.

Cipher suites

TLS 1.3 has exactly five, and they're all fine:



Key exchange and signature algorithms are negotiated separately. TLS 1.2's combinatorial suite names (`ECDHE-RSA-AES128-GCM-SHA256`) are the reason its configuration is a minefield.

For TLS 1.2, if you must support it: ECDHE key exchange only (forward secrecy), AEAD ciphers only (GCM or ChaCha20-Poly1305), no CBC, no RC4, no 3DES, no export ciphers, no anonymous suites.

Don't hand-write cipher strings. Use Mozilla's SSL Configuration Generator or OpenSSL's `@SECLEVEL / SSL_CTX_set_security_level`.

PKI and certificates

An X.509 certificate binds a public key to an identity, signed by a CA. Verification walks the chain from the server's leaf certificate up through intermediates to a root in your trust store.

Checks that must all pass:

1. **Signature chain** validates to a trusted root
2. **Not expired** (`notBefore` / `notAfter`)
3. **Not revoked** (CRL/OCSP — see below)
4. **Basic constraints** — intermediates must be marked as CAs, leaves must not be
5. **Key usage / EKU** — appropriate for server authentication
6. **Hostname matches** — the check libraries historically left to you

Certificate #6 is where applications fail. See section 3.

Revocation is genuinely weak. CRLs are large and stale; OCSP leaks browsing behavior to the CA and most clients soft-fail on error (so an attacker who blocks OCSP wins). **OCSP stapling** is better — the server fetches its own signed status and includes it in the handshake:

```
SSL_CTX_set_tlsext_status_type(ctx, TLSEXT_STATUSTYPE_ocsp);
```

The industry's actual answer has been **short-lived certificates** (90 days, Let's Encrypt) — revocation matters less when everything expires quickly.

mTLS

Both sides present certificates. Standard for service-to-service authentication inside a cluster. Server side:

```
SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER | SSL_VERIFY_FAIL_IF_NO_PEER_CERT, NULL);  
SSL_CTX_load_verify_locations(ctx, "/etc/ssl/client-ca.pem", NULL);
```

2. OpenSSL API

Use **OpenSSL 3.x** or **1.1.1+**. The 1.0.x API (`SSL_library_init`, explicit locking callbacks) is obsolete; if a tutorial shows those, it's stale.

Client

```
#include <openssl/ssl.h>  
#include <openssl/err.h>
```

```

SSL_CTX *make_client_ctx(void) {
    SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());    // NOT TLSv1_2_method()
    if (!ctx) return NULL;

    SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION);

    // Verification: MUST be explicit. Default for clients is SSL_VERIFY_NONE.
    SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
    if (!SSL_CTX_set_default_verify_paths(ctx)) {        // system trust store
        SSL_CTX_free(ctx); return NULL;
    }
    return ctx;
}

int tls_connect(SSL_CTX *ctx, const char *host, const char *port, SSL **out) {
    int fd = tcp_connect(host, port);                    // section 3
    if (fd < 0) return -1;

    SSL *ssl = SSL_new(ctx);
    SSL_set_fd(ssl, fd);

    // SNI – required by most modern servers, and required for correct cert
    selection
    SSL_set_tlsext_host_name(ssl, host);

    // Hostname verification – THE critical call. Without it, any valid
    // certificate from any CA is accepted, and MITM is trivial.
    SSL_set_hostflags(ssl, X509_CHECK_FLAG_NO_PARTIAL_WILDCARDS);
    if (!SSL_set1_host(ssl, host)) { SSL_free(ssl); close(fd); return -1; }

    if (SSL_connect(ssl) != 1) {
        ERR_print_errors_fp(stderr);
        long v = SSL_get_verify_result(ssl);
        if (v != X509_V_OK)
            fprintf(stderr, "cert: %s\n", X509_verify_cert_error_string(v));
        SSL_free(ssl); close(fd); return -1;
    }
    *out = ssl;
    return fd;
}

```

Three lines do the security work: `SSL_CTX_set_verify(SSL_VERIFY_PEER)`, `SSL_CTX_set_default_verify_paths`, and `SSL_set1_host`. Omit any one and you have encryption without authentication — which is to say, no security against an active attacker.

Server

```

SSL_CTX *make_server_ctx(const char *cert_chain, const char *key) {
    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION);

```

```

// Use the FULL CHAIN file (leaf + intermediates), not just the leaf.
// Missing intermediates is the #1 TLS deployment bug – it works in your
// browser (which caches them) and fails for everyone else.
if (SSL_CTX_use_certificate_chain_file(ctx, cert_chain) != 1) goto err;
if (SSL_CTX_use_PrivateKey_file(ctx, key, SSL_FILETYPE_PEM) != 1) goto err;
if (SSL_CTX_check_private_key(ctx) != 1) goto err;

SSL_CTX_set_options(ctx, SSL_OP_NO_RENEGOTIATION |
SSL_OP_CIPHER_SERVER_PREFERENCE);
SSL_CTX_set_session_cache_mode(ctx, SSL_SESS_CACHE_SERVER);
return ctx;
err:
ERR_print_errors_fp(stderr);
SSL_CTX_free(ctx);
return NULL;
}

// Per connection, after accept4()
SSL *ssl = SSL_new(ctx);
SSL_set_fd(ssl, cfd);
if (SSL_accept(ssl) != 1) { /* handshake failed – log and close */ }

```

One `SSL_CTX` per configuration, one `SSL` per connection. `SSL_CTX` is reference-counted and thread-safe for concurrent `SSL_new`; a single `SSL` object must not be used from two threads at once.

Read/write and error handling

```

ssize_t n = SSL_read(ssl, buf, sizeof buf);
if (n <= 0) {
    int err = SSL_get_error(ssl, (int)n);
    return value
    switch (err) {
        case SSL_ERROR_WANT_READ:
            socket
            case SSL_ERROR_WANT_WRITE:
                read!
                break;
            case SSL_ERROR_ZERO_RETURN:
                // retry when ready
                close_conn(c); break;
            case SSL_ERROR_SYSCALL:
                // clean TLS close_notify
                // check errno; 0 =
                truncation
                case SSL_ERROR_SSL:
                    // protocol error
                    default:
                        ERR_print_errors_fp(stderr);
                        close_conn(c); break;
    }
}

```

`SSL_ERROR_WANT_WRITE` on a read is not a typo. TLS is a record protocol with its own state machine; a renegotiation or post-handshake message means a logical read may require writing. In an event loop this

means:

```
// The TLS event loop rule: register based on what TLS wants, not what you want.
if (err == SSL_ERROR_WANT_READ)  epoll_mod(ep, c, EPOLLIN);
if (err == SSL_ERROR_WANT_WRITE) epoll_mod(ep, c, EPOLLOUT);
// And when EPOLLOUT fires, you may need to retry SSL_read(), not SSL_write().
```

Two more non-obvious rules for non-blocking mode:

- **Retry with the same arguments.** `SSL_write` returning `WANT_WRITE` must be retried with the identical buffer and length, or you get an error. Set `SSL_MODE_ENABLE_PARTIAL_WRITE` and `SSL_MODE_ACCEPT_MOVING_WRITE_BUFFER` if you need to relax this.
- **SSL_read may return data with no socket readability.** A single TCP read can deliver several TLS records; OpenSSL buffers them. If you return to `epoll_wait` after one `SSL_read`, the buffered records sit there and you hang — this is the edge-triggered drain bug from section 7 in a new costume. Loop until `SSL_ERROR_WANT_READ`, or check `SSL_pending()`.

```
for (;;) {
    ssize_t n = SSL_read(ssl, buf, sizeof buf);
    if (n > 0) { process(buf, n); continue; }
    int err = SSL_get_error(ssl, (int)n);
    if (err == SSL_ERROR_WANT_READ || err == SSL_ERROR_WANT_WRITE) break;
    handle_close_or_error(err); break;
}
```

Shutdown

```
int r = SSL_shutdown(ssl);           // sends close_notify
if (r == 0) SSL_shutdown(ssl);       // returns 0 → call again to wait for peer's
SSL_free(ssl);                       // frees the SSL, not the fd
close(fd);
```

A missing `close_notify` is a **truncation attack** — an attacker who can inject a TCP FIN makes the connection look cleanly finished mid-response. If your protocol has its own length framing (section 4), you detect this anyway; if it relies on read-until-EOF, you must require `close_notify`.

Performance notes

- **Session resumption** (tickets in 1.3) skips the full handshake. On by default; make sure ticket keys are rotated and shared across your workers, or `SO_REUSEPORT` (section 9) will scatter clients across processes that can't resume each other's sessions.
- **kTLS** (`SSL_OP_ENABLE_KTLS`) moves record encryption into the kernel and restores `sendfile` zero-copy (section 8).
- Handshakes are CPU-expensive (asymmetric crypto); bulk transfer is cheap (AES-NI). Offload handshakes to the thread pool from section 9, not the event loop.

- **A handshake flood is a DoS vector** — a client can make you do expensive signature operations cheaply. Rate-limit new handshakes separately from connections.

3. Certificate Verification and Hostname Validation

This deserves its own section because **it is the single most commonly broken thing in TLS client code**.

The failure mode: you set `SSL_VERIFY_PEER`, the chain validates, you feel secure — but you never checked that the certificate is *for the host you're talking to*. An attacker with any valid certificate from any trusted CA (including one for a domain they legitimately own) can MITM you. The connection is encrypted, authenticated, and pointed at the wrong party.

This was systematically broken across the industry until roughly 2015. The paper "*The Most Dangerous Code in the World*" (Georgiev et al., 2012) found it in payment SDKs, Amazon's own libraries, and much more.

The fix, since OpenSSL 1.0.2:

```
SSL_set1_host(ssl, hostname);          // per-connection
// or
X509_VERIFY_PARAM *p = SSL_CTX_get0_param(ctx);
X509_VERIFY_PARAM_set1_host(p, hostname, 0);
X509_VERIFY_PARAM_set_hostflags(p, X509_CHECK_FLAG_NO_PARTIAL_WILDCARDS);
```

Do not write your own hostname matcher. The rules involve SAN vs CN precedence, wildcard placement (`*.example.com` matches `a.example.com` but not `a.b.example.com` and not `example.com`), IDN, embedded NULs in CN fields (the classic `www.bank.com\0.attacker.com` bypass), and IP address SANs. Every hand-rolled version has been wrong.

Never do this:

```
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);          // no.
int always_ok(int ok, X509_STORE_CTX *c) { return 1; } // NO.
```

If you're tempted because "it's just internal" or "the cert is self-signed," the correct answer is to add your CA to a private trust store:

```
SSL_CTX_load_verify_locations(ctx, "/etc/myapp/ca.pem", NULL);
```

Or pin the certificate/public key explicitly. Both preserve authentication; disabling verification does not.

SNI

Server Name Indication tells the server which hostname you want, so it can select the right certificate — necessary for virtual hosting.

```
SSL_set_tlsext_host_name(ssl, hostname);    // client: required in practice
```

Forgetting it means you get the server's default certificate, hostname validation fails, and the error looks confusing.

Server side, to select a certificate per name:

```
SSL_CTX_set_tlsext_servername_callback(ctx, sni_cb);
// in the callback: SSL_set_SSL_CTX(ssl, ctx_for_that_hostname);
```

SNi is sent in the clear in TLS 1.2 and 1.3 — it leaks which site you're visiting. Encrypted Client Hello (ECH) addresses this and is still deploying.

ALPN is negotiated the same way and is how HTTP/2 gets selected:

```
SSL_CTX_set_alpn_protos(ctx, (const unsigned char *)"\x02h2\x08http/1.1", 12);
```

4. Alternative Libraries

Library	Use when
OpenSSL 3.x	Default. Ubiquitous, complete, well-audited. Awkward API, large.
BoringSSL	Google's fork. Cleaner, faster, no stable ABI — vendor it. Used by Chrome, Envoy.
LibreSSL	OpenBSD's fork after Heartbleed. Simpler, safer defaults, smaller.
mbedtls	Embedded/IoT. Small footprint, clean API, easy to read. Fewer features.
GnuTLS	LGPL alternative. Some GNU projects require it.
rustls	If you can use Rust — memory-safe by construction, C API via rustls-ffi .
s2n-tls	AWS. Small, deliberately minimal attack surface.
wolfSSL	Embedded, commercial support, FIPS options.

mbedtls is genuinely pleasant to read if you want to understand what TLS is actually doing — it's the one to study, even if you ship OpenSSL.

Regardless of choice: keep it updated. Heartbleed, ROBOT, and dozens of CVEs mean a pinned old version is a liability. Track your dependency versions and patch promptly.

5. Common Vulnerabilities

Buffer overflows

The classic. In network code the source is almost always a length or count from the wire.

```
// VULNERABLE
char buf[256];
uint32_t len = get_u32(pkt);
memcpy(buf, pkt + 4, len);           // len is attacker-controlled

// CORRECT
if (len > sizeof buf) return -1;
if (4 + (size_t)len > pktlen) return -1; // and it must fit the received data
memcpy(buf, pkt + 4, len);
```

Two checks, not one: the length must fit your buffer **and** must not exceed what you actually received. Checking only the first reads past the end of the input.

Heartbleed was exactly this: OpenSSL's heartbeat handler trusted a client-supplied length and copied that many bytes out of a buffer, leaking adjacent memory — private keys included. One missing comparison, in the most-reviewed security library in the world.

Related: `strcpy`, `strcat`, `sprintf`, `gets` — never. `strncpy` doesn't NUL-terminate on truncation. Prefer explicit length tracking, `snprintf` with return-value checks, or `strncpy/strlcat` where available.

Integer overflow

```
// VULNERABLE: len + 1 wraps to 0, malloc(0) succeeds, memcpy writes 4 GB
uint32_t len = get_u32(pkt);
char *buf = malloc(len + 1);
memcpy(buf, data, len);

// VULNERABLE: count * size overflows
void *arr = malloc(count * sizeof(item));

// CORRECT
if (len >= MAX_LEN) return -1;           // bound FIRST
char *buf = malloc((size_t)len + 1);

// CORRECT: checked multiplication
size_t total;
if (__builtin_mul_overflow(count, sizeof(item), &total)) return -1;
void *arr = malloc(total);
```

Also watch **signed/unsigned confusion** — the bug that keeps appearing in parsers:

```
int len = get_i32(pkt);
if (len > MAX) return -1;           // negative passes!
memcpy(buf, pkt + 4, len);         // converted to huge size_t
```

Use unsigned types for lengths, and check `len > MAX` on the unsigned value.

Compile with `-ftrapv` or `-fsanitize=signed-integer-overflow` during development. Note that signed overflow is *undefined behavior* — the compiler may delete your check.

Format strings

```
printf(user_input);           // VULNERABLE: %n writes memory
syslog(LOG_INFO, user_input); // same
printf("%s", user_input);     // correct
```

`-Wformat` `-Wformat-security` catches these. Enable them as errors.

Unvalidated input

Beyond memory safety:

- **Path traversal** — `../../../../etc/passwd` in an HTTP request. Resolve with `realpath()` and verify the result is under your document root. Check *after* URL-decoding, and don't decode twice (`%252e%252e` → `%2e%2e` → `..`).
- **Injection** — user data in SQL, shell commands, log lines. Parameterized queries; never `system()` with user input; sanitize control characters and newlines before logging (log injection is real).
- **Header injection** — a `\r\n` in a value the attacker controls splits your HTTP response. Reject CR and LF in header values, always.
- **Encoding confusion** — overlong UTF-8, mixed encodings, NUL bytes inside strings. Validate and normalize once, at the boundary.

Resource exhaustion

All of section 9's material, restated as security:

Attack	Defense
Slowloris (slow headers)	Header-complete deadline, not <code>SO_RCVTIMEO</code>
Slow POST / slow read	Minimum-throughput requirement
Connection flood	<code>MAX_CONNS</code> , per-IP limits, reserved fd for <code>EMFILE</code>
Memory amplification	Cap header count/size, body size, decompressed size
Zip/decompression bomb	Cap the <i>output</i> size, not just the input
Hash collision DoS	Randomly seeded hash, or a tree-based map
Algorithmic complexity	No backtracking regex on user input (ReDoS)
TLS handshake flood	Rate-limit handshakes separately
SYN flood	<code>net.ipv4.tcp_syncookies=1</code> (section 4)
UDP amplification	Response ≤ request size, or require a token (section 5)

Two that surprise people: **ReDoS** — a regex like `(a+)+$` against `aaaa...`! takes exponential time; use RE2 or a non-backtracking engine for untrusted patterns or inputs. And **decompression bombs** — always bound the decompressed size, since a 1 KB gzip payload can expand to gigabytes.

6. Secure Parser Practices

Your protocol parser (section 10) is the frontline. These rules are cheap and they work.

1. Never trust a length field.

```
if (len > MAX_ALLOWED) return -1;           // your policy limit
if (len > remaining_bytes) return -1;       // physical reality of the buffer
```

Both, in that order, before any allocation or copy.

2. Bounds-check every read. A cursor abstraction makes this systematic instead of ad hoc:

```
typedef struct { const uint8_t *p; size_t len, off; } cursor;

static bool cur_need(cursor *c, size_t n) {
    return c->off + n >= c->off && c->off + n <= c->len;    // overflow-safe
}

static bool cur_u32(cursor *c, uint32_t *out) {
    if (!cur_need(c, 4)) return false;
    const uint8_t *p = c->p + c->off;
    *out = ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) |
           ((uint32_t)p[2] << 8) | (uint32_t)p[3];
    c->off += 4;
    return true;
}

static bool cur_bytes(cursor *c, size_t n, const uint8_t **out) {
    if (!cur_need(c, n)) return false;
    *out = c->p + c->off;
    c->off += n;
    return true;
}
```

Every field access goes through a function that cannot read out of bounds. This also solves the alignment and endianness problems from section 3 in the same stroke.

3. Fail closed. Reject anything malformed, ambiguous, or unexpected. Do not guess, do not repair, do not "be liberal in what you accept" — Postel's principle is how HTTP request smuggling (section 10) became possible.

4. Bound everything. Message size, field count, nesting depth, total connection lifetime, allocation per connection.

```
if (++depth > MAX_DEPTH) return -1;           // recursive parsers: mandatory
```

Or make the parser iterative with an explicit stack — no recursion means no stack exhaustion.

5. Constant-time comparison for secrets. `memcmp` returns early on the first differing byte, leaking length information through timing.

```
static int ct_eq(const void *a, const void *b, size_t n) {
    const uint8_t *x = a, *y = b;
    uint8_t d = 0;
    for (size_t i = 0; i < n; i++) d |= x[i] ^ y[i];
    return d == 0;
}
// or CRYPTO_memcmp() from OpenSSL
```

6. Initialize everything. `= {0}` on every struct. Uninitialized memory sent to a peer is an information leak — this is what the "Cloudbleed" and several kernel bugs were.

7. Zero secrets after use. The compiler will remove a plain `memset` as dead code:

```
OPENSSL_cleanse(key, sizeof key);           // or explicit_bzero() / memset_s()
```

8. Separate parsing from acting. Parse the entire message into a validated structure first, then act on it. Interleaving means a malformed message can leave you in a half-applied state.

9. Drop privileges. Bind port 443 as root, then immediately:

```
setgroups(0, NULL);
setgid(gid);                // GID BEFORE UID – after setuid you
can't
setuid(uid);
if (setuid(0) != -1) abort(); // verify it's irreversible
```

Combine with the fd passing from section 8 and the pre-fork model from section 9. Add `chroot`/namespaces and a `seccomp` filter if the threat model warrants it.

10. Build with every defense on.

```
gcc -O2 -Wall -Wextra -Werror \
    -Wformat -Wformat-security -Wconversion \
    -D_FORTIFY_SOURCE=3 \
    -fstack-protector-strong \
    -fstack-clash-protection \
    -fPIE -pie -Wl,-z,relro,-z,now,-z,noexecstack
```

And test with sanitizers: `-fsanitize=address,undefined` in CI, `-fsanitize=thread` for concurrent code.

11. Fuzz. Restating section 10's exercise 6 because it's the highest-value practice here:

```
// fuzz_target.c – build with: clang -g -fsanitize=fuzzer,address,undefined
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    http_req r = {0};
    parse_request(&r, data, size);
    free_request(&r);
    return 0;
}
```

An hour of libFuzzer or AFL++ against a hand-written parser finds bugs that code review does not. Add it to CI with a corpus.

12. Consider not using C for the parser. The realistic assessment: memory-safety bugs are roughly 70% of CVEs in large C/C++ codebases, and they cluster in exactly this code. Rust, or a parser generator with verified output, removes the entire class. If the surrounding system must be C, a parser in a safe language behind a C ABI is a legitimate architecture — Cloudflare, Firefox, and the Linux kernel have all done versions of this.

Pitfall Checklist

1. `SSL_VERIFY_NONE`, or a verify callback that returns 1 unconditionally
2. **Chain verification without hostname verification** — the most dangerous one
3. Writing your own hostname matcher
4. Forgetting SNI on the client
5. Serving the leaf certificate without intermediates
6. Treating `SSL_get_error` as `errno`, or ignoring `WANT_WRITE` on read
7. Not looping `SSL_read` until `WANT_READ` → buffered records stall the connection
8. Retrying `SSL_write` with different arguments after `WANT_WRITE`
9. Sharing one `SSL` object across threads
10. Ignoring missing `close_notify` when framing depends on EOF
11. 0-RTT data for non-idempotent requests
12. Pinned/outdated TLS library version
13. `memcpy` with an unvalidated length from the wire
14. Only checking the length against the buffer, not against bytes received
15. `malloc(len + 1)` without bounding `len` first
16. `count * size` without an overflow check
17. Signed length that passes a `> MAX` check while negative
18. `printf(user_input)`
19. Path traversal after single-decoding, or double-decoding
20. CR/LF accepted in HTTP header values
21. Recursive parser with no depth limit
22. `memcmp` on secrets → timing oracle
23. Uninitialized struct sent to a peer → memory disclosure
24. `memset` to clear a key (optimized away)

25. `setuid` before `setgid`

26. No fuzzing

Exercises

1. **Add TLS to your HTTP server** (section 10, exercise 1). Generate a local CA and a server certificate with `openssl req/x509`. Then integrate with the epoll loop — the `WANT_READ/WANT_WRITE` handling is the real work, and it will not behave as you expect on the first attempt.
2. **Break your own client.** Write a TLS client, verify it connects to a real site. Then set up `mitmproxy` with its own CA and confirm your client *rejects* it. Now remove `SSL_set1_host` and confirm the MITM succeeds while everything still looks encrypted. This is the single most instructive security exercise in this guide.
3. **Test with `testssl.sh`** against your server. Fix everything it flags. Then check the Qualys SSL Labs grade if it's internet-reachable.
4. **Write the vulnerability, then fix it.** Implement a length-prefixed parser with no bounds check. Send it a length of `0xFFFFFFFF`. Run under ASan and read the report. Add the check. Do the same for `malloc(len + 1)` overflow.
5. **Fuzz for an hour.** libFuzzer against your HTTP or WebSocket parser with ASan+UBSan. Keep the corpus. Add it to CI.
6. **Slowloris your own server.** 500 connections each sending one header byte every 10 seconds. If it dies, implement the header-completion deadline from section 9 and retry.
7. **Decompression bomb.** If your server handles `Content-Encoding: gzip`, feed it a 10 KB payload that expands to 10 GB. Add an output cap.
8. **Privilege separation.** Bind 443 as root, pass the listening fd to an unprivileged child via `SCM_RIGHTS` (section 8), drop privileges, verify with `ls -l /proc/<pid>/fd` and `id` that the child cannot regain root.

Exercise 2 first. Watching your own "secure" client accept a forged certificate is the thing that makes this section stick.

Ready for **Libraries & Frameworks** when you are.

11. Libraries & Frameworks

- Boost.Asio / standalone Asio (C++)
- libevent, libev, libuv
- POCO, cpp-httplib, Beast (Boost)
- ZeroMQ, nanomsg
- DPDK, netmap, XDP/eBPF for kernel-bypass and high-performance packet processing
- `std::net` proposals / C++ networking TS (status awareness)

Libraries & Frameworks

Sections 1–11 were about building things yourself, because you can't evaluate a library you don't understand. This section is about **not** building them.

The honest framing: every library here exists because the raw socket API is portable in theory and platform-specific in practice, and because event loops, buffer management, and TLS integration are the same 3,000 lines of subtle code in every project. Write them once as an exercise. Then use a library.

1. Asio / Boost.Asio (C++)

The dominant C++ networking library, and the direct ancestor of the standardization effort. Available as **standalone Asio** (header-only, no Boost dependency) or **Boost.Asio** — same author, near-identical API, pick standalone unless you already depend on Boost.

Asio presents a **proactor** interface (section 7) — completion-based — implemented over epoll/kqueue on Unix and IOCP on Windows. That's the whole design premise: one programming model across both worlds.

Callback style

```
#include <asio.hpp>
using asio::ip::tcp;

class session : public std::enable_shared_from_this<session> {
    tcp::socket socket_;
    std::array<char, 8192> buf_;
public:
    explicit session(tcp::socket s) : socket_(std::move(s)) {}
    void start() { read(); }
private:
    void read() {
        auto self = shared_from_this();           // keeps the object alive
        socket_.async_read_some(asio::buffer(buf_),
            [this, self](std::error_code ec, std::size_t n) {
                if (ec) return;                     // connection closed or error
                write(n);
            });
    }
    void write(std::size_t n) {
        auto self = shared_from_this();
        asio::async_write(socket_, asio::buffer(buf_, n),
            [this, self](std::error_code ec, std::size_t) {
                if (!ec) read();
            });
    }
};
```

The `shared_from_this` pattern is **not optional**. An async operation completes later; if the session was destroyed in the meantime, the handler touches freed memory. Capturing a `shared_ptr` in the lambda extends the lifetime exactly until the last pending operation completes. This is the deferred-close problem from section 7 and 9, solved by ownership rather than by a pending-close list.

Note `async_write` vs `async_write_some`: the former is a composed operation that loops until everything is sent — the `send_all` from section 4, done for you and non-blocking.

Coroutine style (C++20) — strongly preferred

```
asio::awaitable<void> echo(tcp::socket socket) {
    char data[8192];
    try {
        for (;;) {
            std::size_t n = co_await socket.async_read_some(
                asio::buffer(data), asio::use_awaitable);
            co_await asio::async_write(socket, asio::buffer(data, n),
                asio::use_awaitable);
        }
    } catch (const std::exception&) { /* connection closed */ }
}

asio::awaitable<void> listener(asio::io_context& ctx, unsigned short port) {
    tcp::acceptor acc(ctx, {tcp::v6(), port});
    for (;;) {
        tcp::socket s = co_await acc.async_accept(asio::use_awaitable);
        asio::co_spawn(ctx, echo(std::move(s)), asio::detached);
    }
}

int main() {
    asio::io_context ctx;
    asio::co_spawn(ctx, listener(ctx, 8080), asio::detached);
    ctx.run();
}
```

This is the payoff of the last decade of C++ evolution: **the code reads like the blocking version from section 4, runs like the event loop from section 7**. No state machine, no callback nesting, no manual lifetime juggling — the coroutine frame *is* the connection state.

Concepts worth knowing

- **io_context** — the event loop. `run()` blocks until there's no work left.
- **Strands** — serialize handlers without a mutex. Multiple threads call `ctx.run()`, but handlers on the same strand never run concurrently. This is the cleanest solution to the section 9 problem of "who owns this connection's state."

```
auto strand = asio::make_strand(ctx);
asio::co_spawn(strand, echo(std::move(sock)), asio::detached);
```

- **Multi-threading:** run `ctx.run()` from N threads (with strands), or one `io_context` per thread with `SO_REUSEPORT` (section 9's shared-nothing model).

- **Timers** — `asio::steady_timer` on `CLOCK_MONOTONIC`, integrated with the loop. Section 7's timer problem, solved.
- **TLS** — `asio::ssl::stream<tcp::socket>` wraps OpenSSL. **Set `set_verify_mode(ssl::verify_peer)` and `set_verify_callback(ssl::host_name_verification(host))`** — Asio does not do it for you, and section 11's most dangerous omission applies here exactly as it does to raw OpenSSL.

Costs: heavy template machinery, compile times, and error messages that are genuinely hostile. The learning curve is real. But it's the most complete, best-maintained C++ networking library there is, and the coroutine interface makes it worth the climb.

2. C Event Loops: libevent, libev, libuv

Three answers to "give me section 7's event loop, portably."

	libevent	libev	libuv
Size	Medium	Tiny	Large
Windows	Yes (select-based, weak)	No	Yes (IOCP)
Model	Reactor	Reactor	Reactor + thread pool
Extras	HTTP, DNS, buffers, TLS	Just the loop	File I/O, processes, threads, DNS
Thread safety	Locking available	Single-threaded design	One loop per thread
Notable users	Chromium (historically), Tor, Memcached	Various	Node.js , Julia, Neovim

libevent

The most feature-complete C option. `bufferevent` is the standout abstraction — it gives you the read/write buffering, backpressure watermarks, and partial-write handling from section 9 as a built-in:

```
#include <event2/event.h>
#include <event2/bufferevent.h>
#include <event2/listener.h>

static void read_cb(struct bufferevent *bev, void *ctx) {
    struct evbuffer *in  = bufferevent_get_input(bev);
    struct evbuffer *out = bufferevent_get_output(bev);
    evbuffer_add_buffer(out, in);                // zero-copy echo
}

static void event_cb(struct bufferevent *bev, short events, void *ctx) {
    if (events & (BEV_EVENT_EOF | BEV_EVENT_ERROR))
        bufferevent_free(bev);
}
```

```
static void accept_cb(struct evconnlistener *l, evutil_socket_t fd,
                    struct sockaddr *a, int slen, void *ctx) {
    struct event_base *base = evconnlistener_get_base(l);
    struct bufferevent *bev = bufferevent_socket_new(base, fd,
BEV_OPT_CLOSE_ON_FREE);
    bufferevent_setcb(bev, read_cb, NULL, event_cb, NULL);
    bufferevent_setwatermark(bev, EV_WRITE, 0, 4 << 20); // backpressure
    bufferevent_enable(bev, EV_READ | EV_WRITE);
}
```

`bufferevent_openssl_socket_new()` gives you TLS with the `WANT_READ/WANT_WRITE` dance from section 11 handled internally — a substantial saving.

The bundled `evhttp` is fine for admin endpoints and health checks; don't build a production HTTP service on it.

libev

Minimal and fast — the loop and nothing else. No buffering, no HTTP, no DNS, no Windows. If you want section 7's `epoll` loop with `kqueue` portability and zero opinion about anything else, this is it. Used inside `nginx`-adjacent projects and various embedded systems.

libuv

Extracted from Node.js, and the best choice if you need **Windows**. It's the only one of the three with a genuine IOCP backend rather than a `select`-based fallback, which means the proactor/reactor mismatch from section 7 is handled properly.

```
#include <uv.h>

static void on_read(uv_stream_t *s, ssize_t n, const uv_buf_t *b) {
    if (n < 0) { uv_close((uv_handle_t*)s, on_close); free(b->base); return; }
    uv_write_t *req = malloc(sizeof *req);
    uv_buf_t wb = uv_buf_init(b->base, (unsigned)n);
    uv_write(req, s, &wb, 1, on_write); // frees b->base in on_write
}

int main(void) {
    uv_loop_t *loop = uv_default_loop();
    uv_tcp_t server;
    uv_tcp_init(loop, &server);
    struct sockaddr_in addr;
    uv_ip4_addr("0.0.0.0", 8080, &addr);
    uv_tcp_bind(&server, (const struct sockaddr*)&addr, 0);
    uv_listen((uv_stream_t*)&server, 128, on_connection);
    return uv_run(loop, UV_RUN_DEFAULT);
}
```

libuv also gives you async **file I/O** (via a thread pool, since epoll can't do files usefully — section 7), child processes, and thread primitives. It's the closest thing to a portable async runtime in C. Cost: bigger, and buffer lifetime management across callbacks is easy to get wrong.

Choosing: Windows required → libuv. Want buffers and TLS handled → libevent. Want the smallest possible loop → libev. Writing C++ → Asio instead of any of these.

3. Higher-Level: POCO, cpp-httplib, Beast

cpp-httplib

Single header, blocking, dead simple. For internal tools, admin endpoints, embedded HTTP, and prototypes it's unbeatable on effort-to-result.

```
#include "httplib.h"
int main() {
    httplib::Server svr;
    svr.Get("/hi", [](const httplib::Request&, httplib::Response& res) {
        res.set_content("Hello", "text/plain");
    });
    svr.listen("0.0.0.0", 8080);
}
```

Thread-per-connection (section 4), so it won't handle 10k connections. That's fine — know what it's for.

`CPPHTTPLIB_OPENSSL_SUPPORT` enables HTTPS.

Boost.Beast

HTTP/1.1 and WebSocket built on Asio. Low-level by design — it gives you correct, well-tested message parsing and serialization, not a web framework. You still write the server. Pairs naturally with the Asio coroutine style above:

```
beast::http::request<beast::http::string_body> req;
co_await beast::http::async_read(stream, buffer, req, asio::use_awaitable);
```

This is the section-10 HTTP parser you'd otherwise write yourself, with the smuggling defenses and edge cases already handled. Strong argument for using it.

Beast's WebSocket implementation handles masking, fragmentation, and the close handshake correctly — it passes Autobahn.

POCO

A broad C++ application framework: networking, HTTP client/server, database access, XML/JSON, logging, threading. Java-ish design, blocking/threaded model. Good if you want batteries included and aren't chasing high concurrency.

Others worth knowing

- **Drogon, oatpp** — modern async C++ web frameworks with routing, ORM, and templates
- **libcurl** — the answer for HTTP *clients*. Handles HTTP/1.1, /2, /3, TLS, redirects, proxies, cookies, auth. Do not write an HTTP client; use this.
- **nghttp2** — HTTP/2 reference implementation (section 10)
- **c-ares** — asynchronous DNS, solving the `getaddrinfo`-blocks-your-event-loop problem from section 3

4. Messaging: ZeroMQ and nanomsg

A different abstraction level: **message patterns**, not connections.

ZeroMQ gives you sockets that carry discrete messages (framing solved — section 4), reconnect automatically, queue when the peer is down, and implement patterns:

Pattern	Use
REQ/REP	Request-response, load-balanced across workers
PUB/SUB	Broadcast with topic filtering
PUSH/PULL	Pipeline / work distribution
DEALER/ROUTER	Async request-response, the building block for the rest

```
void *ctx = zmq_ctx_new();
void *sock = zmq_socket(ctx, ZMQ_REP);
zmq_bind(sock, "tcp://*:5555");
for (;;) {
    zmq_msg_t msg;
    zmq_msg_init(&msg);
    zmq_msg_recv(&msg, sock, 0);           // complete message, always
    zmq_msg_close(&msg);
    zmq_send(sock, "world", 5, 0);
}
```

Transports are interchangeable via the URL: `tcp://`, `ipc://` (Unix sockets, section 8), `inproc://`, `pgm://` (multicast, section 5). Switching between them is a string change.

Where it fits: internal service meshes, distributed pipelines, worker fan-out. **Where it doesn't:** anything talking to browsers or third parties (it's not an interoperable wire protocol), and anything needing durability or delivery guarantees — ZeroMQ's queues are in-memory and messages are lost if a process dies. For that you want RabbitMQ, Kafka, or NATS.

nanomsg / NNG — a rewrite by ZeroMQ's original author addressing its design regrets. NNG is the maintained one; thread-safe sockets, cleaner API, better documented. Prefer NNG for new work unless you need ZeroMQ's ecosystem.

5. Kernel Bypass: DPDK, netmap, XDP/eBPF

This is section 7's C10M territory: when the kernel network stack itself is the bottleneck.

DPDK — poll-mode userspace drivers. You take the NIC away from the kernel entirely, busy-poll a queue from a pinned core, and process raw frames with huge pages and lockless rings. Achieves line-rate 100 Gbps packet processing.

Costs are severe: a dedicated CPU core spinning at 100% (no interrupts, no sleeping), the NIC is unavailable to the OS, you need a userspace TCP stack (mTCP, F-Stack, Seastar) for anything above L2, and none of sections 2–9 applies — no sockets, no epoll, no kernel TCP.

netmap — similar idea, lighter, in FreeBSD's tree. Keeps more kernel cooperation.

XDP / eBPF — the pragmatic modern option. Run a verified BPF program **in the kernel at the driver level**, before `sk_buff` allocation. Verdicts: `XDP_DROP`, `XDP_PASS` (continue to the normal stack), `XDP_TX` (bounce back out), `XDP_REDIRECT`.

```
SEC("xdp")
int xdp_filter(struct xdp_md *ctx) {
    void *data = (void *) (long) ctx->data;
    void *end = (void *) (long) ctx->data_end;
    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > end) return XDP_PASS; // the verifier REQUIRES this
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;
    struct iphdr *ip = (void *) (eth + 1);
    if ((void *) (ip + 1) > end) return XDP_PASS;
    if (is_blocked(ip->saddr)) return XDP_DROP; // dropped before the stack
    // sees it
    return XDP_PASS;
}
```

Note the bounds checks: the **verifier statically proves** you can't read out of bounds before loading the program. Section 11's parser discipline, enforced by the kernel.

XDP's advantage over DPDK is that it composes — you drop or redirect the traffic you care about and let everything else flow through the normal stack, so your existing epoll server keeps working. Cloudflare's DDoS mitigation, Facebook's Katran load balancer, and Cilium all work this way. `AF_XDP` sockets give you a zero-copy path into userspace for the packets you *do* want.

When any of this applies: load balancers, DDoS scrubbing, IDS/IPS, trading systems, telco packet processing.
When it doesn't: essentially every normal service. If you're asking whether you need kernel bypass, you don't.

6. C++ Networking Standardization

Short version: **there is no `std::net`, and none is imminent.**

The Networking TS (N4771) was based on Asio and reached a mature draft. It was **not merged into C++23** — the committee concluded that standardizing a synchronous/callback-based design before executors and

async models were settled would lock in the wrong abstraction. TLS support was also absent, which drew justified criticism: a standard networking library without a secure-by-default path is arguably worse than none.

Current state: executors (`std::execution`, P2300) landed in **C++26** as the async foundation. Networking would be built on top of that, so realistically C++29 at the earliest, and it may end up looking quite different from the old TS.

Practical consequence: use Asio. It's the reference design the standard will likely resemble, it's header-only, it's maintained, and if `std::net` ever arrives the migration will be shorter from Asio than from anywhere else.

Adjacent standard-library items that *did* land: `std::jthread` and `std::stop_token` (C++20), `std::atomic_ref`, `std::span` (excellent for the buffer/cursor patterns from section 11), and `std::expected` (C++23) for error handling without exceptions.

7. Choosing

C++, general purpose, any platform	→ Asio (coroutines) [+ Beast for HTTP]
C, need Windows	→ libuv
C, want buffers + TLS handled	→ libevent
C, want the minimum	→ libev
Quick internal HTTP server	→ cpp-httplib
HTTP client, ever	→ libcurl
HTTP/2 server	→ nghttp2
Async DNS	→ c-ares
Internal messaging patterns	→ NNG (or ZeroMQ)
Durable messaging	→ NATS / Kafka / RabbitMQ
Line-rate packet processing	→ XDP first, DPDK if that's not enough
Learning how any of it works	→ write it yourself (sections 1-11)

A note on the build-vs-use decision. Writing your own event loop is worth doing once, for the same reason writing a toy allocator is: you cannot reason about the thing you're using otherwise. Shipping your own is a different question. A library has had its `EPOLLOUT` arm/disarm logic, its TLS `WANT_WRITE` handling, its buffer lifetime rules, and its close sequencing exercised by thousands of users across platforms you'll never test on. That's not effort you can replicate.

Ship your own when you have a genuinely unusual requirement (a custom transport, an unusual threading model, a hard dependency constraint) or when the library's abstraction actively fights your design. Otherwise the interesting problems are in your protocol and your application, not in re-deriving section 7.

Pitfall Checklist

1. Asio async handler without `shared_from_this` → use-after-free
2. Asio TLS without `set_verify_mode` and hostname verification (section 11)
3. Mixing blocking calls into an async handler — the library can't save you from this
4. Multiple threads on one `io_context` without strands
5. libuv buffer lifetime — freeing in the wrong callback, or not at all
6. Assuming libevent's `evhttp` is production-grade for public traffic

7. Assuming ZeroMQ queues are durable — they're in-memory
 8. Using ZeroMQ as a public wire protocol
 9. Choosing nanomsg over NNG (nanomsg is superseded)
 10. Reaching for DPDK when XDP would do, or for either when neither is needed
 11. Waiting for `std::net`
 12. Adopting a library without reading how it handles backpressure — several don't
 13. Assuming a library's defaults are secure defaults
-

Exercises

1. **Port your echo server to Asio twice** — once with callbacks, once with C++20 coroutines. The line-count and readability difference is the argument for coroutines in one page.
2. **Port it to libevent using `bufferevent`**. Note that the watermark call replaces the entire backpressure implementation from section 9. Then check what it actually does when the watermark is hit — does it stop reading, or just notify you?
3. **Deliberately omit `shared_from_this`** in the Asio version, then close a connection mid-write. Run under ASan. This is the C++ equivalent of section 7's deferred-close bug.
4. **Benchmark honestly**. Your hand-written epoll server (section 7/9) vs Asio vs libevent vs libuv, same protocol, `wrk -c 1000 -d 60s`. Measure throughput, p99 latency, and RSS. Expect your version to be competitive on throughput and worse on tail latency and edge-case correctness — that difference is what the library bought you.
5. **Verify a library's TLS defaults**. Point an Asio or libevent TLS client at `badssl.com` variants (expired, self-signed, wrong-host). Anything that connects successfully is a configuration bug in your code, not in the library. Same experiment as section 11 exercise 2, on library-mediated code.
6. **ZeroMQ pipeline**. One PUSH producer, three PULL workers, one PULL sink. Kill a worker mid-run and observe what happens to its in-flight messages — that's the durability answer, demonstrated rather than described.
7. **XDP drop filter**. Attach a program that drops UDP traffic to one port. Measure packets-per-second dropped versus an iptables rule doing the same thing. The gap is why XDP exists.

Exercise 4 is the one that changes how you think — not because your server loses, but because of *where* it loses.

Ready for **Performance** when you are.

12. Performance

- Latency vs throughput, measuring both
- Syscall overhead reduction, batching
- Buffer sizing, memory pools, avoiding allocations in hot paths
- CPU affinity, NUMA awareness, cache-line alignment, false sharing
- Profiling: `perf`, `strace`, `ltrace`, flamegraphs
- Load testing: `wrk`, `ab`, `iperf3`, custom benchmark clients

13. Portability

- Winsock: `WSAStartup`, `SOCKET` type, `closesocket`, differences from POSIX
- Cross-platform abstraction layers
- IPv4/IPv6 agnostic code with `getaddrinfo`

Portability

Two axes of portability, and they're very different problems.

Platform portability (Unix ↔ Windows) is a genuine impedance mismatch: different types, different error reporting, different I/O models. It's tedious but mechanical, and the boundaries are well known.

Protocol portability (IPv4 ↔ IPv6) is easy to get right and easy to get wrong, and the wrong version usually works fine until it doesn't. It's mostly a matter of never writing `AF_INET` and never assuming an address is 4 bytes.

Notably, we skipped section 12 (Performance) in your list ordering — this is section 13. Happy to come back to Performance after.

1. Winsock

Windows sockets are BSD-derived, so the shape is familiar. The details differ enough that naive Unix code won't compile, and code that *does* compile can still be subtly wrong.

Initialization and cleanup

```
#ifdef _WIN32
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "Ws2_32.lib") // MSVC; MinGW: -lws2_32
#endif

int net_init(void) {
#ifdef _WIN32
    WSADATA wsa;
    int rc = WSAStartup(MAKEWORD(2, 2), &wsa); // 2.2 is the modern version
    if (rc != 0) return -1;                     // returns the error DIRECTLY,
    not via WSAGetLastError
#endif
    return 0;
}

void net_cleanup(void) {
#ifdef _WIN32
    WSACleanup();
#endif
}
```


`WSAStartup` is refcounted — each call needs a matching `WSACleanup`. Forgetting it means every socket call fails with `WSANOTINITIALISED`, which is the first thing to check when nothing works on Windows.

Header order matters. `windows.h` includes the ancient `winsock.h`, which conflicts with `winsock2.h`. Either include `winsock2.h` first, or define `WIN32_LEAN_AND_MEAN` before `windows.h`. This produces baffling redefinition errors otherwise.

The `SOCKET` type

```
// Unix: int, -1 on failure, valid values are small non-negative ints
int fd = socket(...);
if (fd < 0) { /* error */ }

// Windows: SOCKET is UINT_PTR (unsigned, 64-bit on x64)
SOCKET s = socket(...);
if (s == INVALID_SOCKET) { /* error */ } // INVALID_SOCKET == (SOCKET)(~0)
```

Three consequences:

- **`s < 0` is always false** — `SOCKET` is unsigned. Code that checks `fd < 0` silently treats every failure as success on Windows.
- **Storing a `SOCKET` in an `int` truncates** on 64-bit. It usually works because handles are small, and then one day it doesn't.
- Sockets are **not file descriptors** — you can't `read/write/close` them, and they don't participate in `select` on regular files.

Portable typedef:

```
#ifdef _WIN32
typedef SOCKET sock_t;
#define INVALID_SOCK INVALID_SOCKET
#define SOCK_ERR SOCKET_ERROR
#else
typedef int sock_t;
#define INVALID_SOCK (-1)
#define SOCK_ERR (-1)
#endif
```

API differences

Unix	Windows	Note
<code>close(fd)</code>	<code>closesocket(s)</code>	<code>close</code> won't work
<code>errno</code>	<code>WSAGetLastError()</code>	Thread-local on both
<code>EWOULDBLOCK</code>	<code>WSAEWOULDBLOCK</code>	All error names get a <code>WSA</code> prefix

Unix	Windows	Note
<code>strerror</code>	<code>FormatMessage</code>	Or a hand-written table
<code>fcntl(O_NONBLOCK)</code>	<code>ioctlsocket(FIONBIO)</code>	No <code>fcntl</code> at all
<code>readv/writev</code>	<code>WSARecv/WSASend</code> with <code>WSABUF</code>	Different struct layout
<code>sendmsg/recvmmsg</code>	<code>WSASendMsg/WSARecvMsg</code>	Function pointer via <code>WSAIoctl</code>
<code>poll</code>	<code>WSAPoll</code>	Vista+, and it has a known bug: doesn't report failed non-blocking <code>connect</code>
<code>epoll/kqueue</code>	IOCP	Fundamentally different model
<code>SIGPIPE</code>	Doesn't exist	One problem you don't have
<code>AF_UNIX</code>	Supported since Win10 1803	<code>SOCK_STREAM</code> only, path-based
<code>socketpair</code>	Not available	Emulate with a loopback TCP pair
<code>optval</code> is <code>void*</code>	<code>optval</code> is <code>char*</code>	Needs a cast
<code>socklen_t</code>	<code>int</code>	Typedef it

Error handling

```
static int sock_errno(void) {
#ifdef _WIN32
    return WSAGetLastError();
#else
    return errno;
#endif
}

#ifdef _WIN32
#define SOCK_EWOULDBLOCK WSAEWOULDBLOCK
#define SOCK_EINPROGRESS WSAEWOULDBLOCK // Windows uses WOULDLOCK for
connect
#define SOCK_EINTR WSAEINTR
#define SOCK_ECONNRESET WSAECONNRESET
#else
#define SOCK_EWOULDBLOCK EWOULDBLOCK
#define SOCK_EINPROGRESS EINPROGRESS
#define SOCK_EINTR EINTR
#define SOCK_ECONNRESET ECONNRESET
#endif
```

Note the `EINPROGRESS` mapping: a non-blocking `connect` on Unix gives `EINPROGRESS`, on Windows `WSAEWOULDBLOCK`. Code that checks only for `EINPROGRESS` treats it as a hard failure.

Windows does not set `errno` for socket operations. `perror()` after a failed `send()` prints whatever was in `errno` from some unrelated CRT call — misleading garbage.

Non-blocking

```
static int set_nonblocking(sock_t s) {
#ifdef _WIN32
    u_long mode = 1;
    return ioctlsocket(s, FIONBIO, &mode) == 0 ? 0 : -1;
#else
    int flags = fcntl(s, F_GETFL, 0);           // read-modify-write, section 6
    if (flags < 0) return -1;
    return fcntl(s, F_SETFL, flags | O_NONBLOCK);
#endif
}
```

There is **no way to query** non-blocking state on Windows. Track it yourself.

`SO_REUSEADDR` — the dangerous one

Restating section 6 because it's the most consequential semantic difference:

- **Unix:** allows binding while old connections are in `TIME_WAIT`. Two live listeners still can't share an address.
- **Windows:** allows **two sockets to actively bind the same address**. Another process — including a hostile one — can steal your port and intercept connections.

```
#ifdef _WIN32
    // Windows: this is what Unix SO_REUSEADDR actually means
    int yes = 1;
    setsockopt(s, SOL_SOCKET, SO_EXCLUSIVEADDRUSE, (char*)&yes, sizeof yes);
#else
    int yes = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
#endif
```

Windows doesn't need `SO_REUSEADDR` for the restart case anyway — it permits rebinding after `TIME_WAIT` by default. Blindly `#define`-ing your way across this is a real security bug.

Other Windows behaviors worth knowing

- **`select` ignores its first argument (`nfds`)**, and `FD_SETSIZE` is a soft limit you can raise by defining it before including the headers. It's still $O(n)$.
- **`WSAPoll` doesn't report connection failure** on a non-blocking `connect` — you have to also watch the exception set with `select`. This bug was reported in 2012 and won't be fixed.
- **Overlapped I/O + IOCP** is the only model that scales on Windows (section 7). Emulating `epoll` with `WSAPoll` works but caps out early.

- `shutdown()` constants differ: `SD_SEND/SD_RECEIVE/SD_BOTH` instead of `SHUT_WR/SHUT_RD/SHUT_RDWR`.
- `getaddrinfo` and `inet_pton/inet_ntop` **are** available (`ws2tcpip.h`, Vista+). Use them; the modern API is the portable API.

2. Abstraction Layers

The honest recommendation

Use a library. From section 12: `libuv` for C (the only one with a real IOCP backend), `Asio` for C++. Both were written specifically because this problem is larger than it looks — the readiness-vs-completion mismatch (section 7) doesn't reduce to `#defines`, and neither does Windows' lack of `fork`, `signalfd`, or `SCM_RIGHTS`.

Write your own only for a small, well-bounded surface: a client, a simple server, or a library that can't take dependencies.

If you do write one

Keep it thin and put every `#ifdef` in one place. The rule: **`#ifdef` in `net_compat.h` and `net_compat.c` only.** The moment platform conditionals leak into your protocol logic, every code path needs testing on every platform.

```
/* net_compat.h — the entire platform surface */
#ifdef NET_COMPAT_H
#define NET_COMPAT_H

#ifdef _WIN32
#ifdef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#include <winsock2.h>
#include <ws2tcpip.h>
typedef SOCKET sock_t;
typedef int socklen_t_compat;
#define INVALID_SOCKET INVALID_SOCKET
#define OPTVAL_CAST(p) ((char *)(p))
#else
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
typedef int sock_t;
typedef socklen_t socklen_t_compat;
#define INVALID_SOCKET (-1)
#define OPTVAL_CAST(p) ((void *)(p))
#endif
#endif
```

```
int    net_init(void);
void   net_cleanup(void);
int    sock_close(sock_t s);
int    sock_errno(void);
const char *sock_strerror(int err);
int    sock_set_nonblocking(sock_t s);
int    sock_set_cloexec(sock_t s);          // no-op on Windows
sock_t sock_connect(const char *host, const char *port); // IPv4/IPv6 agnostic
sock_t sock_listen(const char *port, int backlog);

#endif
```

```
/* net_compat.c - implementations */
int sock_close(sock_t s) {
#ifdef _WIN32
    return closesocket(s);
#else
    return close(s);
#endif
}

int sock_set_cloexec(sock_t s) {
#ifdef _WIN32
    // Windows handles aren't inherited unless explicitly marked; nothing to do.
    // (SetHandleInformation(HANDLE_FLAG_INHERIT, 0) if you need certainty)
    (void)s; return 0;
#else
    int f = fcntl(s, F_GETFD, 0);          // F_GETFD, not F_GETFL (section
6)
    return f < 0 ? -1 : fcntl(s, F_SETFD, f | FD_CLOEXEC);
#endif
}
```

What resists abstraction

Be realistic about the boundary. These do **not** wrap cleanly:

Feature	Problem
Event loop	Reactor (epoll/kqueue) vs proactor (IOCP) — section 7. Different buffer ownership, different code shape.
fork / pre-fork	No Windows equivalent. Threads only (section 9).
SIGPIPE	Doesn't exist on Windows — the <code>#ifdef</code> is trivial, but the <i>reasoning</i> differs
FD passing (SCM_RIGHTS)	Windows has <code>WSADuplicateSocket</code> , but it's a different mechanism

Feature	Problem
<code>signalfd</code> / self-pipe	Windows uses events and <code>WaitForMultipleObjects</code>
<code>sendfile</code>	<code>TransmitFile</code> on Windows — different signature and semantics
Unix domain sockets	Recent Windows only, <code>SOCK_STREAM</code> only, no <code>SO_PEERCRED</code>

For the event loop specifically: either accept `poll`-level performance everywhere, or write two backends behind a common interface — which is exactly what libuv did, and it's thousands of lines.

Portability across Unixes

Less dramatic but still real:

- `SO_REUSEPORT` — Linux 3.9+, FreeBSD/macOS have it with slightly different semantics. Guard with `#ifdef SO_REUSEPORT`.
- `MSG_NOSIGNAL` — Linux/FreeBSD. macOS uses the per-socket `SO_NOSIGPIPE` instead (section 4).
- `accept4`, `epoll`, `signalfd`, `eventfd`, `timerfd`, `splice`, `io_uring` — Linux only.
- `kqueue` — BSD/macOS only.
- **TCP keepalive tuning** — `TCP_KEEPIDLE` on Linux, `TCP_KEEPALIVE` on macOS (section 6).
- **IPv6_V6ONLY default** — 0 on Linux, 1 on FreeBSD/Windows/OpenBSD. Always set it explicitly.
- **`accept()` flag inheritance** — BSD inherits `O_NONBLOCK`, Linux doesn't.
- **EAGAIN vs EWOULDBLOCK** — same value on Linux, not guaranteed by POSIX. Check both.

```
#ifndef MSG_NOSIGNAL
#define MSG_NOSIGNAL 0
#endif
// ... and at socket creation, for macOS:
#ifdef SO_NOSIGPIPE
    int one = 1;
    setsockopt(s, SOL_SOCKET, SO_NOSIGPIPE, &one, sizeof one);
#endif
```

Feature-detect, don't platform-detect. `#ifdef SO_REUSEPORT` survives a new OS release; `#if defined(__linux__) && KERNEL >= ...` does not. CMake's `check_symbol_exists` or Autoconf's `AC_CHECK_FUNCS` do this at configure time.

3. IPv4/IPv6-Agnostic Code

This one is genuinely easy — the modern API was designed for it. The failures come from habit, not difficulty.

The rules

1. **Never write `AF_INET`.** Use `AF_UNSPEC` in hints for clients, or `AF_INET6` with `IPV6_V6ONLY=0` for dual-stack servers.

2. **Never declare `struct sockaddr_in`** for anything that receives an address. Use `sockaddr_storage` (section 3) — `sockaddr_in` silently truncates IPv6 and bare `sockaddr` is only 16 bytes.
3. **Never `inet_addr/inet_ntoa/gethostbyname`**. They are IPv4-only, and `inet_ntoa` returns a static buffer.
4. **Never size a text buffer at 16.** `INET6_ADDRSTRLEN` is 46.
5. **Always iterate `ai_next`**. DNS returns multiple addresses; some are unreachable.
6. **Never store an address as a 32-bit integer** or a fixed 4-byte array. Configuration, ACLs, logs, and databases all need to hold 128-bit addresses.
7. **Never parse `host:port` by splitting on the last colon** — `::1:8080` is ambiguous. IPv6 literals in URLs use brackets: `[::1]:8080`.

The core pattern (from section 3, restated as the portability rule it is):

```

sock_t sock_connect(const char *host, const char *port) {
    struct addrinfo hints = {0}, *res, *rp;
    hints.ai_family    = AF_UNSPEC;           // ← the whole point
    hints.ai_socktype  = SOCK_STREAM;
#ifdef AI_ADDRCONFIG
    hints.ai_flags     |= AI_ADDRCONFIG;      // skip families this host can't
use
#endif

    int rc = getaddrinfo(host, port, &hints, &res);
    if (rc != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rc));
        return INVALID_SOCKET;
    }

    sock_t s = INVALID_SOCKET;
    for (rp = res; rp; rp = rp->ai_next) {    // ← try every candidate
        s = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
        if (s == INVALID_SOCKET) continue;
        if (connect(s, rp->ai_addr, (socklen_t_compat)rp->ai_addrlen) == 0) break;
        sock_close(s);
        s = INVALID_SOCKET;
    }
    freeaddrinfo(res);
    return s;
}

```

There is no IPv4 or IPv6 anywhere in that function. That's the target.

Server: one socket or two

Section 6 covered the trade-off. Restated as a decision:

One dual-stack socket — simpler, one accept loop, but you get IPv4-mapped addresses (`::ffff:192.0.2.1`) which complicate logging and ACLs, and it doesn't work on OpenBSD (which forces `IPV6_V6ONLY=1`).

Two sockets — portable everywhere, clean address handling, but two listeners to multiplex. **The IPv6 socket must set `IPV6_V6ONLY=1`** or the IPv4 bind fails with `EADDRINUSE`.

For maximum portability, do both and fall back:

```
sock_t sock_listen_dual(const char *port, sock_t *v4_out) {
    // Try dual-stack first
    sock_t s6 = try_listen(AF_INET6, port, /*v6only=*/0);
    if (s6 != INVALID_SOCKET) { *v4_out = INVALID_SOCKET; return s6; }

    // Fall back to two sockets (OpenBSD, or bindv6only=1 systems)
    s6 = try_listen(AF_INET6, port, /*v6only=*/1);
    *v4_out = try_listen(AF_INET, port, 0);
    return s6;
}
```

Handling mapped addresses

Critical for ACLs — an allowlist entry for `192.0.2.0/24` will not match `::ffff:192.0.2.5`. This is a firewall-bypass bug, and it's easy to miss because it only shows up on dual-stack sockets.

```
void normalize_addr(struct sockaddr_storage *ss, socklen_t_compat *len) {
    if (ss->ss_family != AF_INET6) return;
    struct sockaddr_in6 *s6 = (struct sockaddr_in6 *)ss;
    if (!IN6_IS_ADDR_V4MAPPED(&s6->sin6_addr)) return;

    struct sockaddr_in s4 = {0};
    s4.sin_family = AF_INET;
    s4.sin_port = s6->sin6_port;
    memcpy(&s4.sin_addr, ((const uint8_t *)&s6->sin6_addr) + 12, 4);
    memcpy(ss, &s4, sizeof s4);
    *len = sizeof s4;
}
```

Normalize once, at the boundary, before anything makes a policy decision — the same principle as section 11's "validate and normalize at the edge."

Formatting and parsing

```
void addr_to_string(const struct sockaddr *sa, socklen_t_compat len,
                  char *out, size_t cap) {
    char host[NI_MAXHOST], serv[NI_MAXSERV];
    if (getnameinfo(sa, len, host, sizeof host, serv, sizeof serv,
                    NI_NUMERICHOST | NI_NUMERICSERV) != 0) {
        snprintf(out, cap, "<unknown>");
        return;
    }
    if (sa->sa_family == AF_INET6)
```



```

        snprintf(out, cap, "[%s]:%s", host, serv);        // brackets for v6
    literals
    else
        snprintf(out, cap, "%s:%s", host, serv);
}

```

`NI_NUMERICHOST` | `NI_NUMERICSERV` is the safe default (section 3) — without them you trigger reverse DNS, which is slow and attacker-influenced.

Parsing the reverse: check for a leading `[`, find the matching `]`, take the port after it. Otherwise split on the *only* colon — and if there's more than one, it's a bare IPv6 literal with no port.

The Happy Eyeballs problem

The naive sequential loop above is correct but can be slow: if DNS returns an IPv6 address first and IPv6 is broken on the path, you wait for a full TCP timeout before trying IPv4. Users experience this as "the site takes 30 seconds to load."

Happy Eyeballs (RFC 8305) is the fix: start the IPv6 connection, and if it hasn't completed within ~250 ms, start the IPv4 one in parallel. Take whichever succeeds first, cancel the other. This needs the non-blocking `connect` + `poll` + `SO_ERROR` pattern from section 3, plus a timer.

Browsers and `libcurl` do this. Your internal service client probably doesn't need it. A public-facing client does.

Testing

You cannot verify IPv6 support by reading code. Concretely:

```

# Both must work with zero code changes
./client 127.0.0.1 8080
./client ::1 8080
./client localhost 8080

ss -tlnp | grep 8080          # confirm what you actually bound

```

Test on an IPv6-only network if you can — mobile carriers in the US and much of Asia are IPv6-only with NAT64, so IPv4-literal assumptions genuinely break for real users.

Pitfall Checklist

Windows

1. Missing `WSAStartup` → `WSANOTINITIALISED` on every call
2. `if (s < 0)` on an unsigned `SOCKET` → failures read as success
3. Storing a `SOCKET` in an `int` → truncation on 64-bit
4. `close()` instead of `closesocket()`
5. `errno/perror` instead of `WSAGetLastError`

6. Checking only `EINPROGRESS` for non-blocking connect
7. `#define SO_EXCLUSIVEADDRUSE SO_REUSEADDR` or vice versa → port hijacking
8. `windows.h` before `winsock2.h` → redefinition errors
9. Missing the `char*` cast on `setsockopt` optval
10. Trusting `WSAPoll` to report connect failures
11. Expecting `socketpair`, `fork`, or `SCM_RIGHTS`

Cross-platform 12. `#ifdef` scattered through protocol logic instead of confined to one file 13. Platform detection instead of feature detection 14. Assuming `EAGAIN == EWOULDBLOCK` 15. Assuming `accept()` inherits `O_NONBLOCK` (BSD yes, Linux no) 16. `MSG_NOSIGNAL` without the `SO_NOSIGPIPE` fallback for macOS 17. Assuming an event-loop abstraction over `epoll` and `IOCP` is cheap

IPv4/IPv6 18. `AF_INET` hardcoded anywhere 19. `sockaddr_in` where `sockaddr_storage` belongs → IPv6 truncation 20. `inet_addr` / `inet_ntoa` / `gethostbyname` 21. `char buf[16]` for an address string 22. Ignoring `ai_next` → fails on dual-stack hosts 23. Storing addresses as `uint32_t` in config, logs, or a database 24. Splitting `host:port` on the last colon 25. IPv6 literals in URLs without brackets 26. ACLs that don't normalize IPv4-mapped addresses → policy bypass 27. Not setting `IPV6_V6ONLY` explicitly → different behavior per platform 28. Two listeners without `IPV6_V6ONLY=1` on the v6 socket → `EADDRINUSE` 29. Claiming IPv6 support without testing on `::1`

Exercises

1. **Build `net_compat.{h,c}`** with the functions listed above. Then port your echo client and server from section 3 to use it exclusively. Grep for `#ifdef` in your protocol code afterward — there should be zero.
2. **Actually build on Windows.** MinGW or MSVC, or cross-compile with `x86_64-w64-mingw32-gcc`. Reading about Winsock differences teaches you nothing; the compiler errors teach you everything. Budget an hour for header ordering alone.
3. **Prove the `SOCKET` sign bug.** On Windows, make `socket()` fail (invalid family) and check `if (s < 0)`. Watch it not fire. Then check `INVALID_SOCKET` and watch it work.
4. **IPv6 with no code changes.** Your section-3 client must connect to `127.0.0.1, ::1`, and `localhost` with the same binary and no flags. Then run it against a v6-only server and confirm it still works.
5. **Find your IPv4 assumptions.** Grep an existing codebase (yours or an open-source one) for `AF_INET`, `sockaddr_in`, `inet_ntoa`, `INADDR_`, and `[16]`. Every hit is a portability bug or a latent one.
6. **Break an ACL.** Write a dual-stack server with an allowlist for `127.0.0.1`. Connect over IPv6 loopback and observe the address arrives as `::ffff:127.0.0.1` and fails the check — or worse, if your check is a prefix match, *passes* something it shouldn't. Add normalization.
7. **Happy Eyeballs.** Add a 250 ms IPv6-first race to your connect function. Test by blackholing IPv6 with `ip6tables -A OUTPUT -j DROP` and confirming the connection still completes quickly.

Exercise 2 is the one people skip and shouldn't — Windows portability is entirely a matter of having compiled it once.

We skipped **Performance** (section 12) to follow your list order. Want to circle back to that, or go on to **Tools & Debugging** (14)?

14. Tools & Debugging

- `netstat`, `ss`, `lsof`, `ip`, `ifconfig`, `route`
- `ping`, `traceroute`, `dig`, `nslookup`
- `telnet`, `netcat`, `socat`, `curl`
- Wireshark / `tcpdump` filters
- `iptables`/`nftables` basics
- `gdb`, Valgrind, ASan/TSan/UBSan for network code
- `tc` for simulating latency/loss

Tools & Debugging

Everything in sections 1–13 was about writing code. This is about seeing what it actually does — which, in networking, is frequently not what you thought.

The diagnostic order that works: **is it listening** → **is the packet arriving** → **is the protocol correct** → **is my code correct**. Most wasted debugging time comes from starting at step four.

1. Connection and Socket State

`ss` — use this, not `netstat`

`netstat` is deprecated and reads `/proc` line by line; `ss` uses the netlink API and is dramatically faster with thousands of sockets.

```
ss -tlnp          # TCP listening, numeric, with process
ss -tanp          # all TCP connections
ss -unp           # UDP
ss -tan state time-wait # filter by state
ss -tan '( dport = :443 or sport = :443 )'
ss -tin           # ← internal TCP info: rtt, cwnd, retrans, bytes
ss -tm           # socket memory usage
ss -s            # summary counts by state
```

`ss -tin` is the one to remember. It surfaces the `TCP_INFO` struct from section 6:

```
ESTAB 0 0 10.0.0.5:8080 10.0.0.9:51234
      cubic wscale:7,7 rto:204 rtt:3.5/1.75 mss:1448 cwnd:10
      bytes_sent:52301 bytes_acked:52301 segs_out:41 retrans:0/2 send 33.1Mbps
```

`rtt`, `cwnd`, and `retrans` tell you whether you're looking at a network problem or a code problem, without touching a packet capture.

Diagnostic patterns:

Symptom in ss	Meaning
Recv-Q growing on your socket	Your app isn't calling <code>recv()</code> fast enough
Send-Q growing	Peer isn't reading, or the network is stalled (section 9 backpressure)
Many TIME-WAIT	You're closing first; consider connection reuse (section 1)
Many CLOSE-WAIT	You received FIN and never called <code>close()</code> — an fd leak in your code
Many SYN-RECV	SYN flood, or your accept queue is full
retrans climbing	Real packet loss

CLOSE-WAIT accumulation is almost always your bug, not the network's. It means the peer closed and your code never noticed.

lsof

```
lsof -i :8080          # who's using port 8080
lsof -i TCP -sTCP:ESTABLISHED # established TCP connections
lsof -p <pid>          # everything a process has open
lsof -p <pid> | wc -l   # fd count — for chasing EMFILE (section 9)
```

For fd leaks specifically, `ls -l /proc/<pid>/fd` is faster and shows exactly what each descriptor points to — including sockets inherited across `fork/exec` when you forgot `FD_CLOEXEC` (section 6).

ip — not ifconfig/route

The `net-tools` suite (`ifconfig`, `route`, `netstat`, `arp`) is unmaintained and doesn't show modern features (multiple addresses per interface, policy routing, IPv6 properly).

```
ip addr show          # interfaces and addresses
ip -6 addr            # IPv6 only
ip route get 8.8.8.8  # ← which route and source IP would be used
ip neigh              # ARP/NDP cache (section 1)
ip link set dev eth0 mtu 1400 # for reproducing PMTU issues
ip -s link            # interface counters: errors, drops
```

`ip route get <dest>` answers "which interface and source address will this connection use," which is the fastest way to diagnose multi-homed problems — including the UDP source-address issue that `IP_PKTINFO` solves (section 8).

Kernel counters

```

ss -s                                # socket summary
netstat -su                          # ← UDP: "packet receive errors" = buffer
overflow
netstat -st                          # TCP: retransmits, resets, listen queue
overflows
nstat -az | grep -i drop
cat /proc/net/softnet_stat           # per-CPU packet drops

```

`netstat -su` is the check for section 5's silent UDP loss. If "receive errors" is climbing, your receive loop is too slow or `SO_RCVBUF` is too small — not the network.

2. Reachability and DNS

```

ping -c 4 host                       # ICMP echo; may be firewalled – absence proves
nothing
ping -M do -s 1472 host              # ← PMTU probe: DF set, 1500-byte total (section
5)
traceroute host                      # UDP-based by default
traceroute -T -p 443 host            # TCP-based; gets through more firewalls
mtr host                             # continuous traceroute + loss stats – best of
the three

```

`ping -M do -s <size>` is the practical PMTU black hole test. Binary-search the size until packets stop getting through; that's your path MTU.

DNS:

```

dig example.com A
dig example.com AAAA                # ← check this too, before blaming your v6 code
dig @8.8.8.8 example.com            # specific resolver
dig +trace example.com              # full delegation chain from the root
dig +short example.com
dig -x 1.1.1.1                     # reverse

```

`nslookup` is legacy; `dig` gives you the full message and is what you want when debugging a resolver you wrote (section 10). `dig +trace` shows you exactly which nameserver is returning stale data.

3. Interacting With Services

`curl` — the HTTP workhorse

```

curl -v https://example.com          # headers + TLS details
curl -i -X POST -d '{"a":1}' -H 'Content-Type: application/json' url
curl --http1.1 / --http2 / --http3 url # force a version

```

```
curl -v --resolve example.com:443:1.2.3.4 https://example.com # bypass DNS
curl -w '@curl-format.txt' -o /dev/null -s url # detailed timing breakdown
curl --trace-ascii - url # full wire dump
```

The timing breakdown is underused and excellent for splitting "slow DNS" from "slow TLS handshake" from "slow server":

```
time_namelookup, time_connect, time_appconnect, time_starttransfer, time_total
```

`time_connect - time_namelookup` is the TCP handshake; `time_appconnect - time_connect` is TLS; `time_starttransfer - time_appconnect` is the server thinking.

nc / socat

```
nc -l 8080 # listen (BSD/OpenBSD nc; GNU nc uses -l -p 8080)
nc host 8080 # connect
nc -u host 9000 # UDP
nc -z -v host 20-25 # port scan
printf 'GET / HTTP/1.0\r\n\r\n' | nc host 80
```

socat is nc with everything. It's the tool for testing protocol edge cases:

```
# TLS client with cert verification – test your TLS server (section 11)
socat - OPENSSL:host:443,verify=1

# TLS-terminating proxy in front of a plaintext server
socat OPENSSL-LISTEN:443,cert=server.pem,fork TCP:localhost:8080

# Unix socket ↔ TCP bridge (section 8)
socat UNIX-CONNECT:/tmp/app.sock TCP-LISTEN:8080,fork

# Send exact bytes, no newline translation – for binary protocols
printf '\x00\x00\x00\x05hello' | socat - TCP:localhost:8080

# Bidirectional tap: proxy and dump everything to stderr
socat -v TCP-LISTEN:8080,fork TCP:realserver:80
```

That last one is a poor man's MITM proxy and it's superb for debugging your own protocol — you see exactly what each side sent, in order, without a packet capture.

telnet

Still the fastest way to hand-drive a text protocol:

```
telnet localhost 8080
GET / HTTP/1.1
Host: localhost
```

Useful for exactly the reason section 10 gave: text protocols are debuggable by hand. Note it does newline translation, so it's unsuitable for binary protocols — use `socat` or `nc` there.

4. Packet Capture

tcpdump

```
tcpdump -i any -nn port 8080          # -nn: no name resolution (fast)
tcpdump -i eth0 -nn -X 'tcp port 80'  # hex + ASCII payload
tcpdump -i any -nn -w cap.pcap 'host 1.2.3.4' # write for Wireshark
tcpdump -r cap.pcap -nn               # read back
tcpdump -i any -nn -s0 'port 8080'    # full packets (default is already
full)
```

Filters worth knowing (BPF syntax — the same filters `libpcap` compiles, section 8):

```
'tcp port 80 and host 1.2.3.4'
'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0' # SYNs only
'tcp[tcpflags] & tcp-rst != 0'                                   # RSTs — who's
resetting?
'icmp[icmptype] == 3 and icmp[icmpcode] == 4'                   # PMTU needed
(section 5)
'tcp and (((ip[2:2] - ((ip[0]&0xf)<<2)) - ((tcp[12]&0xf0)>>2)) != 0)' # data
packets only
'udp and port 53'                                               # DNS
'net 239.0.0.0/8'                                              # multicast
(section 5)
```

The RST filter is the highest-value one. "The connection dropped" almost always means someone sent an RST — a firewall, a load balancer, or your own code calling `close()` with unread data (section 4). The capture tells you which direction it came from, which usually identifies the culprit immediately.

Handshake anatomy in a capture — this is what you should be able to read at a glance:

```
IP A.51234 > B.8080: Flags [S], seq 1000
IP B.8080 > A.51234: Flags [S.], seq 5000, ack 1001
IP A.51234 > B.8080: Flags [.], ack 5001
...
IP A.51234 > B.8080: Flags [F.], seq 1500, ack 5001
```

[S] SYN, [S.] SYN-ACK, [.] ACK, [P.] PSH-ACK (data), [F.] FIN, [R] RST.

Wireshark

For anything beyond a few packets. Key features:

- **Follow TCP Stream** (right-click) — reassembles the byte stream, which is the fastest way to see your protocol as the application sees it
- **tcp.analysis.flags** as a display filter — surfaces retransmissions, dup ACKs, zero-window, out-of-order
- **tcp.analysis.zero_window** — directly shows the receiver's window closing, i.e. section 9's backpressure working (or your app not reading)
- **Statistics** → **Conversations** — top talkers, byte counts
- **Statistics** → **TCP Stream Graphs** → **Time Sequence (tcptrace)** — visualizes the sliding window and retransmissions from section 1
- **Expert Information** — auto-flagged anomalies
- **Decode As...** — tell it your custom protocol runs on port 9999

Display filters differ from capture filters: `tcp.port == 8080 && http.request.method == "POST"`.

Decrypting your own TLS: set `SSLKEYLOGFILE` and point Wireshark at it (Preferences → Protocols → TLS → Pre-Master-Secret log file):

```
SSLKEYLOGFILE=/tmp/keys.log curl https://example.com
```

`curl`, browsers, and OpenSSL-based apps support this. In your own code, `SSL_CTX_set_keylog_callback()`. This is the only sane way to debug an application protocol running under TLS.

Capture practicalities

```
tcpdump -i lo          # loopback traffic is NOT on eth0
tcpdump -i any         # all interfaces – but note DLT_LINUX_SLL (16-
byte                  # header, not 14) – section 8's datalink lesson
```

Capture on loopback when testing locally, or you'll see nothing and conclude your code isn't sending. This trips up almost everyone once.

Give `tcpdump` permission without root:

```
sudo setcap cap_net_raw,cap_net_admin=eip $(which tcpdump)
```


5. Firewalls

Firewall rules are the most common cause of "my server works locally but not remotely."

```
# nftables (modern)
sudo nft list ruleset
sudo nft add rule inet filter input tcp dport 8080 accept

# iptables (legacy but ubiquitous)
sudo iptables -L -n -v          # -v shows packet counters – check if rules are
HIT
sudo iptables -A INPUT -p tcp --dport 8080 -j ACCEPT
```

The **-v** counters are the diagnostic: if a DROP rule's counter is incrementing while you test, you've found your problem.

Simulating failures — genuinely useful for testing sections 4, 6, and 9:

```
# Blackhole a peer: no RST, no ICMP, just silence.
# This is how you test keepalive and TCP_USER_TIMEOUT detection.
sudo iptables -A INPUT -s 10.0.0.5 -j DROP

# REJECT instead: peer gets an immediate RST → ECONNREFUSED
sudo iptables -A INPUT -s 10.0.0.5 -j REJECT

# Block ICMP to create a PMTU black hole (section 5)
sudo iptables -A INPUT -p icmp --icmp-type fragmentation-needed -j DROP

sudo iptables -F          # flush everything
```

DROP vs **REJECT** is the distinction between "peer vanished" and "peer refused" — exactly the two failure modes from section 4's disconnect table, and you can produce either on demand.

Also check: cloud security groups, **firewalld**, Docker's iptables rules, and SELinux (**ausearch -m avc**) — all of which can block traffic that your local rules permit.

6. Debugging Your Code

gdb

```
gdb --args ./server 8080
gdb -p <pid>          # attach to a running process
gdb ./server core      # post-mortem
```

```
(gdb) set follow-fork-mode child      # for fork-per-client servers (section 4)
(gdb) set detach-on-fork off
(gdb) info threads
(gdb) thread apply all bt            # ← all stacks; the go-to for a hung server
(gdb) catch syscall accept
(gdb) p *conn
(gdb) p/x buf[0]@64                 # dump 64 bytes in hex
(gdb) handle SIGPIPE nostop noprint # if you haven't SIG_IGN'd it
```

thread apply all bt on a hung process identifies deadlocks and blocking-in-the-event-loop instantly. Enable core dumps first:

```
ulimit -c unlimited
cat /proc/sys/kernel/core_pattern
```

strace / ltrace

Often faster than gdb for network code, because the syscall trace *is* the behavior:

```
strace -f -e trace=network -p <pid>      # -f follows threads/forks
strace -c ./server                       # ← syscall counts + time. Find the hot
path.
strace -T -e trace=epoll_wait,recvfrom,sendto ./server  # -T shows time per call
strace -f -e trace=network -s 200 ./client             # -s: show more of
buffers
```

strace -c is the quickest way to confirm section 8's claims — run your **writetv** version and your two-**send** version and compare the counts.

ltrace -l libssl.so ./client traces library calls, useful for OpenSSL state machine debugging.

Sanitizers — the highest-value tools here

```
# ASan: buffer overflows, use-after-free, leaks
gcc -g -O1 -fsanitize=address -fno-omit-frame-pointer server.c

# UBSan: integer overflow, misaligned access, UB
gcc -g -fsanitize=undefined -fno-sanitize-recover=all server.c

# Both together (recommended default for development)
gcc -g -O1 -fsanitize=address,undefined -fno-omit-frame-pointer server.c

# TSan: data races. CANNOT combine with ASan.
gcc -g -O1 -fsanitize=thread server.c
```

Runtime options:

```
ASAN_OPTIONS=detect_leaks=1:abort_on_error=1:strict_string_checks=1 ./server
UBSAN_OPTIONS=print_stacktrace=1 ./server
TSAN_OPTIONS=halt_on_error=1 ./server
```

Why these matter more in network code than anywhere else: every bug from section 11 — the unvalidated length, the integer overflow, the use-after-free from freeing a connection inside the dispatch loop (section 7) — is caught by ASan the first time it executes. Code review does not find these reliably; sanitizers do, deterministically.

TSan is the one people skip and shouldn't if you have a thread pool (section 9). Data races are non-deterministic and can pass every test for months. TSan finds them on the first execution of the racing path. Run your load test under TSan at least once.

Valgrind

```
valgrind --leak-check=full --track-origins=yes ./server
valgrind --tool=helgrind ./server      # race detection
valgrind --tool=callgrind ./server     # profiling
```

Slower than ASan (10–50× vs 2×) but needs no recompilation and catches uninitialized-memory reads that ASan misses — relevant because section 11 flagged uninitialized memory sent to a peer as an information leak. Memcheck is the best tool for that specific bug.

Rule of thumb: ASan+UBSan for everyday development, TSan for concurrency, Valgrind Memcheck for uninitialized reads, and all of them in CI.

Load and correctness tools

```
wrk -t4 -c1000 -d60s --latency http://localhost:8080/ # HTTP; reports p99
ab -n 100000 -c 100 http://localhost:8080/           # simpler, less accurate
h2load -n 10000 -c 100 https://localhost:8443/        # HTTP/2 (nghttp2)
iperf3 -s / iperf3 -c host                             # raw throughput
iperf3 -c host -u -b 100M                             # UDP with target bitrate
```

For fuzzing (sections 10 and 11), [AFL++](#) or libFuzzer against your parser.

7. Network Impairment with `tc`

The tool that turns "works on my machine" into "works." Everything in sections 5, 9, and 10 assumed real-network conditions you can't get on loopback.

```
# Add impairments to loopback
sudo tc qdisc add dev lo root netem delay 100ms 20ms distribution normal
sudo tc qdisc add dev lo root netem loss 5%
sudo tc qdisc add dev lo root netem duplicate 1%
sudo tc qdisc add dev lo root netem reorder 25% 50% delay 10ms
sudo tc qdisc add dev lo root netem corrupt 0.1%

# Combined — a realistic bad mobile link
sudo tc qdisc add dev lo root netem delay 150ms 40ms loss 2% reorder 5%

# Bandwidth limiting
sudo tc qdisc add dev lo root tbf rate 1mbit burst 32kbit latency 400ms

# Modify / inspect / remove
sudo tc qdisc change dev lo root netem loss 10%
sudo tc qdisc show dev lo
sudo tc qdisc del dev lo root
```

Note `tc` affects egress only, so on loopback both directions are impaired (packets traverse `lo` twice) — the delay you set effectively doubles the RTT. On a real interface you're only shaping outbound.

For finer control, filter by port so you don't impair SSH into the box:

```
sudo tc qdisc add dev eth0 root handle 1: prio
sudo tc qdisc add dev eth0 parent 1:3 handle 30: netem delay 200ms loss 10%
sudo tc filter add dev eth0 protocol ip parent 1:0 prio 3 u32 \
    match ip dport 8080 0xffff flowid 1:3
```

What to test with it:

Impairment	Reveals
loss 20%	Your UDP reliability layer (section 5) — or its absence
delay 200ms	Stop-and-wait throughput ceiling; connection-reuse value
reorder	Sequence-number handling; modular comparison bugs
duplicate	Missing duplicate detection (section 5)
rate 1mbit	Backpressure and write buffering (section 9)
Delay + a large transfer	BDP and buffer sizing (section 6)

Alternatives: `toxiproxy` (userspace, scriptable, no root, good in CI), Linux network namespaces for isolated multi-host topologies on one machine:

```
sudo ip netns add test
sudo ip netns exec test ./server
```

8. A Diagnostic Playbook

"Connection refused" `ss -tlnp | grep <port>` — is anything listening? On the right address (`0.0.0.0` vs `127.0.0.1`)? Then firewall counters, then `ip route get <dest>`.

"Connection times out" Different from refused: no RST came back. Firewall **DROP**, wrong route, or a host that's down. `traceroute -T -p <port>` and `tcpdump` on both ends — if SYNs leave but nothing returns, it's between you.

"Connection drops randomly" `tcpdump 'tcp[tcpflags] & tcp-rst != 0'` on both ends. Whoever sent the RST is the answer. Common causes: idle timeout on a middlebox/NAT (section 1), `close()` with unread data (section 4), or a load balancer's connection limit.

"Works locally, fails over the network" Suspect MTU first: `ping -M do -s 1472`. Handshake succeeds but large transfers hang = PMTU black hole. Second suspect: you were relying on loopback's coalescing behavior and your framing is broken (section 4).

"Slow, but no errors" `ss -tin` for `rtt` and `retrans`. Then check for the 40 ms Nagle/delayed-ACK stall (section 6) — if your latency is suspiciously close to a multiple of 40 ms, that's it. Then `strace -c` for syscall counts.

"Server hangs" `gdb -p <pid>` then `thread apply all bt`. Blocked in `recv`? Deadlock on a mutex? Stuck in `getaddrinfo` on the event loop thread (section 3)?

"Memory grows without bound" `ss -tm` for socket buffers, then ASan leak check, then check your write buffers — an unbounded tx queue with a slow client is the section 9 failure.

"Runs out of file descriptors" `ls -l /proc/<pid>/fd | wc -l`, then look for **CLOSE-WAIT** in `ss -tan`. That state means you're not closing what the peer already closed.

"UDP packets disappearing" `netstat -su` for receive errors → your buffer or your loop is too slow. Then check datagram size against MTU (section 5).

Pitfall Checklist

1. Capturing on `eth0` when testing over loopback — see nothing, blame the code
2. Forgetting `-nn` on `tcpdump` — DNS lookups make it unusably slow and add noise
3. Using `netstat/ifconfig/route` instead of `ss/ip`
4. Assuming `ping` failure means unreachable — ICMP is frequently blocked
5. Ignoring **CLOSE-WAIT** accumulation (it's your fd leak, not the network's)
6. Not checking `iptables -L -n -v counters`, only the rules
7. Testing only on loopback, where loss and reordering never occur
8. Forgetting `tc` on loopback doubles the delay
9. Leaving a `tc` rule in place and debugging the wrong problem for an hour
10. Combining ASan and TSan (mutually exclusive)
11. Never running TSan on threaded code
12. `strace` without `-f` on a forking or threaded server

13. Not setting `SSLKEYLOGFILE` and trying to debug an encrypted protocol from packets
 14. `telnet` for binary protocols (newline translation corrupts data)
 15. Testing without `ulimit -n` raised, then blaming your code for `EMFILE`
-

Exercises

1. **Read a handshake.** `tcpdump -i lo -nn port 8080`, then run your section-3 client and server. Identify SYN, SYN-ACK, ACK, data, FIN, and the TIME-WAIT that follows. Then kill the client with `SIGKILL` mid-connection and find the RST.
2. **Reproduce the 40 ms stall visually.** Run section 6's exercise 2 under `tcpdump` and watch the delayed ACK arrive 40 ms after the data. Enable `TCP_NODELAY` and watch it vanish. Seeing it in a capture is different from seeing it in a timer.
3. **Diagnose a broken server blind.** Have someone (or a script) break your server in one of five ways — bind to `127.0.0.1` only, add a DROP rule, set `backlog` to 0, introduce a `CLOSE-WAIT` leak, or set an MTU of 1000 on the interface. Diagnose each using only `ss`, `tcpdump`, and `iptables -v`.
4. **tc your UDP protocol.** Run section 5's reliable file transfer under `netem loss 20% reorder 10% duplicate 2%` and verify a byte-exact checksum. Then remove your duplicate detection and watch it corrupt.
5. **Find a real bug with sanitizers.** Take your section-10 HTTP parser, build with `-fsanitize=address,undefined`, and throw malformed input at it. If it finds nothing, your bounds checks are good — then deliberately remove one and confirm ASan catches it with a precise stack trace.
6. **TSan your thread pool.** Load-test the section-9 server under TSan. Look particularly for races on connection state between the loop thread and workers. If it's clean, add a deliberate race (touch `conn->state` from a worker) and confirm TSan catches it.
7. **socat tap.** Put `socat -v TCP-LISTEN:9090,fork TCP:localhost:8080` between your client and server and watch your own protocol on the wire. For a binary protocol, add `,cool-write` and pipe to `xxd`.
8. **Decrypt your own TLS.** `SSLKEYLOGFILE` + Wireshark against your section-11 server. Confirm you can read the plaintext HTTP inside the TLS records.

Exercise 1 first. Fifteen minutes reading your own handshake in `tcpdump` makes every abstract thing in section 1 concrete, permanently.

We still have **Performance** (12) outstanding, and **Suggested Project Progression** (15) remains. Which next?

15. Suggested Project Progression

1. TCP echo server + client
2. Multi-client chat server (threads → `select` → `epoll`)
3. Length-prefixed binary protocol with a custom message format
4. File transfer utility over TCP
5. Reliable UDP layer (ACK + retransmit + sequencing)

6. Minimal HTTP server serving static files
7. WebSocket server
8. DNS resolver client
9. HTTP proxy / reverse proxy
10. TLS-enabled server with OpenSSL
11. Port scanner or packet sniffer with raw sockets/libpcap
12. High-performance event-loop server benchmarked against nginx